APPLYING MACHINE LEARNING TOOLS FOR REAL-WORLD DDOS DETECTION

A Thesis

Presented to

The Faculty of the Department of Computer Science

California State University, Los Angeles

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Computer Science

By

Mary Semerdjian

May 2024

© 2024

Mary Semerdjian

ALL RIGHTS RESERVED

The thesis of Mary Semerdjian is approved.

Dr. Zilong Ye, Committee Chair

Dr. JungSoo Lim

Professor Edmund Gean

Dr. Eun-Young Elaine Kang, Department Chair

California State University, Los Angeles

May 2024

ABSTRACT

Applying Machine Learning Tools for Real-World DDoS Detection

By

Mary Semerdjian

In this study, we aim to develop anomaly-based Intrusion Detection Systems (IDS) that leverage Machine Learning algorithms and tools, including Neural Networks, to detect Distributed Denial of Service (DDoS) attack traffic and differentiate it from normal traffic. Using state-of-the-art machine learning models will improve accuracy, reduce the false positive rate against the Distributed Denial of Service (DDoS) attack traffic, and facilitate real-time threat detection. In addition, we will use DDoS datasets such as APA-DDoS and CIC-DDoS2019 for training, testing, confusion matrix, validation, and evaluation, involving specific metrics to assess experimental results and plot the AUC (Area Under the Curve) of an ROC (Receiver Operating Characteristic) curve.

ACKNOWLEDGMENTS

First and foremost, I would love to thank my father, Minas, who inspired me to pursue education, value education, and follow my dreams wherever the road may lead. I love to thank my mother, Ruzanna, sister, Louiza, and grandmother, Sona, for their deepest sincerity, motivation, guidance, help, and gratitude for those who have supported me throughout my entire education – yes, they are my rock. I would also love to thank my aunt and her husband, who admired, loved, and helped me in any feasible way. I would also love to thank my friends and close relatives. I want to thank the faculty and staff of the Department of Computer Science at California State University, Los Angeles, for all their support and for giving me this opportunity to do a Master's Thesis. I would also like to thank Dr. Zilong Ye for his help, advice, guidance, and tranquility. In addition, I would like to thank Ms. Valentina Ovasapyan, Administrative Support Coordinator, who has guided me since I was a student at California State University, Los Angeles, following my Bachelor years first, then my Master's. Thank Dr. Eun-Young Elaine Kang, the Department Chair, who approved my Master's Thesis. I thank Dr. Mohammad Pourhomayoun for teaching me about Data Science, Machine Learning, and Neural Networks. I want to thank Dr. Jungsoo Lim, whose words inspired me to do my Master's Thesis, such as, "Mary, you will be proud of your accomplishment at the end of your Master's Thesis," and her being my Bachelor's and Master's professor. I want to thank Professor Edmund Gean, who taught me about cybersecurity and network security. I want to thank Dr. Jie Zhong, who taught me about the Theory of Probability and applied those techniques to my Master's Thesis. I also want to thank Professor Cydney Auman for getting me more interested and more profound into finding the bugs in the code.

Thank Cal State LA ECST-NSS High-Performance Computing Center for allowing me to use the Curie cluster computer and Dr. Curtis Wang. Thank the Annex Lab – Library for keeping their doors open until late at night so I could get as much work done as possible.

In addition, I would like to thank my mentors Sergey and Hrant, who persuaded me to continue my mathematical-computer science journey and told me about the importance of mathematics and "to always find the smallest mistake in the graph."

I want to thank my Computer Science Professor from Los Angeles City College, Professor Brian Dauzat, who had always encouraged me to major in Computer Science and pursue higher education, and Dr. Danielle Muller, who taught me what research and writing, reading, and exploring those options, and Dr. Jonathan Ruyle from East Los Angeles College, who taught about Discrete Mathematics and the theory behind the math and why mathematical proofs are so necessary to apply to.

I would also like to thank the Canadian Institute for Cybersecurity, an educational institute in Fredericton, New Brunswick, Canada. It is part of the University of New Brunswick (UNB) and Kaggle for their cybersecurity datasets.

TABLE OF CONTENTS

Abstract iv

Acknowledgments v

List of Tables ix

List of Figures xi

Chapter

1. Introduction 1

Intrusion Detection Systems 7

Problem Statement 8

Findings and Goal 9

2. Machine Learning Algorithms 10

Decision Tree 10

CatBoost 12

Random Forest 14

Support Vector Machine 16

K-Nearest Neighbor 19

Convolutional Neural Network 20

Recurrent Neural Network 23

Long Short-Term Memory 25

3. Implementation Details 27

Hardware and Software Specifications 27

Exploratory Data Analysis 29

Data Preprocessing 39

Data Cleaning.....40

Feature Engineering.....40

Feature Extraction.....41

Train Test Split.....42

Evaluation Metrics.....43

4. CIC-DDoS2019 Dataset and their Results.....48

Results.....61

5. APA-DDoS Dataset and their Results.....105

Results.....107

6. Conclusion and Future Work.....132

References.....134

LIST OF TABLES

Table

1. DDoS Attacks on Training Day and Testing Day 50
2. CIC-DDoS2019 Dataset: Feature Number, Feature Names, Data Types, and Description.....54
3. Decision Tree Classifier and their Hyperparameters without Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) in CIC-DDoS2019 Dataset.....62
4. Decision Tree Classifier Technique 1 without PCA and SMOTE 63
5. Decision Tree Classifier Technique 2 with PCA and without SMOTE 64
6. Decision Tree Classifier Technique 3 with PCA and SMOTE.....64
7. CatBoost Classifier and their Hyperparameters with Principal Component Analysis (PCA).....74
8. CatBoost Classifier with Principal Component Analysis (PCA) in CIC-DDoS2019 Dataset.....75
9. Support Vector Machine Classifier and their Hyperparameters with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) 79
10. Support Vector Machine Classifier with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE).....79
11. K-Nearest Neighbor Classifier and their Hyperparameters with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) 82

12. K-Nearest Neighbor Classifier with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) 83
13. Random Forest Classifier and their Hyperparameters with Principal Component Analysis (PCA) 86
14. Random Forest Classifier with Principal Component Analysis (PCA).....86
15. APA-DDoS Dataset Feature Number, Feature Names, and Description of the Features.....106
16. Decision Tree Classifier and their Hyperparameters with Principal Component Analysis (PCA) in APA-DDoS Dataset.....108
17. Decision Tree Classifier with Principal Component Analysis (PCA).....108
18. CatBoost Classifier with Principal Component Analysis (PCA) in APA-DDoS Dataset.....111
19. Principal Component Analysis (PCA) and Synthetic Minority Oversampling (SMOTE) with Support Vector Machine in APA-DDoS Dataset.....114
20. Hyperparameters with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) in K-Nearest Neighbor Classifier with APA-DDoS Dataset.....117
21. Principal Component Analysis (PCA) and Synthetic Minority Oversampling (SMOTE) with K-Nearest Neighbor Accuracy and Cross-Validation Mean Score and ROC-AUC Score with APA-DDoS Dataset.....118

LIST OF FIGURES

Figure

1. Distributed Denial of Service (DDoS) Attack 6
2. Cybersecurity Tree Structure Example 11
3. First and Second Trees 13
4. N-th tree 13
5. Random Forest 15
6. Big-O Complexity Chart 17
7. Training Time Complexities for Machine Learning Models 17
8. Support Vector Machine Hyperplane 18
9. K-Nearest Neighbor Before and After 20
10. Convolutional Neural Networks (CNN) Architecture 22
11. Recurrent Neural Network 24
12. Unfolded Recurrent Network 24
13. Long Short-Term Memory Contains Four Interacting Layers 26
14. Displays the First Five Rows of the APA-DDoS Dataset 30
15. Displays the Label Column (the Last Column) of our APA-DDoS Dataset 30
16. Displays the First Five Rows of the CIC-DDoS2019 Dataset 31
17. Pearson Correlation Matrix of the APA-DDoS Dataset 32
18. SNS Count Plot 33
19. Pie Chart 34
20. Scatter Matrix of the APA-DDoS Dataset 35
21. UDP-Lag Count 36

22. UDP-Lag Bar Plot.....37

23. UDP-Lag Scatter Plot.....37

24. LDAP Pie Chart.....38

25. Correlation Matrix of the Training CIC-DDoS2019 Dataset.....39

26. Confusion Matrix.....44

27. True Positive Rate VS False Positive Rate at Different Classification Thresholds…..……46

28. Measuring Performance in True Positive Rate and False Positive Rate an AUC (Area Under the Curve) of an ROC (Receiver Operating Characteristic) Curve.....47

29. DDoS Attacks.....49

30. Decision Tree Classifier Technique 1 with AUC-ROC Curve without PCA and SMOTE.....65

31. Decision Tree Classifier Technique 2 with AUC-ROC Curve with PCA and without SMOTE.....66

32. Decision Tree Classifier Technique 3 with AUC-ROC Curve with PCA and with SMOTE.....66

33. Decision Tree Classifier Technique 1 with Confusion Matrix without PCA and SMOTE.....67

34. Decision Tree Classifier Technique 2 with Confusion Matrix with PCA and without SMOTE.....68

35. Decision Tree Classifier Technique 3 with Confusion Matrix with PCA and with SMOTE.....69

36. Decision Tree Classifier Technique 1 with Performance Metrics and F1-Score without PCA and SMOTE. 70
37. Decision Tree Classifier Technique 2 with Performance Metrics and F1-Score with PCA and without SMOTE 70
38. Decision Tree Classifier Technique 3 with Performance Metrics and F1-Score with PCA and SMOTE 71
39. Decision Tree Classifier Technique 1 with Performance Metrics and False Positive Rate without PCA and SMOTE. 71
40. Decision Tree Classifier Technique 2 with Performance Metrics and False Positive Rate with PCA and without SMOTE. 72
41. Decision Tree Classifier Technique 3 with Performance Metrics and False Positive Rate with PCA and SMOTE. 72
42. ROC-AUC Curve of a Multi-Class Classification in CatBoost. 76
43. CatBoost Classifier with Confusion Matrix. 77
44. CatBoost Classifier with Performance Metrics and F1-Score... 77
45. CatBoost Classifier with Performance Metrics and False Positive Rate. 78
46. ROC-AUC Curve of a Multi-Class Classification in Support Vector Machine. ...80
47. Support Vector Machine Classifier with Confusion Matrix. 80
48. Support Vector Machine Classifier with Performance Metrics and F1-Score. 81
49. Support Vector Machine Classifier with Performance Metrics and False Positive Rate. 81
50. ROC-AUC Curve of a Multi-Class Classification in K-Nearest Neighbor. 83
51. K-Nearest Neighbor Classifier with Confusion Matrix. 84

52. K-Nearest Neighbor Classifier with Performance Metrics and F1-Score. 84

53. K-Nearest Neighbor Classifier with Performance Metrics and False Positive Rate... 85

54. Random Forest Classifier with AUC-ROC Curve and with PCA. 87

55. Random Forest Classifier with Confusion Matrix. 87

56. Random Forest Classifier with Performance Metrics and F1-Score. 88

57. Random Forest Classifier with Performance Metrics and False Positive Rate. 88

58. Model Summary of the Convolutional Neural Network (CNN). 89

59. Convolutional Neural Network (CNN) Model. 91

60. Convolutional Neural Network (CNN) Model Accuracy and Loss.....92

61. ROC-AUC Curve of a Multi-Class Classification in Convolutional Neural Network (CNN). 93

62. Convolutional Neural Network (CNN) with Confusion Matrix. 93

63. Convolutional Neural Network with Performance Metrics and F1-Score.....94

64. Convolutional Neural Network with Performance Metrics and False Positive Rate in CIC-DDoS2019 Dataset... 94

65. Model Summary of the Recurrent Neural Network (RNN).....95

66. Recurrent Neural Network (RNN) Model. 96

67. Recurrent Neural Network (RNN) Model Accuracy and Loss.....96

68. ROC-AUC Curve of a Recurrent Neural Network (RNN). 97

69. Recurrent Neural Network (RNN) of a Confusion Matrix. 98

70. Recurrent Neural Network Classification Report 98

71. Recurrent Neural Network with Performance Metrics and F1-Score.....99

72. Recurrent Neural Network with Performance Metrics and False Positive Rate...99

73. Model Summary of the Long Short-Term Memory (LSTM). 100

74. Long Short-Term Memory (LSTM) Model. 101

75. Long Short-Term Memory (LSTM) Model Accuracy and Loss.....102

76. Long Short-Term Memory (LSTM) of a Confusion Matrix.....102

77. ROC-AUC Curve of a Multi-Class Classification in Long Short-Term Memory.....103

78. Long Short-Term Memory with Performance Metrics and F1-score. 103

79. Long Short-Term Memory with Performance Metrics and False Positive Rate in CIC-DDoS2019 Dataset.....104

80. ROC-AUC Curve of a Multi-Class Classification Decision Tree 109

81. Decision Tree of a Confusion Matrix in APA-DDoS Dataset.....109

82. Decision Tree with Performance Metrics and F1-Score in APA-DDoS Dataset.....110

83. Decision Tree with Performance Metrics and False Positive Rate in APA-DDoS Dataset.....110

84. ROC-AUC Curve of a Multi-Class Classification CatBoost.....112

85. Confusion Matrix of a Multi-Class Classification CatBoost 112

86. Performance Metrics and F1-Score with CatBoost. 113

87. Performance Metrics and False Positive Rate with CatBoost 113

88. APA-DDoS Dataset Multi-Class Classification ROC-AUC Curve in Support Vector Machine.....115

89. APA-DDoS Dataset Multi-Class Classification Confusion Matrix in SVM.....115

90. Support Vector Machine Classifier with Performance Metrics and F1-Score in APA-DDoS Dataset 116

91. Support Vector Machine Classifier with Performance Metrics and False Positive Rate in APA-DDoS Dataset 116

92. APA-DDoS Dataset ROC-AUC Curve in K-Nearest Neighbor. 118

93. APA-DDoS Dataset Confusion Matrix in K-Nearest Neighbor. 119

94. K-Nearest Neighbor with Performance Metrics and F1-Score in APA-DDoS Dataset 119

95. K-Nearest Neighbor with Performance Metrics and False Positive Rate in APA-DDoS Dataset 120

96. Model Summary of Convolutional Neural Network (CNN) in the APA-DDoS Dataset 121

97. Model Accuracy and Loss of Convolutional Neural Network (CNN) in the APA-DDoS Dataset 122

98. ROC-AUC Curve of Convolutional Neural Network (CNN) in the APA-DDoS Dataset 122

99. Multi-Class Classification Confusion Matrix of Convolutional Neural Network (CNN) in the APA-DDoS Dataset 122

100. Convolutional Neural Networks with Performance Metrics and F1-Score in APA-DDoS Dataset 123

101. Convolutional Neural Networks with Performance Metrics and False Positive Rate in APA-DDoS Dataset 124

102. Model Summary of Recurrent Neural Network (RNN) in the APA-DDoS Dataset.....125
103. Model Accuracy and Loss of Recurrent Neural Network (RNN) in the APA-DDoS Dataset.....125
104. ROC-AUC Curve of Recurrent Neural Network (RNN) in the APA-DDoS Dataset.....126
105. Confusion Matrix of Recurrent Neural Network (CNN) in the APA-DDoS Dataset.....126
106. Recurrent Neural Networks with Performance Metrics and F1-Score in APA-DDoS Dataset.....127
107. Recurrent Neural Networks with Performance Metrics and False Positive Rate in APA-DDoS Dataset 127
108. Model Summary of Long Short-Term Memory (LSTM) in the APA-DDoS Dataset.....128
109. Long Short-Term Memory Model Accuracy and Loss in APA-DDoS Dataset.....129
110. ROC-AUC Score Long Short-Term Memory in APA-DDoS Dataset 129
111. Confusion Matrix Long Short-Term Memory in APA-DDoS Dataset.....130
112. Long Short-Term Memory with Performance Metrics and F-1Score in APA-DDoS Dataset.....130
113. Long Short-Term Memory with Performance Metrics and False Positive Rate in APA-DDoS Dataset 131

CHAPTER 1

Introduction

Distributed Denial of Service (DDoS) is a malicious cyber-attack designed to shut down a network, service, or website by overwhelming the target server with a massive flood of internet traffic that consumes a large amount of bandwidth, originating from a distributed group of computers, also known as botnets, to attack a single target [1], [2].

In a DDoS attack, botnets cannot be formed or built alone. For botnets to be built, bot-herders will build botnets on request to carry out a specific task as a Distributed Denial of Service attack [3]. Once this happens, botnets will continue, whereas a botnet serves as a tool and a group of robots to perform automated tasks. In automated, one can refer to millions of tasks being performed because one such simple cause is caused by being performed automatically. In addition, a botnet, orchestrated by a malicious actor, is utilized to execute a DDoS attack. The malicious actor instigates the attack as indicated, causing distress.

Over the past twenty years, Distributed Denial of Service has become exploitative. Distributed Denial of Service (DDoS) attacks first occurred in the mid-1990s' and have threatened internet security ever since those years, and till now, the threats have increased. A DDoS attack is a common computer-network security threat and a cyber threat to the real world and real life. The number of network-layer DDoS attacks in 2023 Q4 increased by 175% year over year and 25% quarter over quarter [4].

As the number of DDoS attacks increases, the crimes will not stop. Note that DDoS attacks are illegal and a crime, especially in the United States of America. One can face cruel and unusual punishments and jail time as a felony or misdemeanor. As per se

with the law and according to the Federal Computer Fraud and Abuse Act, an unauthorized DDoS attack can lead to up to 10 years in prison and a $500,000 fine, and conspiring to do so can lead to 5 years and $250,000 [5].

The attacks were called Denial of Service (DOS) attacks in the early years before shifting to Distributed Denial of Service (DDoS) attacks, which means that this is one single attack instead of a distributed one. Distributed Denial of Service (DDoS) is a subsystem of Denial of Service (DoS). In addition to DoS, where DoS attacks are launched from a single malicious client, DDoS attacks use a distributed group of computers to attack a single target. DoS has the same meaning; for example, the motive behind a DDoS attack relies solely on the distribution of the zombies, such as millions of zombies sending millions of packets one million times—an instance of singular vs plural attacks. In singular, we usually refer to one Distributed Denial of Service (DDoS) attack; in the plural, we refer to multiple, two, or more Distributed Denial of Service (DDoS) attacks.

DDoS attacks aim to make your website and servers unavailable to legitimate users [6]. The malicious actor brings your servers and websites down. For example, if one works in a company, a DDoS attack can bring down the company's website and servers, causing the company stress, financial situations, job loss, loss of employees, and much more than the expected average. One way to ensure that companies prevent such DDoS attacks is that the company may need to use firewalls to prevent DDoS attacks from happening today, tomorrow, in the past, and in the future. Once a DDoS attack starts, it usually takes hours or days to get the website back to normal—malicious actors demanding pay cause it. The same applies to political campaigns, universities, colleges,

Web service companies, and more. Although firewalls can mitigate some of the damage caused by DDoS attacks, they do not offer complete protection and may only provide a partial defense against attacks. Firewalls provide perimeter access control by monitoring and tracking permitted network traffic flows; however, firewalls and IPS cannot distinguish between malicious and legitimate users [7]. Once an attack traffic starts, it becomes unstoppable. Only the malicious actor can stop the DDoS attack from occurring as further as possible, even with the most significant Distributed Denial of Service attacks occurring from some of the most prominent American companies, developer platforms, software companies, international organizations, and even governments targeted, and nobody can stop it. Amazon Web Services (AWS), GitHub, Dyn, Spamhaus, and Estonia attacks [8]. have been hit with DDoS attacks in the past and the following year. Each of these companies relies exclusively on its Internet connectivity, specifically its Internet Web site, to conduct a multimillion-dollar business [9]. These DDoS attacks were so extensive that they seemed never-ending. Other preventive measures against DDoS attacks include proper traffic monitoring, including HTTP GET requests, shutting down ports that are not running and not in use, and conducting network vulnerability analysis and vulnerability testing. Also, blocking spoofed IP addresses can help prevent Distributed Denial of Service attacks.

A multi-tier hierarchical structure, a pyramid model, is used in DDoS attack structures to provide infrastructure. A DDoS attack uses an attacker to communicate with a client, which sends commands to zombies. The attacker may install Zombie (Bot) Software on the victim's machine to build a botnet and then perform a DDoS attack. In a DDoS attack, many compromised systems attack a single target, therefore causing a

denial of service for targeted users. However, as of today, we use IOT devices, and IOT devices are being infected by malware. This allows them to be controlled remotely by an attacker. Note that several types of malwares are performed. For example, malware is found in rootkits, Trojan horses, spyware, viruses, bots or botnets, worms, and ransomware [10]. Also, note that an attacker can control remotely in a cloud environment or cloud computing, such as through cloud account hijacking where a hacker manages to steal sensitive data [11].

Moreover, Distributed Denial of Service attacks are conducted using Denial of Service tools and software to launch DDoS attacks. These zombie software and tools are so powerful that attackers can use them to perform DDoS attacks. Such an attack is launched by exhausting the laptop and system from all available resources and filling up the whole file system where an attack is launched locally from within a local network. In other words, these tools and or software are called Tribe Flood Network – a program to conduct DDoS attacks, Tor, short for The Onion Router, Hping, LOIC (Low Orbit Ion Cannon), and many other tools and or software.

Attacks initiated within a local network, such as Fork Bomb attacks, target the system's resources by rapidly spawning processes. Conversely, when an attack is launched remotely across the network, it can exhaust resources through various DDoS attacks, affecting the local network and Wi-Fi connections. There are three common DDoS attack types. The first common DDoS attack type is a Volume-based attack, the second is a Protocol attack, and the third is an Application Layer attack. These three types define DDoS attack types. Volume-based attacks aim to overwhelm a site's bandwidth by sending as much traffic as possible, exhausting its resources. Application

layer attacks, such as those involving HTTP GET and HTTP POST requests, target the top layer of the OSI model to disrupt real-time traffic on websites and web services. Protocol attacks, an essential type of attack, are packet floods identified as SYN Flood, a common type of flood where a SYN Flood performs a TCP 3-way handshake. Note that, for a TCP 3-way handshake to complete, a connection between a user and server must complete, meaning that for a TCP 3-way handshake to complete: SYN (synchronize*)*, SYN-ACK (synchronize-acknowledgment*)*, ACK (acknowledgment) must be performed. However, if the TCP 3-way handshake is incomplete, for example, ACK fails. Therefore, this means a TCP, the 3-way handshake, has failed. Typically, in a SYN Flood attack, attackers abuse the TCP 3-way handshake by sending multiple SYN requests to a target server and do not respond to the server's SYN-ACK response, causing it to fail.

OSI models can help understand the importance of how Distributed Denial of Service attacks are structured through a hierarchy, the network communication through attacks, and operate across different layers. There are seven layers in the OSI Model of the Internet, which are most often viewed in Distributed Denial of Service (DDoS) attacks. The seven layers are Layer 1, the Physical Layer; Layer 2, the Data Link Layer; Layer 3, the Network Layer; Layer 4, the Transport Layer; Layer 5, the Session Layer; Layer 6, the Presentation Layer; and Layer 7, the Application Layer. Typically, DDoS attacks target Layers 3 to 7, exploiting vulnerabilities in the Network, Transport, Session, Presentation, and Application Layers.

An image will be depicted below to further describe a Distributed Denial of Service attack.

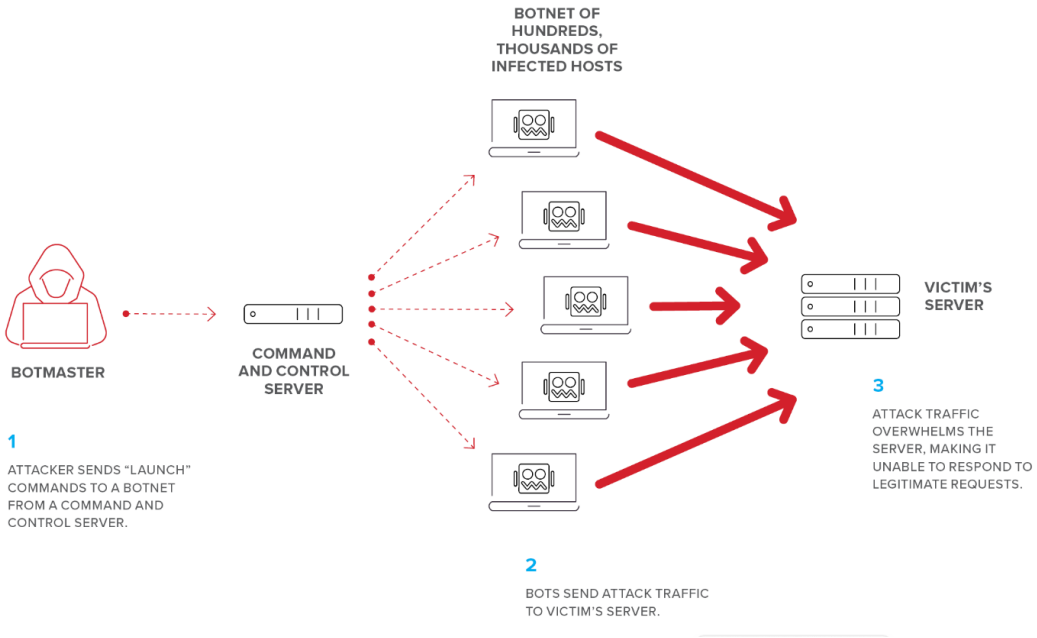


Figure 1. Distributed Denial of Service (DDoS) Attack [12]

In Figure 1, we see a Distributed Denial of Service attack. We see an "attacker" known as a botmaster and a malicious actor. What happens here is that the attacker uses a Command-and-Control Server to send "Launch" commands to a botnet through their Personal Computer or Laptop. Through the Command-and-Control Server, the attacker plans the attack setup, with TCP flooding attacks being a common type of Distributed Denial of Service attacks. The attacker compromises thousands of compromised and infected hosts. The attacker begins sending attack traffic to the victim's server. The attack traffic overwhelms the server, renders the target unreachable or inaccessible, denying legitimate users access to the service [13].

In this work, we use Machine Learning in Cybersecurity to detect attacks (normal traffic – attack traffic) and Neural Networks and show network vulnerabilities. By

analyzing data, we learn and find such patterns to detect traffic. In addition, our goal is to classify attacks into one where an attack is normal and two where attacks are classified as attacks and to distinguish DDoS traffic from normal traffic. We will use Machine Learning algorithms such as Decision Tree (DT), Support Vector Machine (SVM), CatBoost (CB), K-Nearest Neighbor (KNN), and Random Forest (RF), as well as Neural Networks such as Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), and Long Short-Term Memory (LSTM) to detect both traffics. We are evaluating the True Positive, True Negative, False Positive, and False Negative (TP-TN-FP-FN, which are abbreviations). We measure the accuracy, precision, detection rate, false positive rate, and F1- score using multi-class classification, a type of supervised learning. Furthermore, when evaluating Intrusion Detection Systems (IDS) using DDoS datasets such as the APA-DDoS and CIC-DDoS2019, the process of training, testing, graphing, validation, evaluation, and analyzing experimental results will play a vital role in further predictions, discussed in the subsequent chapters.

Intrusion Detection Systems

In cybersecurity, Intrusion Detection Systems, or IDS for short, play a significant role in computer networking and systems security, detecting malicious activities such as DDoS (Distributed Denial of Service) attacks - DoS (Denial of Service) attacks, cyber threats harm, and more. An Intrusion detection system (IDS) is software that automates the intrusion detection process and Intrusion detection is the process of monitoring the events occurring in a computer system or network and analyzing them for signs of possible incidents, which are violations or imminent threats of violation of computer security policies, acceptable use policies, or standard security practices [14]. Therefore,

IDS can detect DoS-DDoS attacks. Moreover, these IDS datasets are utilized in machine learning algorithms such as Decision Tree (DT), Support Vector Machine (SVM), CatBoost, K-Nearest Neighbor (KNN), and Random Forest (RF), to name a few, and other machine learning algorithms for training, testing, graphing, evaluation, accuracy, and prediction. Intrusion detection systems (IDS) and Intrusion detection mitigate certain types of DDoS attacks.

Problem Statement

This research analyzes a deterministic-holistic approach to detecting and identifying DDoS (Distributed Denial of Service) attacks using Cybersecurity datasets that are utilized for Intrusion Detection Systems (IDS), such as raw packet data from Packet capture (PCAP) files. With a conversion from Packet Captures to Comma Separated Values (CSV) files using CICFlowMeter-V3, these Cybersecurity datasets are essential for testing, training, graphing, and evaluating Intrusion Detection Systems (IDS). We want to detect malicious attacks. We are contributing to several improvements in cybersecurity that will help us advance cybersecurity. The accuracy of analyzing and detecting Normal - DDoS attacks will further improve with enhancements in Machine Learning algorithms and Neural Networks. Moreover, the Area Under the Curve (AUC) of a ROC (Receiver Operating Characteristic) curve will be plotted to show the True Positive Rate (TPR) and the False Positive Rate (FPR) to distinguish between the two classes. In addition to the AUC of an ROC curve, a confusion matrix will be depicted with True Positive, True Negative, False Positive, and False Negative.

Findings and Goal

In this work, we use Machine Learning in Cybersecurity to detect attacks (normal traffic – attack traffic) and Neural Networks and show network vulnerabilities. By analyzing data, we learn and find such patterns to detect traffic. In addition, our goal is to classify attacks into one where an attack is normal and two where attacks are classified as attacks and to distinguish DDoS traffic from normal traffic. We will use Machine Learning algorithms such as Decision Tree (DT), Support Vector Machine (SVM), CatBoost, K-Nearest Neighbor (KNN), and Random Forest (RF), as well as Neural Networks such as Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), and Long Short-Term Memory (LSTM) to detect both traffics. We are evaluating the True Positive, True Negative, False Positive, and False Negative (TP-TN-FP-FN, which are abbreviations) by using a confusion matrix. We measure the accuracy, precision, detection rate, false positive rate, and F1- score using multi-class classification, a type of supervised learning. Also, the Area Under the Curve (AUC) of an ROC (Receiver Operating Characteristic) curve will be plotted to assess the metrics and evaluation. Furthermore, when evaluating Intrusion Detection Systems (IDS) using DDoS datasets such as the APA-DDoS and CIC-DDoS2019, the process of training, testing, graphing, validation, evaluation, and analyzing experimental results will play a vital role in further predictions, discussed in the subsequent chapters.

CHAPTER 2

Machine Learning Algorithms

Chapter 2 discusses Machine Learning algorithms, their background, the importance of Statistical Learning, and why we apply these algorithms to our model. Followed by the Machine Learning algorithms, we will continue discussing the Neural Networks, backgrounds, and the importance of Neural Nets and why we apply them to our model for better understanding. Note that, in Machine Learning algorithms, we are usually categorizing the algorithms into supervised-unsupervised learning and neural networks.

Decision Tree

A Decision Tree is a supervised machine-learning algorithm [15] that employs nonparametric methods and is used for data classification. One such nonparametric method is nonparametric statistics. Nonparametric statistics encompasses statistical methods that do not make any assumptions about the underlying distribution of data, and it can be contrasted with parametric statistics, which makes explicit assumptions about the distribution of observed data and uses the data to estimate the parameters of that distribution [16]. Nonparametric statistics serve as an approach in most Decision Trees. Classification and regression tasks are used in Decision trees, such as building a model in a tree structure. Decision trees may easily represent complex concepts whose definitions can be described based on the feature space [17].

In Figure 2, we will show a cybersecurity tree structure example; see below.

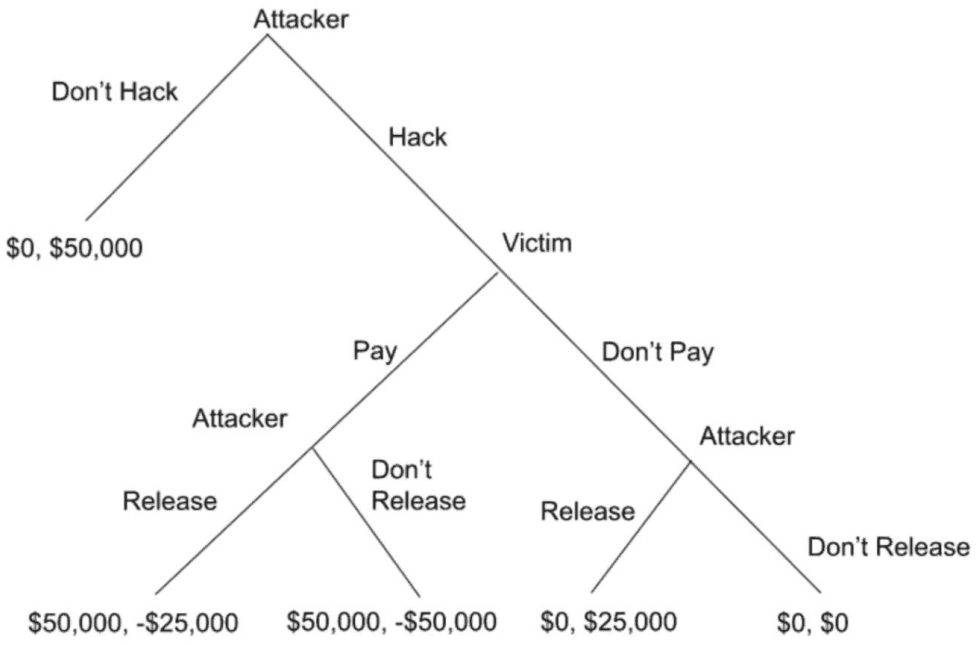


Figure 2. Cybersecurity Tree Structure Example [18]

Figure 2 exemplifies a cybersecurity tree structure example, as seen above. This image depicts an example of an attacker in cybersecurity. The attacker has two options: to Hack, "Yes" and Don't Hack, "No". A decision tree is a hierarchical data structure [19] that presents data through a divide-and-conquer strategy. This approach exemplified above utilizes the divide-and-conquer, systematically diving the problem into a tree structure. Divide-and-conquer is one where the underlying technique, such as a decision tree, is used as above. For example, the root node, splitting the nodes, the leaf nodes, selecting the feature of each node, and so on. In the beginning, a root node divides the dataset, splitting it into two nodes, and then, lastly, it shows the leaves as the final output. As seen in Figure 2, the Attacker is given Yes and No options. In Yes, the Attacker

succeeded, meaning that the Attacker proceeds further, and in Don't Hack, the Attacker does not proceed and reaches that point and stops there.

The advantages of decision trees are that they are inexpensive to construct, speedy at classifying unknown records, easy to interpret for small-sized trees, their accuracy is comparable to other classification techniques for many simple data sets, and they exclude unimportant features [20]. Decision trees have many advantages where DDoS attacks succeed within that scope. Decision trees provide unique insights into the problem of identifying malicious activity and can assist in the creation of technology-specific techniques to defend against attacks and data and identifying significant characteristics in network data that indicate malicious activities [21]. This is critical when classifying threats like attacks vs. normal traffic in real-time and the real world.

CatBoost

Catboost is a high-performance open-source library that is a machine learning algorithm that uses gradient boosting on decision trees [22] and was developed by Yandex for classification and regression tasks. Catboost supports Grading Boost on Decision Trees (GBDT). Catboost performs well within a Graphic Processing Unit (GPU), has quicker speed, faster prediction, improves model accuracy, and supports **c**ategorical features. Cat Boost differs from other gradient algorithms like XGBoost and LightGBM because CatBoost builds balanced trees that are symmetric in structure, meaning that in each step, the same feature-split pair that results in the lowest loss is chosen and applied to all the nodes in that level [23].

Additionally, Catboost utilizes gradient boosting techniques to reduce the model's overfitting. One of the critical strengths of Catboost is that categorical variables are used

in the graph to prevent the model from overfitting. In many real-world applications, categorical data is prominent. Thus, we generalize unseen data and the model's accuracy.

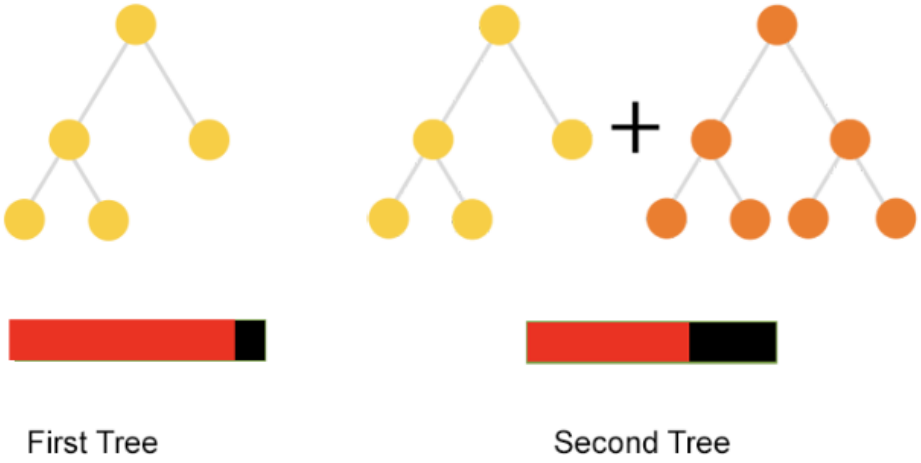


Figure 3. First and Second Trees [24]

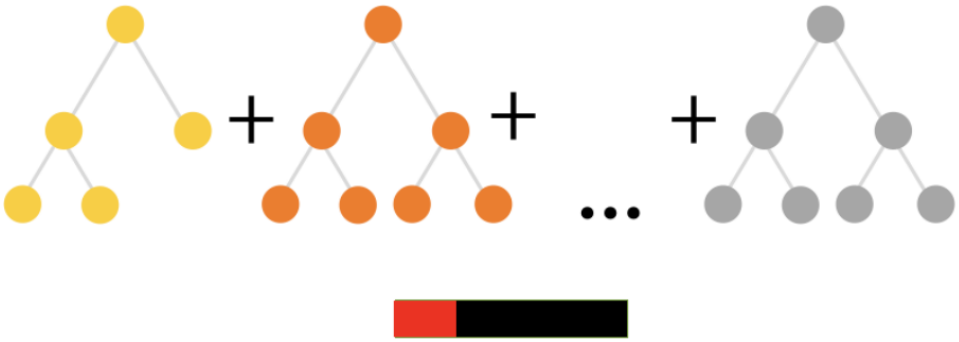


Figure 4. N-th Tree [24]

Figure 3 shows the first tree and the second three. Usually, these include signification errors. As we can see, both trees in Figure 3 include an error where the tree causes the error. Typically, having numerous nodes means many trees can overfit the data, which is a bad example in Catboost. In Figure 4, we have the N-th tree. The N-th tree refers to the tree at position N in the sequence of trees built during the training stage.

For instance, if K = 4, then the 4th tree would be the tree constructed after three other trees have been added to the model, and this is one way of the trees working in Catboost.

Random Forest

Leo Breiman and Adele Cutler developed the Random Forest algorithm [25] in 2001. It is a machine learning technique for classification, prediction, and regression tasks and uses ensemble learning methods. Just like decision trees, the random forest also uses trees. Note that random forest uses an ensemble of trees and lets them vote for the most popular class, and to grow these ensembles, often random vectors are generated that govern the growth of each tree in the ensemble [25]. Random Forest employs a bagging technique. Usually, it predicts and provides higher accuracy than most machine learning algorithms-models in predictions. It can enhance cybersecurity through its Intrusion Detection Systems (IDS) application. While Decision Trees offer good generalization, they become less interpretable as more layers are added to the model in the Random Forest approach.

Moreover, we will show a Random Forest algorithm and its methodology. As mentioned earlier and depicted below, Random Forest uses a decision tree to make predictions. Figure 5 illustrates how a Random Forest operates.

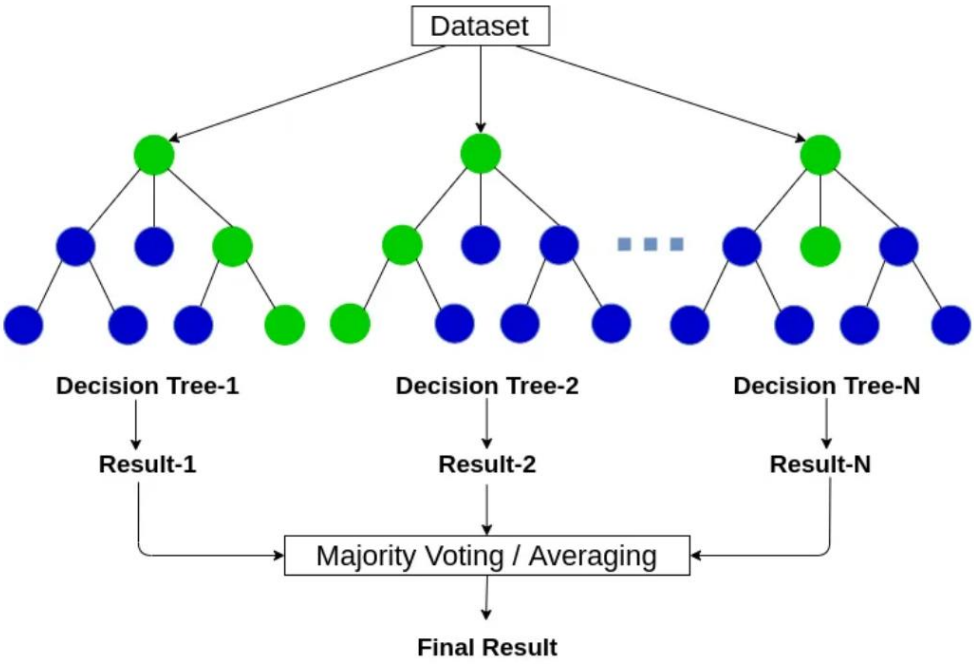


Figure 5. Random Forest [26]

In Figure 5, we start with a dataset, beginning with the dataset we possess, which will be used for training and testing. Subsequently, multiple decision trees will be conducted. The final Decision Tree – is N, where N refers to the Decision Tree at position N, and each tree yields a result. Then, finally, once all Decision Trees have been combined, we find the majority voting or averaging for prediction. Lastly, we obtained the result from the Random Forest.

A Random Forest is an excellent example of DDoS attacks in preventing splitting the decision trees across and predicting the majority votes or averaging and finalizing an output to detect DoS-DDoS attacks; a random forest tree structure can be used to categorize the threats and attacks and what is happening based on these attacks.

Support Vector Machine

The Support Vector Machine (SVM) is a supervised machine learning algorithm. Like many other machine learning algorithms, SVM is commonly used for classification, regression tasks, and outlier detection. In 1964, Vladimir N. Vapnik and Alexey Ya Chervonenkis first proposed the concept of the Support Vector Machine. After the proposal, Vladimir N. Vapnik developed the Support Vector Machine in the 1990s, and the algorithm has since become widely used in statistics and machine learning. SVM uses the kernel trick. The kernel trick was used by Boser, Guyon, and Vapnik [27] to extend the Generalized Portrait hyperplane classifier to nonlinear Support Vector Machines [28].

In the past decade, SVMs (Support Vector Machine) have been developed at a fast pace both in theory and practice [29]. SVM can solve linear and non-linear problems, works well for many practical issues, and works well when there is a clear margin of separation between classes. SVM is more effective in high-dimensional spaces and effective in cases where the number of dimensions is greater than the number of samples [30]. In cyber-security, using an SVM can improve the accuracy of IDSs (Intrusion Detection Systems); the classifier created by this technique is useful to predict between two possible outcomes (i.e., malicious and non-malicious network traffic) [31].

The advantage of Support Vector Machines (SVM) is their ability to achieve high accuracy in predictions. However, the disadvantage is that they require more extended training datasets, which can be problematic. Training with larger datasets can take days or even hours. One reason it takes a prolonged training duration is time and space complexity. Additionally, Big-O notation for time and space complexities can be complex challenges to test-train where time is critical. According to the Big-O

Complexity Chart, the Support Vector Machine training times are longer due to their complexity, where the complexity is denoted by O(n^2). Consequently, this indicates that SVM will take and require a more extended training period, based on the graph as illustrated in Figure 6.

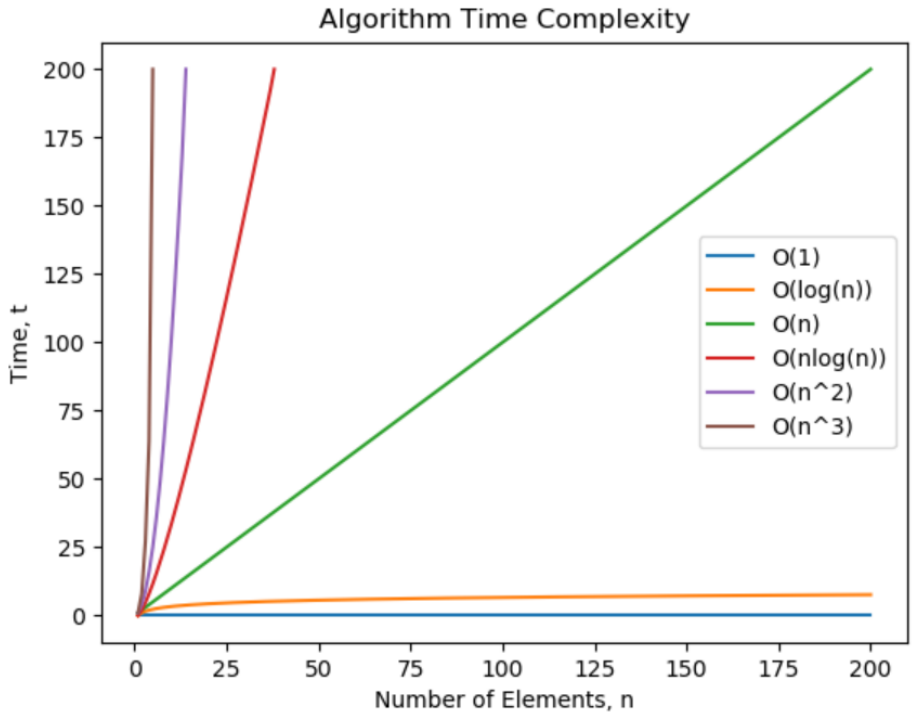


Figure 6. Big-O Complexity Chart [32]

Model	Complexity
Logistic Regression	O(*nd*)
KNN	O(*knd*)
SVM	O(n^2)
Decision Tree	O(*nlog(d)*)
Random Forest	O(*nlog(n)dk*)

Figure 7. Training Time Complexities for Machine Learning Models [32]

Figure 6 depicts a Big-O Complexity Chart. Time is defined by t, whereas n represents the number of elements. The colors utilize various levels of time complexity associated with some colors' time complexity based on supervised machine learning algorithms, which can significantly play a role in these machine learning algorithms during the training phase. In Figure 7, a machine learning model and complexity can be exhibited. We saw in Figure 6 how the time depicts how time is represented, and based on the timings, we can infer an idea of how the time impacts the training and testing dataset with these specific machine learning models.

Support Vector Machine includes the hyperplane used in Support Vector Machine analysis. An SVM constructs a hyperplane to classify the data points by separating the negative and positive classes in the SVM graph.

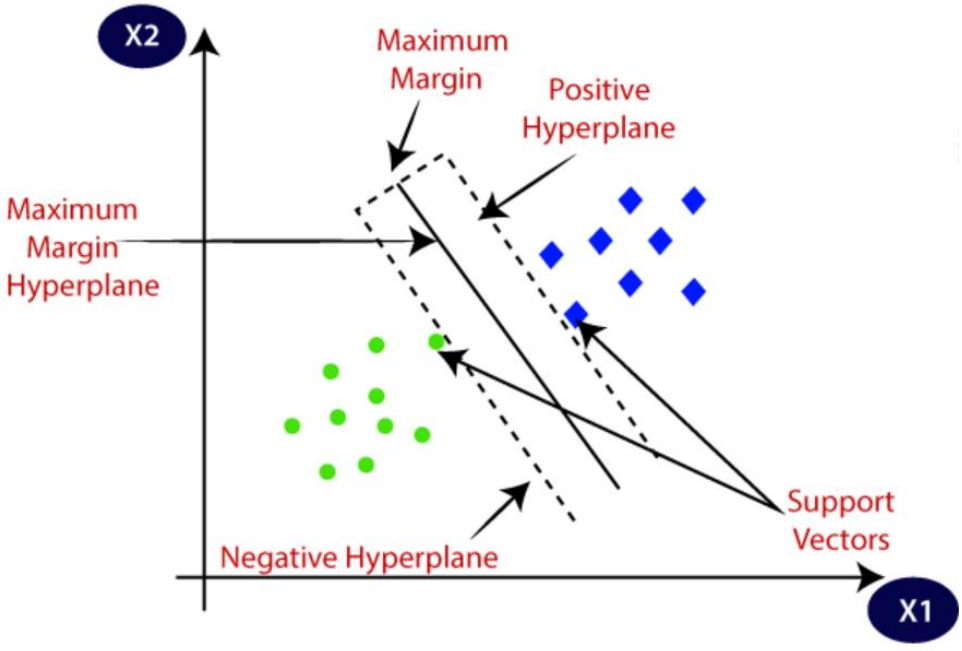


Figure 8. Support Vector Machine Hyperplane [33]

Figure 8 shows the Support Vector Machine hyperplane; we can observe the Support Vectors, the Negative-Positive Hyperplane, and the Maximum Margin – Maximum Margin Hyperplane. In the graph, the axes X1 and X2 are labeled. When analyzing Support Vectors, we must calculate the distance between the hyperplane and the data points, which is essential in finding the optimal hyperplane. Green and blue represent the supporting vectors, where the colors are the classes. In other words, the Support Vector Machine finds the best line between the supporting vectors and separates the classes for these two colors. We can see here that the two classes are a binary classification problem, as shown below in Figure 8.

K-Nearest Neighbor

The K-Nearest Neighbor (KNN) is a non-parametric supervised learning classifier. KNN is used for classification and regression. Many data scientists use KNN because of its popular machine-learning algorithm. The algorithm is simple to implement, and there is no need to build a model, tune multiple parameters, or make additional assumptions [34]**.** KNN provides high accuracy, which is a good fit for prediction. In KNN, we usually first select the number K of the neighbors. K can be any number. We use odd numbers for k, a customary practice in binary classification tasks. We also calculate the distance in K-Nearest Neighbor (KNN). However, be wary of the disadvantage that KNN becomes much slower as the number of observations and independent variables increases [34]. In Figure 9, we will show a before-after KNN to show the difference between what happens before a KNN and after. See below.

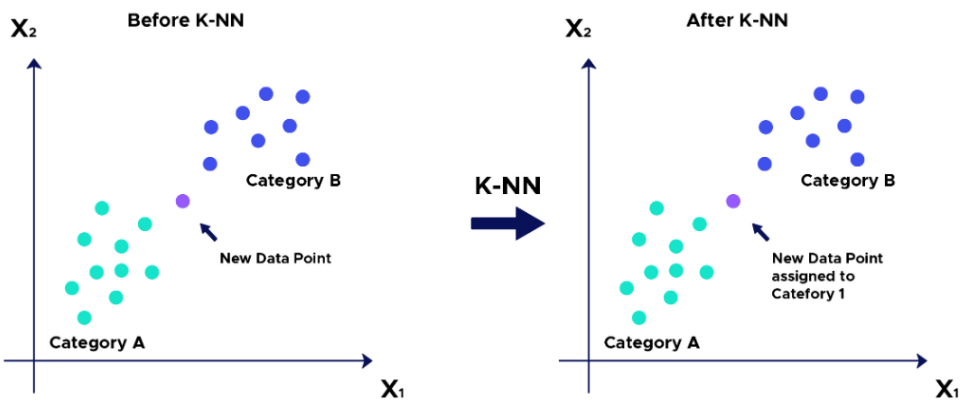


Figure 9. **K-Nearest Neighbor Before and After** [34]

In Figure 9, K-Nearest Neighbor Before and After is shown. The axes X1 and X2 are labeled in the graph for both before and after. We have two categories, Category A and Category B, which can mean this is a binary classification example. Before K-NN, we have a New Data Point, shown in Figure 9, the purple circle dot. After K-NN, our New Data Point is assigned to Category A, as shown above. Consequently, we can visualize Figure 9, where Category A identifies as a normal attack, and Category B identifies as attack traffic. From then on, we can validate the KNN classifier and compare the accuracy performance for which Category A or B predicts best on K-Nearest Neighbor.

Convolutional Neural Network

Convolutional Neural Networks (CNN), also known as ConvNets, are a type of network design for deep learning algorithms, methods, and neural networks. The concept of Convolutional Neural Networks was introduced by Yann LeCun and Yoshua Bengio in 1995. CNNs (Convolutional Neural Networks) have been around for more than twenty years. However, Yann LeCun demonstrated Convolutional Networks in 1989. Yann

LeCun's previous work on simple digit images (LeCun, 1989) showed that the architecture of the network strongly influences the network's generalization ability [35].

Tasks like image processing, face recognition, object recognition, object detection, semantic segmentation, classification tasks, and image recognition are used in Convolutional Neural Networks.

CNNs have one-dimensional, two-dimensional, and three-dimensional aspects, usually representing spatial dimensions. Each spatial dimension represents a specific part of the images in 1D-2D-3D. One dimension corresponds to a line, two dimensions form a flat panel, and three dimensions create a sphere or cube. In each of these dimensions, backward and forward operations apply. In Convolutional Neural Networks, backward and forward operations are used along the X-Y-Z axes.

CNN is designed to automatically and adaptively learn spatial hierarchies of features through backpropagation by using multiple building blocks, such as convolution layers, pooling layers, and fully connected layers [36]. Training Convolutional Neural Networks in deep learning requires developers to collect a large, labeled dataset and configure a network architecture capable of learning the features and modeling [37]. CNNs use pooling layers, which are part of dimension reduction. There are three types of layers: convolutional, pooling, and fully connected layers. The first two, convolution and pooling layers, perform feature extraction, whereas the third, a fully connected layer, maps the extracted features into final output, such as classification [36].

CNNs can automatically extract features, they take the image's raw pixel data, train the model, and automatically extract features for better classification and accuracy. CNNs are effective in feature extraction, image classification, pattern recognition, and

semantic segmentation. However, CNNs are also ineffective because they require a large amount of training data to be influential, and more time is needed for training. Additionally, CNN cannot encode an object's orientation and position and can't process spatially invariant data [38].

CNN is designed to mimic the human brain, specifically the visual cortex. The human brain takes input information (something you see or hear or feel), operates on the input, and puts on output information (something you say or write, for example) [39]. In Figure 10, an architecture is shown, and a more semantic meaning will be provided to demonstrate better how the convolutional neural network functions.

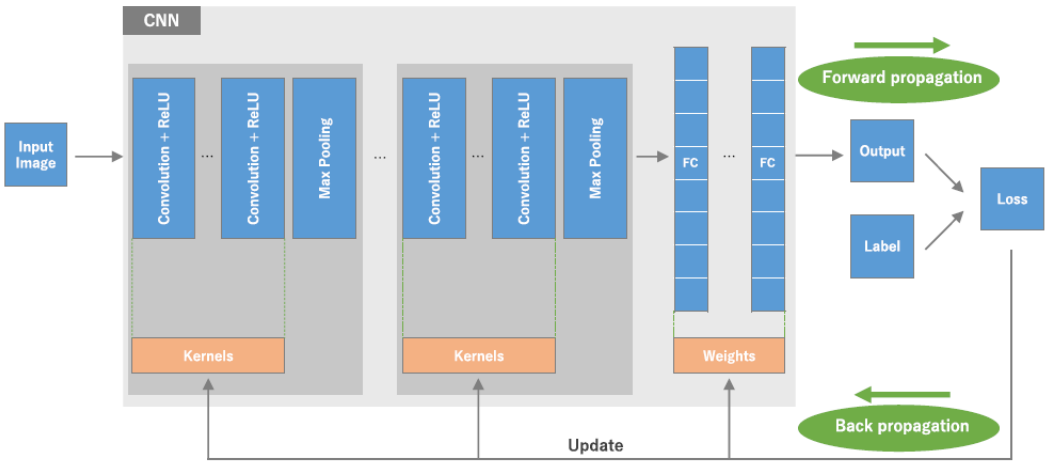


Figure 10. Convolutional Neural Networks (CNN) Architecture [36]

Figure 10 depicts the architecture of a Convolutional Neural Network (CNN), which is crucial for understanding how CNNs function. Typically, a CNN begins with an input image and produces an output image, which is mostly how the operation of a computer works and functions.

CNNs have kernels such as Convolutional Layers, the Rectified Linear Unit (ReLU) layer, and the Max Pooling Layer. The Fully Connected (FC) Layers represent

the weights. In CNNs, we use forward propagation for the forward pass and backward propagation for the backward pass. Generally, Convolution + ReLU and Max Pooling are part of the feature learning process, whereas Fully Connected (FC) layers are involved in classification. At the end of the CNN architecture, we have the loss function and the backpropagation. Where a model's performance under particular kernels and weights is calculated with a loss function through forward propagation on a training dataset, and learnable parameters, i.e., kernels and weights are updated according to the loss value through backpropagation with gradient descent optimization algorithm [36].

Recurrent Neural Network

Recurrent Neural Network is a type of artificial neural network (ANN). RNN is used both for supervised and unsupervised learning. Recurrent Neural Networks are used in various tasks, including speech recognition, sentiment analysis, natural language processing (NLP), gesture recognition, robotics, anomaly detection, time series prediction, and healthcare. Moreover, RNNs (Recurrent Neural Network) are used in sequence generation.

The structure and function of the human brain serve as inspiration for the RNN. A neural network consists of multiple layers of interconnected nodes, or "neurons," which process input data and learn to extract increasingly abstract features and patterns from it [40]. RNN has at least one feedback connection, and RNNs allow feedback connections in neurons. In RNN, any neuron can be connected to any other, even to itself; allowing the presence of feedback connections among neurons has an advantage; it leads naturally to an analysis of the networks as dynamic systems, in which the state of a network, at one moment in time, depends on the state at a previous moment in time [41].

The advantage of RNNs is that they predict with better accuracy and higher precision. This is because RNNs can learn from past experiences, particularly sequential data [42]. Furthermore, a recurrent neural network will show how the Recurrent Neural Network functions with the layers in Figure 11 and Figure 12. Also, the Recurrent Neural Network differs from the Convolutional Neural Network in its layers.

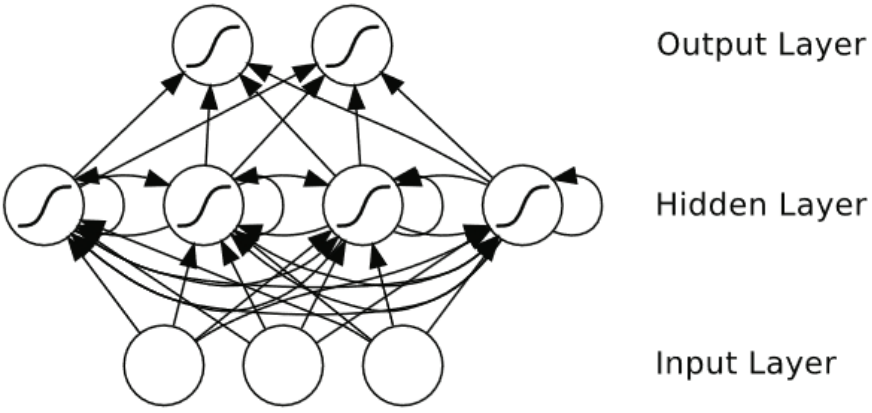


Figure 11. Recurrent Neural Network [43]

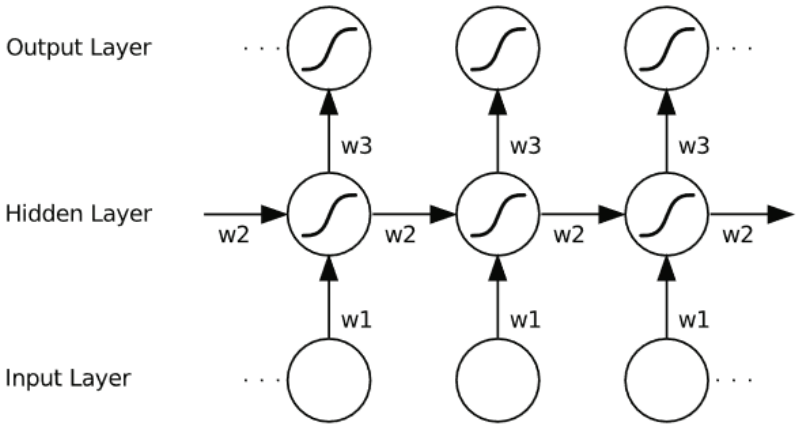


Figure 12. Unfolded Recurrent Network [43]

These figures depict a Recurrent Neural Network architecture. Figure 11 shows an RNN with the Output Layer, the Hidden Layer, and the Input Layer. All three of these layers are essential in a Recurrent Neural Network. Figure 12 conveys the same meaning

as Figure 11 but with weights and nodes instead. In Figure 12, each node represents a layer of network units at a single timestep. The weighted connections from the input layer to the hidden layer are labeled ‘w1’, those from the hidden layer to itself (i.e., the recurrent weights) are labeled ‘w2’, and the hidden to output weights are labeled ‘w3’. Note that the same weights are reused at every timestep, and bias weights are omitted for clarity [43].

RNNs are an excellent tool for detecting DDoS attacks because they utilize sequence data. RNNs effectively identify DDoS attacks due to their inherent ability to detect network traffic patterns and malicious behavior based on the intrusion detection system (IDS). Each node can represent and analyze patterns in network traffic to detect DDoS attacks and distinguish between normal and attack traffic.

Long Short-Term Memory

Long Short-Term Memory is a type of Recurrent Neural Network. Long Short-Term Memory was introduced by Hochreiter and Schmidhuber. LSTM shares some similarities and meanings with RNN. LSTM have recurrent connections so that the state from previous activations of the neuron from the previous time step is used as context for formulating an output, and unlike other RNNs, the LSTM has a unique formulation that allows it to avoid the problems that prevent the training and scaling of other RNNs [44].

Long Short-Term Memory (LSTM) is an RNN architecture specifically designed to address the vanishing gradient problem [44]. Typically, LSTM (Long Short-Term Memory) networks have a different architecture compared to traditional Recurrent Neural Networks (RNNs), including different gating mechanisms. The difference lies in the neural network layer. RNN has one recurrent layer, while LSTMs (Long Short-Term

Memory) contain multiple neural network layers within their RNN layer, consisting of four neural network layers. The architecture can be seen below.

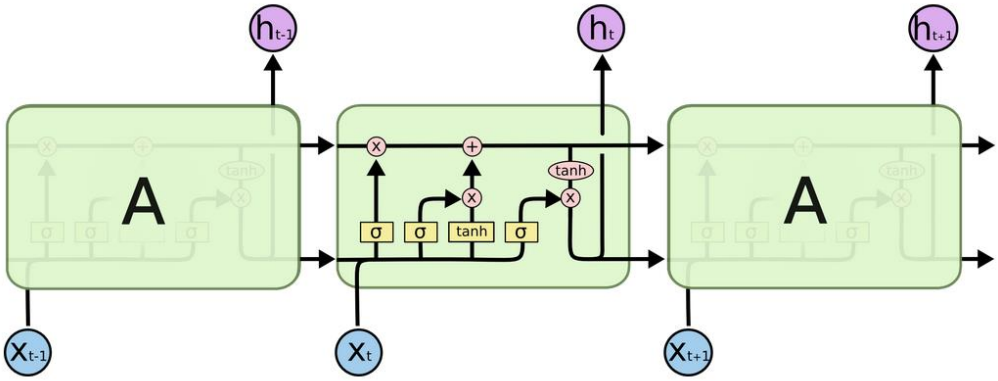


Figure 13. Long Short-Term Memory Contains Four Interacting Layers [45]

An LSTM with four neural network layers is depicted in Figure 13. These four neural network layers represent the sigma. Three sigmoid are indicated by Greek letters, and tanh represents the hyperbolic tangent function. There are four gates: the forget gate, the input gate, the memory gate, and the output gate. Each gate corresponds to vector transfer. An arrow sign denotes these vector transfers. Pink circles represent the pointwise operation; 'x,' '+,' and 'tanh' are represented within the circles. There are two more arrows: one arrow is split into two, indicating concentration, and the other is a copy, where one-line splits into two arrows. Also, we can see the X denoted X_{t-1}, X_t, X_{t+1} on the bottom with the blue circles. On the top, we see the purple circles denoted with h, such as h_{t-1}, h_t, h_{t+1}. X_{t-1}, X_t, X_{t+1} are input sequences at time steps with $t-1, t, t+1$. The same is true at h, but h are the hidden states at time steps.

CHAPTER 3

Implementation Details

Chapter 3 will discuss implementation details based on our datasets and data and the following: Hardware and Software Specifications, Exploratory Data Analysis, Data Preprocessing, Evaluation Metrics, Feature Engineering, Feature Extraction, Data Cleaning, Feature Scaling, and Train Test Split.

Hardware and Software Specifications

At the beginning of our Master's Thesis, a Central Processing Unit (CPU), specifically the 12th Gen Intel(R) Core (TM) i7-12700H, was used. This PC has 16.0 GB installed RAM to run Machine Learning datasets with a 64-bit operating system and Windows 11 Home edition—a Graphic Processing Unit (GPU). This PC, specifically the NVIDIA GeForce RTX 3060, was used for parallel processing for the APA-DDoS dataset discussed in Chapter Five. A Central Processing Unit (CPU) and Graphic Processing Unit (GPU) can handle computational memory in our dataset. As discussed in the following chapters, we can get accuracy, Area Under the Curve, confusion matrix, and more with the APA-DDoS dataset. GPUs (Graphic Processing Units) are designed to handle parallel processing with your PC and the specifications needed to run them.

In addition, we want to ensure that everything has been installed correctly and has the correct version numbers to match the NVIDIA GeForce RTX 3060 specifications. If incorrect version numbers do not match the NVIDIA GeForce RTX 3060 GPU, we cannot use the GPU, and all else fails. The NVIDIA GeForce RTX 3060 GPU has 3840 CUDA Cores, 1425 Graphics Boost Clock (MHz), 14.00 Gbps Memory data rate, and GDDR6 Memory Type.

To get the second dataset to run, Anaconda 3 on Windows 11 had been installed to use Anaconda. Next, an Anaconda Prompt and a command-line interface (CLI) were used, using the Anaconda distribution of Python. Anaconda Prompt is primarily done by activating an environment. Within this environment, TensorFlow is installed and activated in the Conda environment. Once an activated name has been activated, an environment such as the base name is created and activated through Anaconda Prompt. TensorFlow and Jupyter Notebook were installed within this environment, using Python 3.9 as the base version being used, and have been installed to use Python and their libraries and to launch Jupyter Notebook and get the first dataset, the APA-DDoS dataset to work locally in localhost through a web browser of our PC.

Our GPU cannot handle the memory computation with the dataset, which will be discussed in Chapter 4, the CIC-DDoS2019 dataset. Therefore, we ran out of GPU memory. In addition, a Currie cluster was used to test and train our model with the second dataset. TensorFlow and PyTorch are already being set up in the base environment and used for our dataset.

The Currie specifications are that Curie consists of a login node, 20 compute nodes, and a 0.75 PB BeeGFS parallel file system with a dedicated metadata server. Where each compute node has two Intel Xeon Gold 6338 CPUs with 32 cores each (for a total of 64 cores) and 256 GB of RAM. The compute nodes are connected together via Omni-Path, a high bandwidth (100 Gbps), low latency interconnect that is similar to InfiniBand. In addition, 8 of the compute nodes contain 4 NVIDIA A30 GPUs each. Python is also a programming language that we use. The Currie cluster also used NumPy, Seaborn, Pandas, Matplotlib, and Scikit-learn. Note that the size of the datasets and the

size of the Comma-Separated Values file will be covered in the following chapters. This shows we cannot use parallel processing with the second dataset because the Comma-Separated Values file is too large for the NVIDIA GeForce RTX 3060 GPU's computational memory to handle and is insufficient. Lastly, we launched JupyterLab and got the dataset, the CIC-DDoS2019, to work locally on localhost through a web browser on our PC.

Exploratory Data Analysis

Exploratory Data Analysis (EDA) is a concept utilized in statistics, mathematics, and probability theory and is vital in the real world. Exploratory Data Analysis (EDA) is an approach/philosophy for data analysis that employs a variety of techniques (primarily graphical) to maximize insight into a data set, uncover underlying structure, extract important variables, detect outliers and anomalies, test underlying assumptions, develop parsimonious models; and determine optimal factor settings [46]. EDA plays a pivotal role in machine learning because it involves preparing and visualizing the dataset through various methods such as bars, graphs, and more. This visualization is an essential step at the beginning of analyzing our dataset. EDA helps us comprehend how to represent the dataset's characteristics and patterns. Exploratory data analysis is also beneficial because it helps us make better decision-making with machine-learning algorithms for predictions and use. It also allows us to portray our data visually and analytically. Matplotlib and Seaborn are used to plot in Python, a programming language. However, Excel, Tableau, and R are also used for EDA, which are programming languages.

We will analyze and visualize the datasets in EDA using the Exploratory Data Analysis (EDA). Chapters 4 and 5 mentioned two datasets: the APA-DDoS and the CIC-DDoS2019. To begin with, the dataset will be analyzed and visualized, usually at the beginning of our start, by reading the dataframe with ‘pd.read_csv()’ as this is the first step and a crucial and vital step in understanding and reading the dataset. The first name, the pd, is usually the pandas as pd that we import the Pandas library to use in Python. After reading the dataframe, we print and show the first five rows of the dataframe to show what it reads.

	ip.src	ip.dst	tcp.srcport	tcp.dstport	ip.proto	frame.len	tcp.flags.syn	tcp.flags.reset	tcp.flags.push	tcp.flags.ack	...	tcp.seq	tcp.ack	frame.time
0	192.168.1.1	192.168.23.2	2412	8000	6	54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071112000 Mountain Dayli...
1	192.168.1.1	192.168.23.2	2413	8000	6	54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071138000 Mountain Dayli...
2	192.168.1.1	192.168.23.2	2414	8000	6	54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071146000 Mountain Dayli...
3	192.168.1.1	192.168.23.2	2415	8000	6	54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071152000 Mountain Dayli...
4	192.168.1.1	192.168.23.2	2416	8000	6	54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071159000 Mountain Dayli...

5 rows × 23 columns

Figure 14. Displays the First Five Rows of the APA-DDoS Dataset.

frame.len	tcp.flags.syn	tcp.flags.reset	tcp.flags.push	tcp.flags.ack	...	tcp.seq	tcp.ack	frame.time	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Label
54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071112000 Mountain Dayli...	8	432	4	216	4	216	DDoS-PSH-ACK
54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071138000 Mountain Dayli...	10	540	5	270	5	270	DDoS-PSH-ACK
54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071146000 Mountain Dayli...	12	648	6	324	6	324	DDoS-PSH-ACK
54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071152000 Mountain Dayli...	10	540	5	270	5	270	DDoS-PSH-ACK
54	0	0	1	1	...	1	1	16-Jun 2020 20:18:15.071159000 Mountain Dayli...	6	324	3	162	3	162	DDoS-PSH-ACK

Figure 15. Displays the Label Column (the Last Column) of our APA-DDoS Dataset.

	Flow ID	Source IP	Source Port	Destination IP	Destination Port	Protocol	Timestamp	Flow Duration	...	Active Std	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	SimillarHTTP	Inbound	Label
	·192.168.50.1-983-1158-17	172.16.0.5	983	192.168.50.1	1158	17	2018-12-01 11:07:15.390517	1	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	1	DrDoS_DNS
	92.168.50.1-599-41651-17	172.16.0.5	599	192.168.50.1	41651	17	2018-12-01 11:07:54.700476	1	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	1	DrDoS_DNS
	92.168.50.1-945-41801-17	172.16.0.5	945	192.168.50.1	41801	17	2018-12-01 11:07:27.807072	1	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	1	DrDoS_DNS
	92.168.50.1-792-44080-17	172.16.0.5	792	192.168.50.1	44080	17	2018-12-01 11:07:12.399211	1	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	1	DrDoS_DNS
	·192.168.50.1-625-1381-17	172.16.0.5	625	192.168.50.1	1381	17	2018-12-01 11:07:38.086104	1	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	1	DrDoS_DNS

Figure 16. Displays the First Five Rows of the CIC-DDoS2019 Dataset.

Figures 14, 15, and 16 display the first five rows, the label column, and the last five rows of the APA-DDoS and CIC-DDoS2019 datasets. The head() method can be used to analyze the APA-DDoS dataset and the CIC-DDoS2019, as well as the tail() method, where the tail() method reads the last five rows in Python. To print the first five rows of the dataframe, we use a ‘head()’ method. And to print the last five rows of the dataframe, we use a ‘tail()’ method. We can see the columns and the labels in the dataset, as more will be described in Chapters 4 and 5. We will visualize and read the last five rows of the APA-DDoS and the CIC-DDoS2019 dataset.

Moreover, a Pearson Correlation Matrix for the APA-DDoS Dataset will be described shortly in Figure 17. The Pearson Correlation matrix is part of Exploratory Data Analysis (EDA).

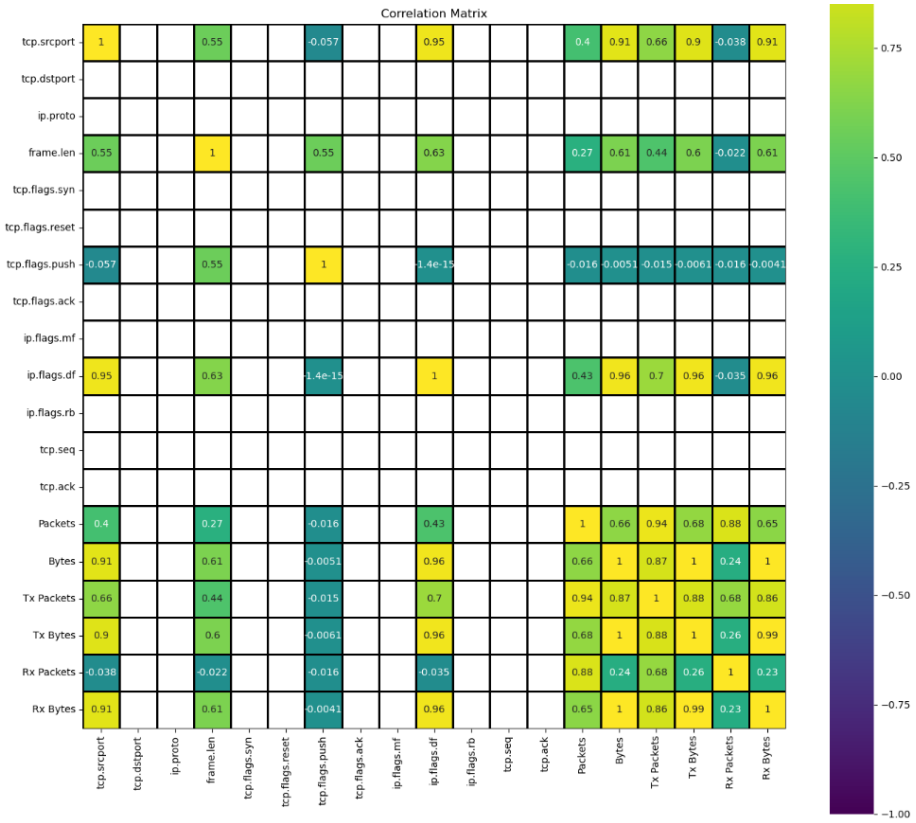


Figure 17. Pearson Correlation Matrix of the APA-DDoS Dataset.

Figure 17 depicts a Pearson Correlation Matrix of our APA-DDoS Dataset. It displays the feature names and is used wisely in Mathematics and Statistics. Additionally, it helps understand the relationship between x and y. One way to quantify this relationship is by using the Pearson correlation coefficient, which measures the linear association between two variables*; it* has a value between -1 and 1, where -1 indicates a perfectly negative linear correlation between two variables, 0 means no linear correlation between two variables, and 1 indicates a perfectly positive linear correlation between two variables [47]. So, these colors represent the correlation coefficient between the variables and between -1, 0, and 1. Colors represent different correlation coefficients; three falls between -1, 0, and 1. A negative correlation is -1, no correlation is identified as 0, and +1 is recognized as positive. These three correlations are crucial because, generally, the

numbers are based on the axis line in a graph. The white empty boxes in the correlation matrix do not clearly show the missing or non-existent correlations between -1, 0, +1, and variables. In other words, the correlation matrix in Python uses the ‘corr()’ method.

Our APA-DDoS Dataset has three labels, according to Chapter 5. Three labels are Benign, DDoS-ACK, and DDoS-PSH-ACK. The counts for each label are 75,600 for Benign, 37,800 for DDoS-ACK and 37, 800 for DDoS-PSH-ACK.

An SNS count plot is shown below as well as a pie chart.

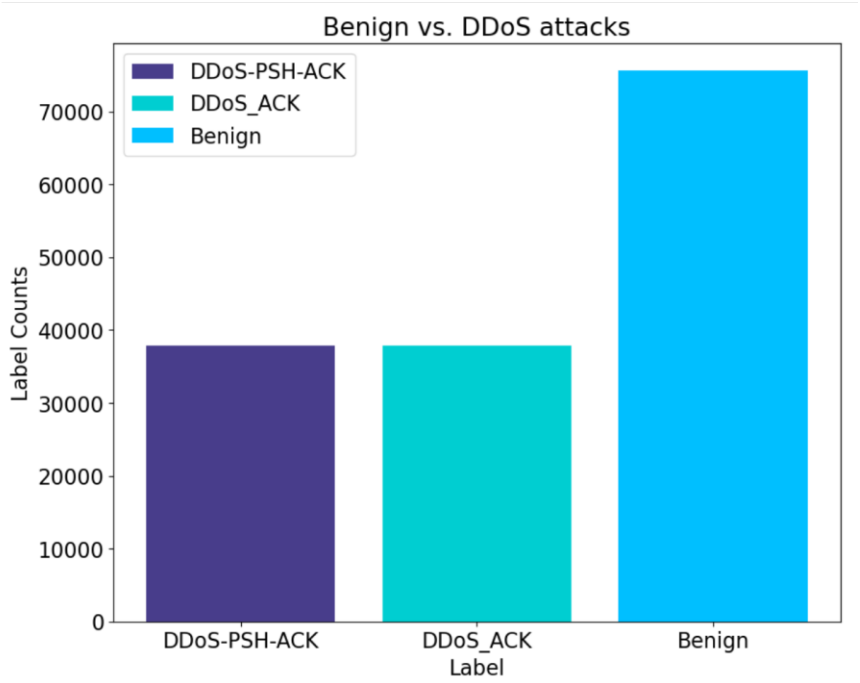


Figure 18. SNS Count Plot

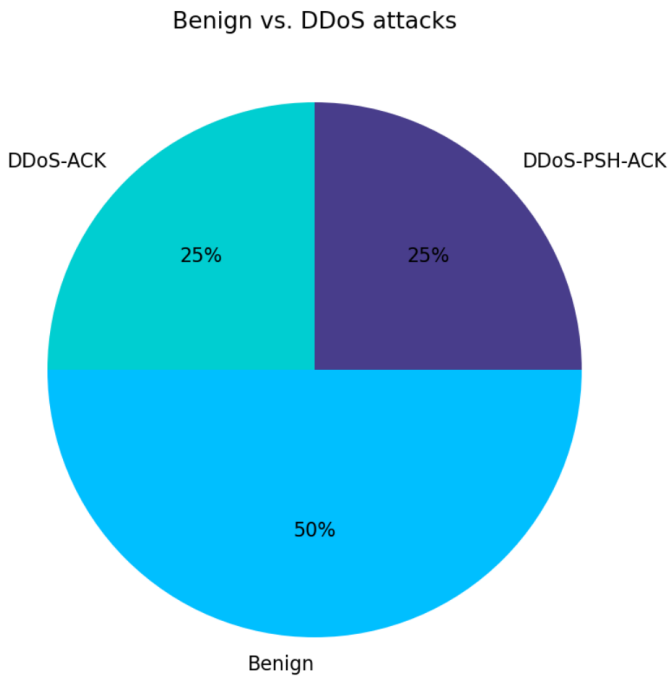


*Figure 19***.** Pie Chart

Figure 18 and Figure 19 display the labels through plot figures. According to the count of the labels with three labels represented, each count depicts a column of its label. In Figure 19, the pie chart is divided into three parts, meaning each percentage represents its specific label. As we can see, the DDoS-ACK and the DDoS-PSH-ACK denote 50% of the pie chart because 25% + 25% = 50%. This 50% constitutes a DDoS attack, while the remaining 50% represents a Benign attack. In Figure 19, the count represents the pie plot. Furthermore, in Figure 20, a scatter matrix will be shown below. A scatter matrix will show a grid of scatterplots where each variable is plotted against every other variable in a dataset, as shown in Figure 20.

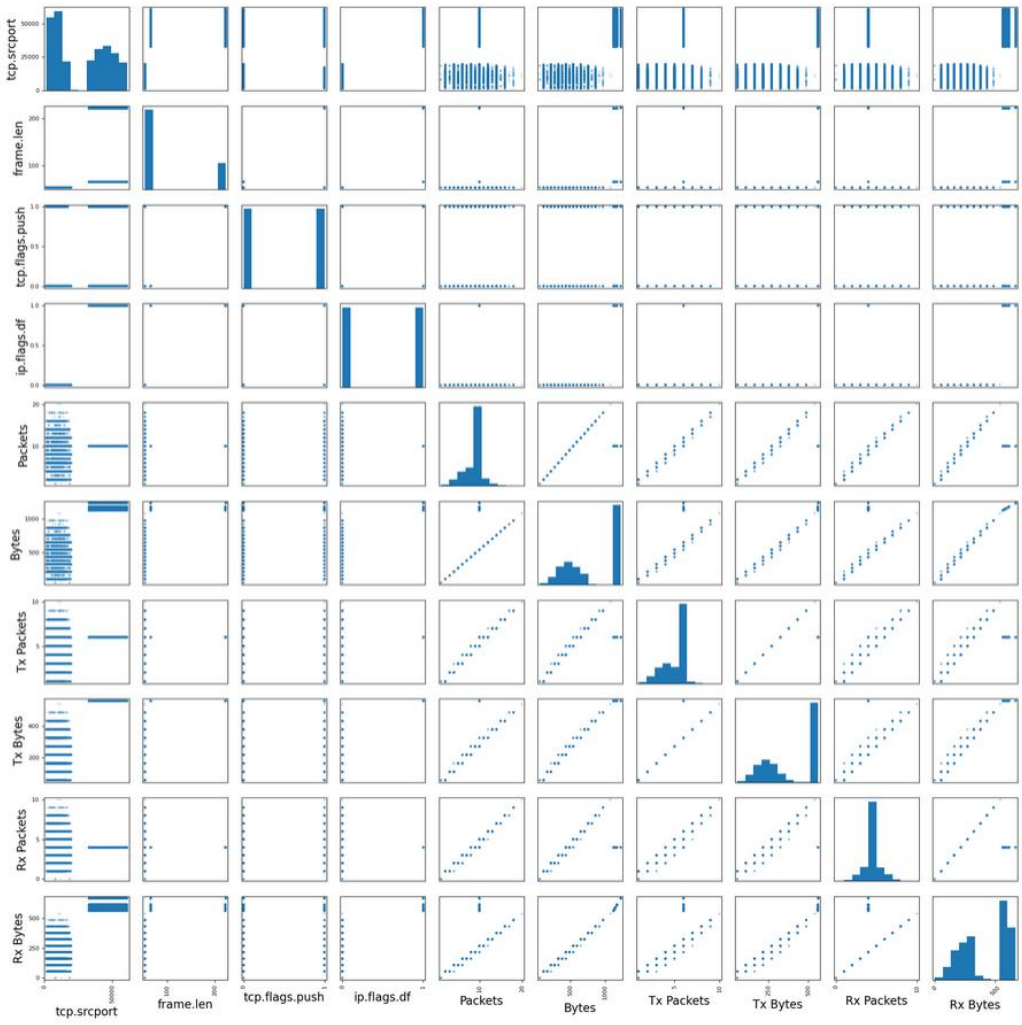


Figure 20. Scatter Matrix of the APA-DDoS Dataset

The second dataset, the CIC-DDoS2019 Dataset, compromises multiple CSV files, as described in Chapter 4. In the second dataset, we will select several EDAs (Exploratory Data Analysis) from different CSV files from the training and testing dataset. It is important to note that, when mentioning EDA, we generally mean that the data has yet to undergo the preprocessing stage and is yet to analyze and display the dataset. These analyses will demonstrate solid and profound ways in which they can help us understand the DDoS attack, make those contributions, and improve predictions in the following sections.

A sample will be chosen from the training dataset, and another sample will be selected from the testing dataset to illustrate the Exploratory Data Analysis (EDA) of Distributed Denial of Service (DDoS) Attacks before and help us determine which are beneficial for our machine learning algorithms.

This sample selected from the training dataset from the UDP-Lag CSV file depicts three labels: UDP-Lag, Benign, and WebDDoS. The counts for each label are 366,461 for UDP-Lag, 3,705 for Benign, and 439 for WebDDoS. The following figures can be shown below to show the counts for the UPD-Lag training CSV file. In Figure 18, we have the counts shown on the axis line. Our count exceeds 350,000, and the percentages for those labels are displayed. Figure 22 shows a bar plot of the UDP-Lag, while Figure 23 displays a scatter plot with just one dot representing the count for each label.

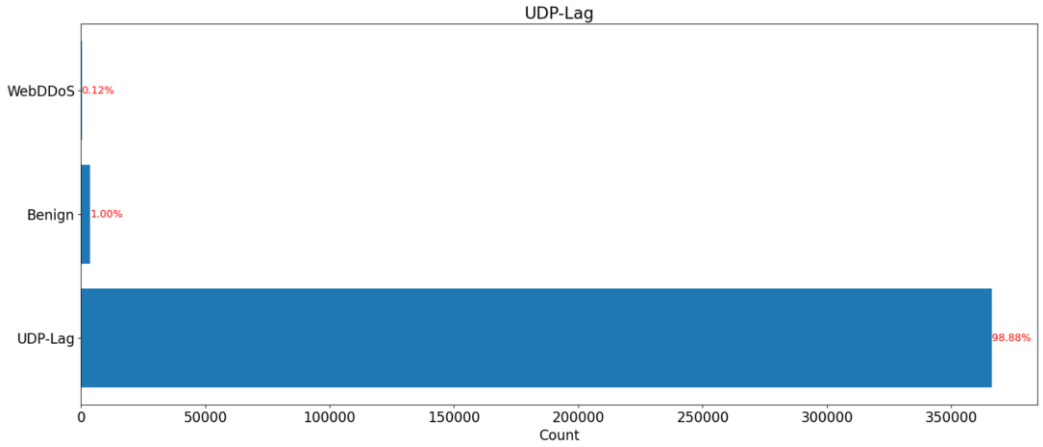


Figure 21. UDP-Lag Count

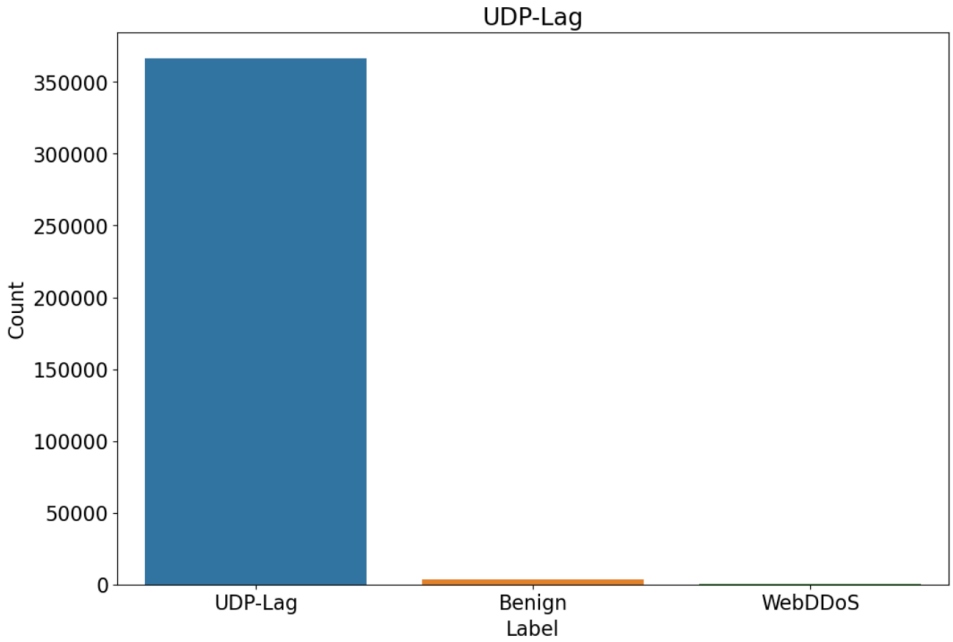


Figure 22. UDP-Lag Bar Plot

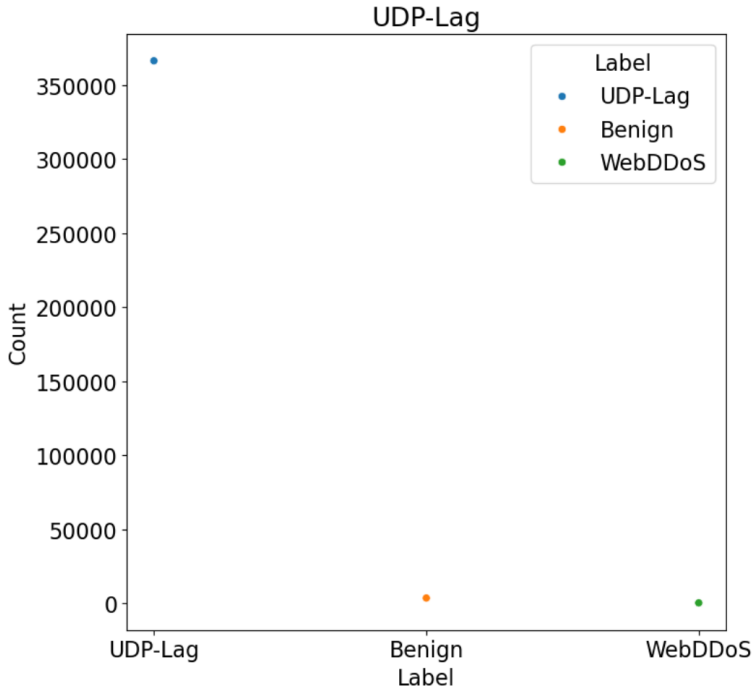


Figure 23. UDP-Lag Scatter Plot

The following sample will be chosen from the testing dataset and display an Exploratory Data Analysis (EDA) of Distributed Denial of Service (DDoS) Attacks. This sample is the LDAP CSV file on the training day, as mentioned in Chapter 4. The LDAP CSV file contains three labels: LDAP, NetBIOS, and Benign. Each label's counts are 1,905,191 for LDAP, 202,919 for NetBIOS, and 5,124 for Benign.

Figure 24 shows a pie chart. The pie chart represents a percentage of the labels count out of one hundred percent. Our count does not exceed 2,000,000 counts, which is normal. LDAP counts 90.2%, Benign 0.2%, and NetBIOS 9.6%. Figure 25 also shows a correlation matrix of the CIC-DDoS2019 training dataset of int64 and float64. The labels are shown in which there are a lot. Also, there are empty cells in the white, meaning missing values.

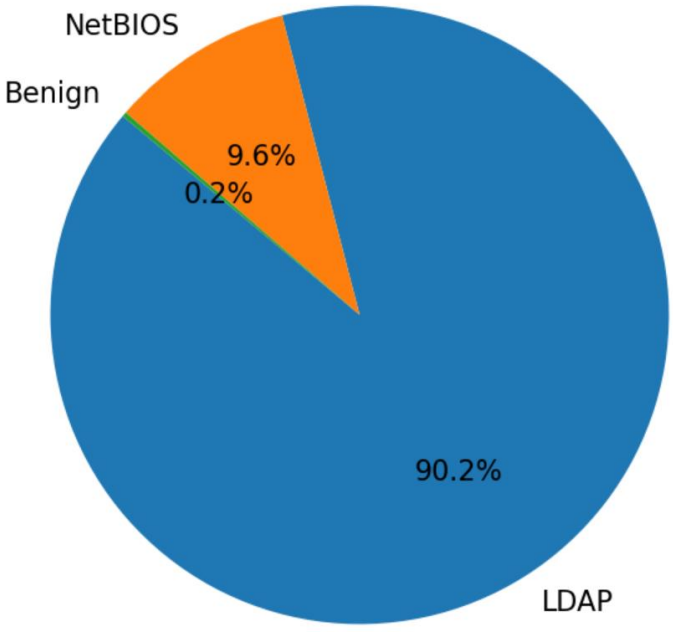


Figure 24. LDAP Pie Chart

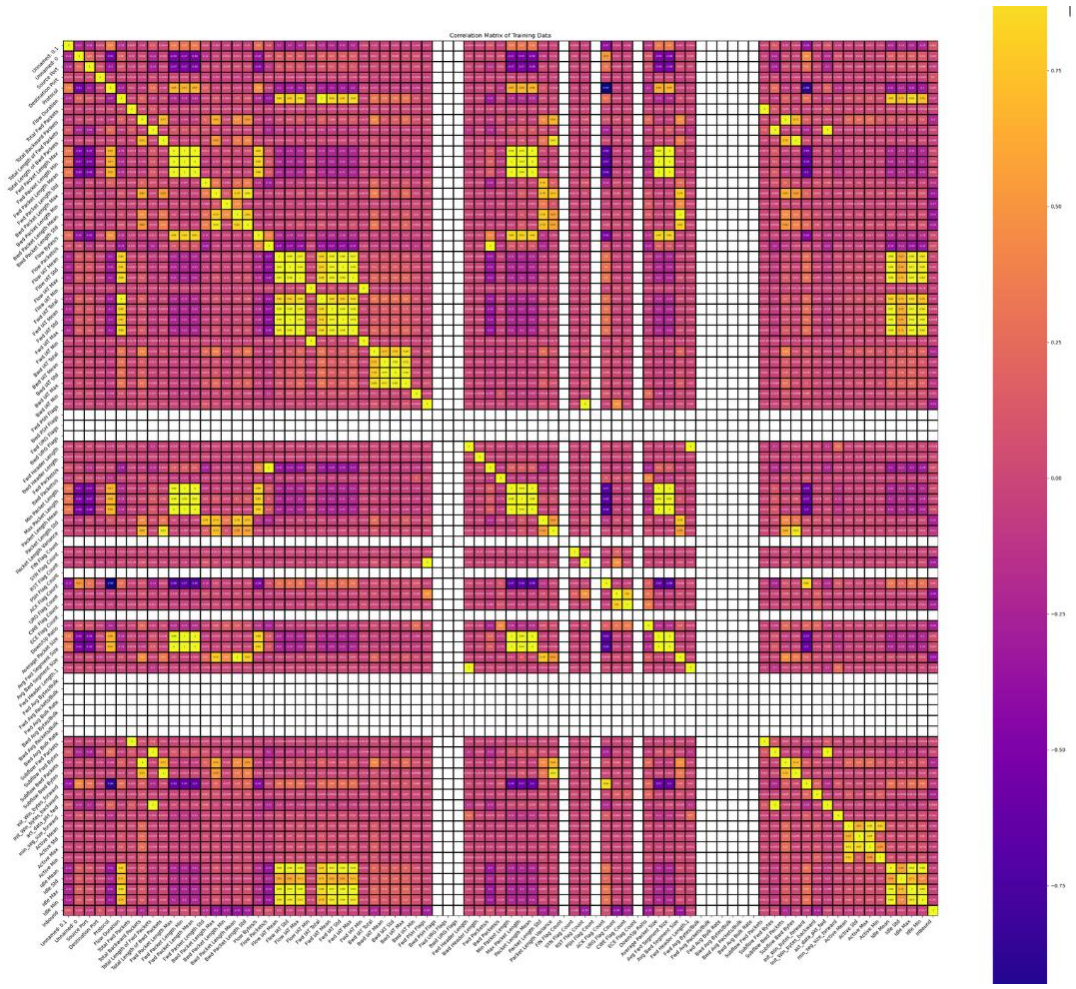


Figure 25. Correlation Matrix of the Training CIC-DDoS2019 Dataset.

Data Preprocessing

Data preprocessing is a crucial step in analyzing our dataset and correcting errors. At the beginning of Chapter 3, we mentioned that an NVIDIA GeForce RTX 3060 and a Currie Cluster were used. The first important step is to load the data in data preprocessing. Specifically, this is an essential step when working with the CSV file in the Jupyter Notebook and working with a machine learning algorithm. After loading the data with a CSV file in the Jupyter Notebook and JupyterLab, the next step involves importing and installing libraries for training and testing data. Once the data has been

loaded, the next step is to check for missing data and handle it. Before doing so, a good approach would be to read the dataframe with ‘pd.read_csv()’ to see what the data looks like before performing data cleaning, as Figures 14, 15, and 16 show an Exploratory Data Analysis example of the APA-DDoS and CICDDoS2019 dataset.

Data Cleaning

Data cleaning is performed to remove inaccurate data and ensure data cleanliness. Both datasets required data-cleaning processes. In the APA-DDoS Dataset, the frame.time column was dropped using the drop() method. In the CIC-DDoS2019 Dataset, the following columns were dropped: the Flow ID, Source IP, Destination IP, Timestamp, SimillarHTTP, and Unnamed: 0. The ‘drop()’ method was used again. Duplicated rows were removed from both datasets. Furthermore, in the APA-DDoS dataset, ip.flags.mf and ip.flags.rb columns were dropped from the dataframe. Subsequently, we copied the dataframe with the dropped columns. Additionally, in the CICDDoS2019, Infinity, 0, NaN, np. inf, -np. inf, and np.nan was replaced. The word np represents the use of numpy as np. Other methods, such as ‘dropna()’ and ‘isnull()’ were also utilized to ensure data cleaning was done accurately. However, in the CIC-DDoS2019 dataset, two columns required data cleaning: Flow Bytes/s and Flow Packet/s.

Feature Engineering

Feature engineering means transforming raw data into a feature vector [48]. In feature engineering, we select the most important features to test, train our machine learning model, and select the best columns. Additionally, we transform and create essential features for use in our model. The APA-DDoS dataset we chose had fourteen feature columns, and the CIC-DDoS2019 dataset had eighty-two for training and testing.

Feature Extraction

The Principal Component Analysis was utilized with both datasets in feature extraction. Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) are feature extraction methods and machine learning methods. However, this paper did not use the LDA mainly because of the imbalanced dataset and most class labels. Principal Component Analysis (PCA) takes a large data set with many variables per observation and reduces them to a smaller set of summary indices [49]. Linear Discriminant Analysis (LDA) is a dimensionality reduction technique commonly used for supervised classification problems; LDA aims to project the dataset onto a lower-dimensional space while maximizing the class separability [50].

Feature Scaling

Feature scaling through standardization, also called Z-score normalization, is an essential preprocessing step for many machine learning algorithms. It is also a critical step in the preprocessing stage of the dataset. Note that different scaling techniques have also been used in many datasets, such as Standard Scaler and Max Abs Scaler.
These datasets use Robust Scaler and Min Max Scaler. Robust scaler, scale features using statistics that are robust to outliers [51]. Min Max Scaler, transform features by scaling each feature to a given range [52]. Using different scaling techniques on different algorithms is often recommended in these datasets as it can lead to different accuracies and prevent them from overfitting. This is because different scaling methods are most suitable for each algorithm, such as supervised and unsupervised neural networks. In addition, they may have different distributions of the input features.

Train Test Split

A train test split is used to evaluate and validate machine learning algorithms. The train-test split usually divides the dataset into two parts: the first to train the model and the second to test its performance.

The APA-DDoS dataset was split into testing and training sets. In training, we split it by 60%; and we split it by 40% because of the test size. This split was achieved using the train-test-split (TTS) function with a test size of 0.4 and a random state of five.

In X_train, 90,720 samples and 14 features are included in the training dataset. y_train, there are 90,720 labels. X_test, there are 60,480 samples and 14 features in the testing dataset. y_test, there are 60,480 labels.

The CIC-DDoS2019, a different training and testing split was applied. The CIC-DDoS2019 was allocated for 30% testing and 70% for training. The random state was set to 42, shuffle was used to shuffle and split the data for both datasets randomly and stratify was set to true.

X_train prints the shape of our training feature dataset. Our training set has 489,995 samples and 82 features, with 489,995 labels corresponding to our training samples. X_test prints the shape of our testing feature dataset. There are 209,999 samples and 82 features in our testing set. y_test prints the shape of our testing label dataset. There are 209,999 labels corresponding to our testing samples.

In the Support Vector Machine, a different approach for the train test split was employed, such as not using 30% testing and 70% training. Instead, 90% are for training and 10% for testing because of the time it takes to test-train, knowing that the Support

Vector Machine takes a longer training time due to higher space complexity. Also, in Cat Boost, 90% are for training and 10% for testing.

Catboost, the following machine learning algorithm, also has a different testing and training split. The shape of our training feature dataset, X_train, is (69,999, 82), meaning that our training set has 69,999 samples and 82 features. Additionally, there are 69,999 labels corresponding to our training samples. The shape of our testing feature dataset, X_test, is (629,995, 82), and there are 629,995 samples and 82 features in our testing set. Also, 629,995 labels correspond to our testing samples.

Evaluation Metrics

Evaluation metrics are utilized in most machine learning models to predict the probabilities associated with a specific target. Evaluation metrics use four cases: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). False Positive and False Negative, in other words, are called Type-1 and Type-2 errors. In True Positive (TP), the actual attacks are classified as attacks; in True Negative (TN), the actual normal is classified as normal. In False Positive (FP), the actual normal is classified as attacks, and in False Negative (FN), actual attacks are classified as normal. The evaluation metric also includes the following: accuracy, precision, detection rate, the false positive rate, the F1-Score, and the recall. A confusion matrix contains information about actual and predicted classifications, which is done by a classification system. To visualize better, we will show a Confusion Matrix below; see Figure 26.

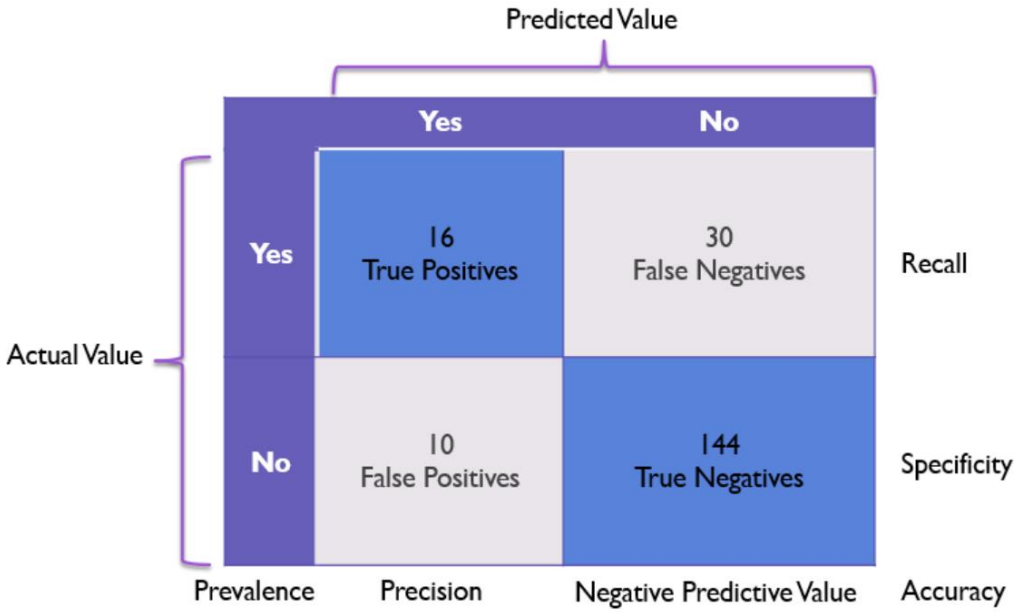


Figure 26. Confusion Matrix [53]

Figure 26 displays a Confusion Matrix, which is prominent in predicting the probabilities and outcomes of specific targets. Specifically, those numbers associated with the confusion matrix categories, such as "Yes" and "No." Conditional probability is used to define the hypothesis.

Furthermore, the evaluation metric simplifies into more meaning than just TP-TF-FP-FN. It also includes more measures such as the Accuracy, Precision, Recall, F-1 score, Detection Rate, and False Positive Rate.

Accuracy is the ratio of the accurate classification over the total number of data records. For example, the formula to calculate the Accuracy as seen in [54].

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{54}$$

Precision is the ratio of the number of actual attacks detected over the total number of attacks classified as attacks. These include normal traffic that is inaccurately

classified to be attacks. The Precision gives the formula to calculate the precision as seen in (54).

$$Precision = \frac{TP}{TP + FP} \quad (54)$$

The Detection Rate is the rate at which the attacks are detected. These actual attacks are detected over the total number of attacks, for example, including those attacks that are classified as normal. The Detection Rate formula as seen in (54).

$$Detection\ Rate = \frac{TP}{TP + FN} \quad (54)$$

The False Positive Rate is the rate at which normal traffic is classified as attacks. This normal traffic is inaccurately classified as attacks over the total number of normal traffic. The False Positive Rate formula as seen in (54).

$$False\ Positive\ Rate = \frac{FP}{FP + FN} \quad (54)$$

The F1 score combines precision and recall computing a single value. Note that the Precision formula and Recall formula are seen below. The F1 Score as seen is (54).

$$F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (54)$$

Recall is also considered to have the same meaning as a detection rate. Recall is given by the formula as seen in (54) is:

$$Recall = \frac{TP}{TP + FN} \quad (54)$$

Furthermore, an AUC (Area Under the Curve) of a ROC (Receiver Operating Characteristic) curve is also an evaluation metric. Generally, in the Area Under the Curve (AUC) of a ROC (Receiver Operating Characteristic) curve, we graph the (TPR) against the False Positive Rate (FPR). In True Positive Rate, we predict the positive class; in False Positive Rate, we predict the negative classes. In the Area Under the Curve of an

ROC (Receiver Operating Characteristic) curve, the line closer to the True Positive Rate is considered good practice because it is closer to the TPR and predicts the True Positive and the positives. The formula given for True Positive Rate (TPR) and False Positive Rate (FPR) is: True Positive Rate formula is $TPR = \frac{TP}{TP + FN}$ and $FPR = \frac{FP}{FP + FN}$ as seen in [54].

A graph will be visualized in Figure 27 to show the AUC (Area Under the Curve) of an ROC (Receiver Operating Characteristic) curve, specifically the True Positive Rate VS False Positive Rate.

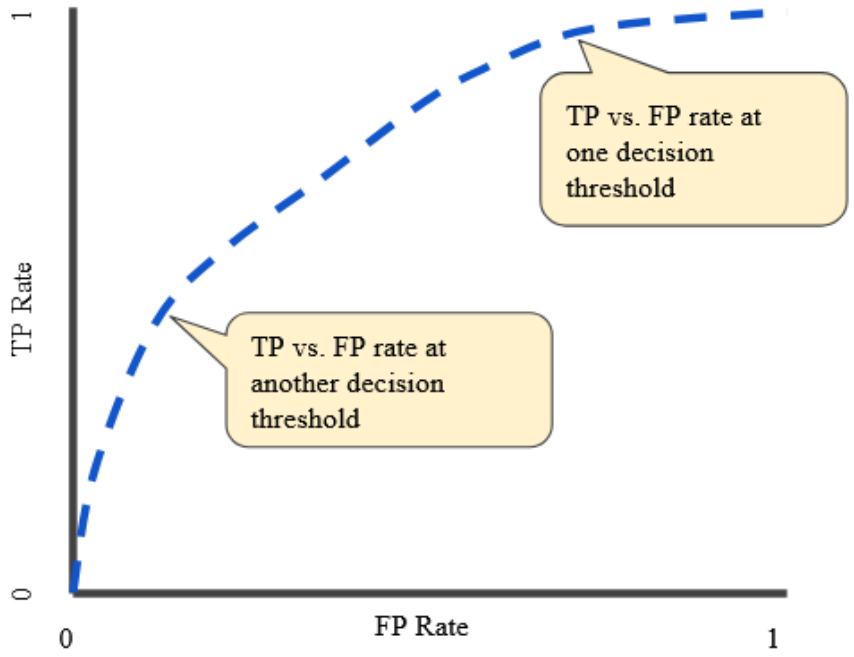


Figure 27. True Positive Rate VS False Positive Rate at Different Classification Thresholds [55]

As shown in Figure 27, the graph states two classification threshold decisions. One classification threshold decision is at another decision threshold. The next is one decision threshold. The importance of the True Positive Rate and False Positive Rate lies in our AUC (Area Under the Curve) of a ROC (Receiver Operating Characteristic) curve.

We need to plot the TPR against FPR at threshold values to compare the Area Under the Curve of different machine learning models and distinguish which machine learning classifier performs best and which performs low.

Next, Figure 28 shows a Measuring Performance in True Positive Rate and False Positive Rate an AUC (Area Under the Curve) of an ROC (Receiver Operating Characteristic) curve.

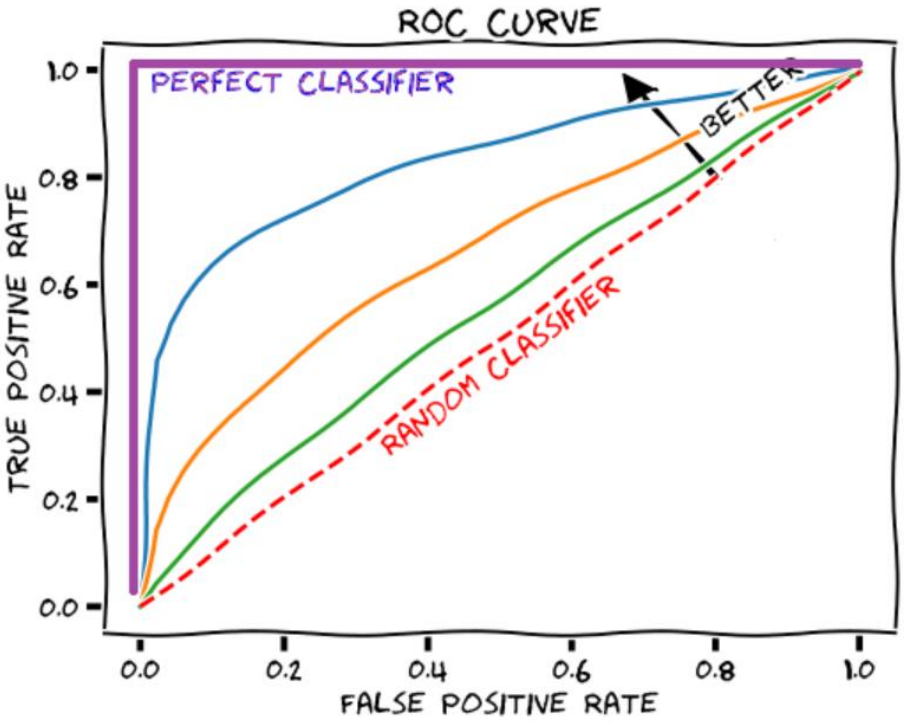


Figure 28. Measuring Performance in True Positive Rate and False Positive Rate an AUC (Area Under the Curve) of an ROC (Receiver Operating Characteristic) Curve [56]

Figure 28 shows a perfect classifier that depicts the purple line as it is the closest to the True Positive Rate. Also, we see blue lines, orange, and green. Lastly, we see the Random Classifier, the red dotted line 0.5, which is the random line. Therefore, based on the Random Classifier, the line above it and closest to the True Positive Rate is considered better. This is because of the probabilities, such as 1 for True and 0 for False.

CHAPTER 4

CIC-DDoS2019 Dataset and their Results

Chapter 4 will discuss the CIC-DDoS Dataset and its results, the hyperparameters associated with each machine learning algorithm, and the neural network. The CIC-DDoS2019 is used for evaluating an Intrusion Detection Systems (IDS). This chapter will classify the machine learning algorithms, specifically DDoS and normal attacks.

The CIC-DDoS2019 dataset is a dataset that was provided by the University of New Brunswick – Canadian Institute for Cybersecurity. This CIC-DDoS2019 Dataset includes multiple Packet Capture (PCAP) files and compromises various comma Separated Files (CSV) files. Regarding the DDoS attacks in this dataset, we usually have reflection and exploitation attacks, which are generally two types of DDoS attacks in this dataset. It is important to note that in a reflection attack, an attacker involves spoofing a target's IP address and sending a request for information [57]; these attacks can be carried out through application layer protocols using transport layer protocols, i.e., Transmission control protocol (TCP), User datagram protocol (UDP) or through a combination of both [58]. The second attack, the exploitation-based attack, where the packets are sent to reflector servers by attackers with the source IP address set to the target victim's IP address to overwhelm the victim with response packets; these attacks can also be carried out through the application layer protocols using transport layer protocols, i.e., TCP and UDP and in TCP-based exploitation attacks include SYN flood and UDP-based attacks include UDP flood and UDP-Lag [58].

In Reflection Attacks, MSQL and SSDP are the common attacks in TCP based attacks; in TCP-UDP based attacks, five DDoS attacks include DNS, LDAP, NETBIOS,

SNMP, PORTMAP, and the last the UDP based attack includes CharGen, NTP, TFTP. The second is exploitation attacks; in that attack, there are two-based attacks: TCP and UDP based. There are SYN flood attacks in TCP based attacks, and in UDP based attacks, there are UDP Flood and UDP-Lag. Figure 24 shows the architecture of the CIC-DDoS2019.

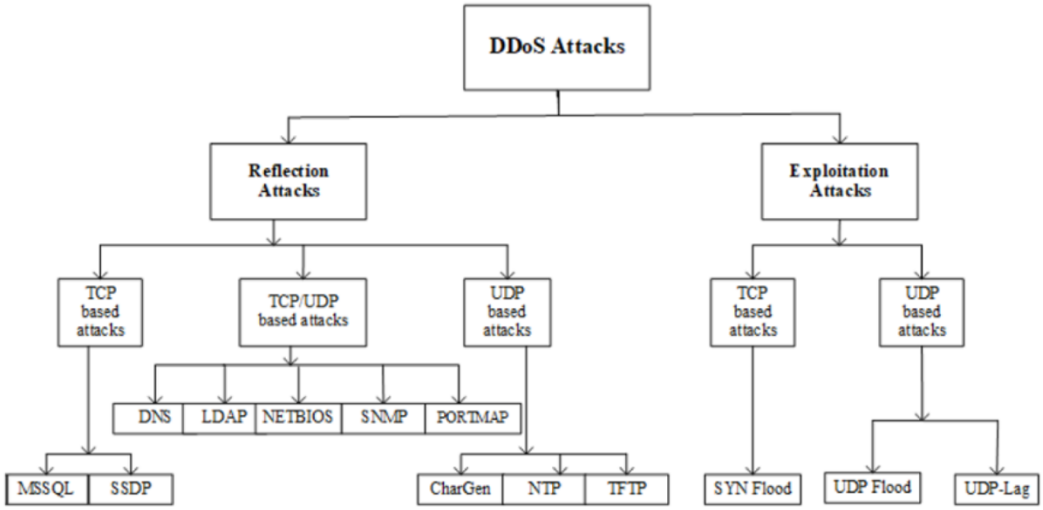


Figure 29. DDoS Attacks [58]

CICDDoS2019 contains benign and the most up-to-date common DDoS attacks, which resembles the true real-world data (PCAPs). It also includes the results of the network traffic analysis using CICFlowMeter-V3 with labeled flows based on the time stamp, source, and destination IPs, source and destination ports, protocols and attack (CSV files) [58]. CICFlowMeter is a network traffic flow generator and analyser [59].

In this paper, the PCAP file is being converted to a CSV file using CICFlowMeter-V3 so that it can be used for the Machine Learning algorithms and Neural Networks. Therefore, CSV files were mostly utilized for testing and training.

Consequently, the dataset size is entirely large, exceeding 20 GB. The CIC-DDoS2019 is an imbalanced dataset that is very challenging, especially when using

neural networks with non-image data. You can use Neural Networks on non-image data, and it works. Note that running the CIC DDoS 2019 will take longer. Training, testing, outputting accuracy, and graphing the Area Under the Curve of the entire dataset will take days to run. The training CSV file dates may vary within the specific times and days, and they are intended to capture traffic analysis flow. There are nineteen compromised CSV files, twelve CSV files for the training dataset, and seven CSV files for the testing dataset. The names of these CSV files are the following: DrDoS_DNS, DrDoS_LDAP, DrDoS_MSSQL, DrDoS_NetBIOS, DrDoS_NTP, DrDoS_SNMP, DrDoS_SSDP, WebDDoS, DrDoS_UDP, Syn, TFTP, WebDDoS, and UDPLag for the training. The following CSV file names for the testing dataset are LDAP, MSSQL, NetBIOS, Portmap, Syn, UDP, and UDPLag. However, it is important to note that there was one missing file, which was the WebDDoS. This WebDDoS CSV file was not found and can be a missing file; therefore, the WebDDoS CSV file was not employed for testing and training. Each CSV file represents a Distributed Denial of Service (DDoS) attack. Table 1 will show the DDoS attacks on training day and testing day. Two days occurred, one for testing the DDoS attack and one for training the DDoS attack, as shown below.

Table 1

DDoS Attacks on Training Day and Testing Day [58]

Days	**Attacks**	**Attack Time**
First Day	PortMap	9:43 - 9:51
	NetBIOS	10:00 - 10:09
	LDAP	10:21 - 10:30
	MSSQL	10:33 - 10:42
	UDP	10:53 - 11:03
	UDP-Lag	11:14 - 11:24
	SYN	11:28 - 17:35
Second Day	NTP	10:35 - 10:45
	DNS	10:52 - 11:05

	LDAP	11:22 - 11:32
	MSSQL	11:36 - 11:45
	NetBIOS	11:50 - 12:00
	SNMP	12:12 - 12:23
	SSDP	12:27 - 12:37
	UDP	12:45 - 13:09
	UDP-Lag	13:11 - 13:15
	WebDDoS (Not used)	13:18 - 13:29
	SYN	13:29 - 13:34
	TFTP	13:35 - 17:15

Table 1 shows twelve DDoS attacks, including NTP, DNS, LDAP, MSSQL, NetBIOS, SNMP, SSDP, UDP, UDP-Lag, WebDDoS, SYN and TFTP, executed on the training day; seven attacks, including PortScan, NetBIOS, LDAP, MSSQL, UDP, UDP-Lag and SYN, executed on the testing day [58], excluding the WebDDoS attack as the DDoS attack file was not found in the CSV (Comma Separated Values) files. In the above Table 1, the attacks names are shown to show what type of attacks occurred in the training and testing days. On the first day, the testing day, there are seven DDoS attacks: PortMap, NetBIOS, LDAP, MSSQL, UDP, UDP-Lag, and SYN. In addition, to better understand these DDoS attacks, they will be described below, along with their definitions and meanings.

DNS

DNS, for short, Domain Name System, has the ability to translate names to IP addresses and back [2]. For example, translating the website's domain name to IP addresses and not the website.

LDAP

Many applications use Lightweight Directory Access Protocol (LDAP); two of the most common are Active Directory and Exchange. LDAP is used to interact with and organize databases [2].

MSSQL

SQL stands for Structure Query Language. MSSQL, part of the Microsoft SQL Server, is a relational database management system (RDBMS). SQL is used in many products, including Microsoft's SQL Server [2].

NetBIOS

NetBIOS stands for Network Basic Input/Output System, where the BIOS is abbreviated. Many years ago, Sytek, known as Hughes LAN Systems (HLS) and IBM (International Business Machines), developed NetBIOS for the LANs [2].

NTP

Network Time Protocol, which stands for NTP, is used to synchronize the clocks across the hosts on a network and uses UDP port 123 for communication purposes [2].

PortMap

Port Map, also known as Port Mapping, enables you to assign different port numbers to signatures that normally detect attacks only on predefined ports [9].

SNMP

The abbreviated word of SNMP stands for, Simple Network Management Protocol (SNMP). SNMP is a protocol used to manage and monitor network devices and hosts [2].

SSDP

Simple Service Discovery Protocol, also known as SSDP. A Simple Service Discovery Protocol (SSDP) attack is a reflection-based distributed denial-of-service (DDoS) attack that exploits Universal Plug and Play (UPnP) networking protocols in order to send an amplified amount of traffic to a targeted victim, overwhelming the target's infrastructure and taking their web resource offline [60].

SYN

Syn stands for synchronize. In Syn, all TCP connections begin with a three-way handshake, which starts with a packet with the SYN control bit set being transmitted to an open port [1].

TFTP

Trivial File Transfer Protocol (TFTP) is used to send files using IP at the Network layer and UDP at the Transport layer [61].

UDP

UDP stands for User Datagram Protocol and is a connectionless transport layer protocol in the TCP/IP protocol stack that simply allows datagrams to be exchanged without acknowledgments or delivery guarantees, requiring other protocols to handle error processing and retransmission [61].

UDP-Lag

The UDP-Lag attack is that kind of attack that disrupts the connection between the client and the server. This attack is mostly used in online gaming where the players want to slow down/ interrupt the movement of other players to outmaneuver them. This attack can be carried in two ways, i.e., using a hardware switch known as a lag switch or

by a software program that runs on the network and hogs the bandwidth of other users [58].

WebDDoS

WebDDoS or Web Distributed Denial of Service is where a website or web applications are associated with a distributed denial of service (DDoS) attack where a distributed denial of service (DDoS) attack is a form of Denial of Service (DoS) attack in which the attack launched against a victim host or network is launched from multiple attacking hosts [9].

Furthermore, we will diverge more profoundly into the CIC-DDoS2019 dataset, as seen in Table 2. Note that the original CIC-DDoS2019 dataset consists of over forty million rows and over eighty columns, which is considered too large to train and test, and therefore, only a sample was used, as stated in Chapter 3.

Table 2

CIC-DDoS2019 Dataset: Feature Number, Feature Names, Data Types, and Description

Feature Number (#)	**Feature Name**	**Data Type**	**Description**
0	Unnamed: 0.1	int64	No name 0.1
1	Unnamed: 0	int64	No name 0
2	Flow ID	object	Type of network address that is associated with a collection of flow interfaces that trace the path of a multipoint or point-to-point connectivity-based service through the device interfaces and connectivities [62].
3	Source IP	object	This field indicates the network and host where the packet originates [1].
4	Source Port	int64	

5	Destination IP	object	This field indicates the network and host where the packet is going [1].
6	Destination Port	int64	
7	Protocol	int64	This field describes the protocol that is being carried by this IP packet. It is often set to a value corresponding to TCP or UDP [1].
8	Timestamp	object	This message type includes the time of the sending machine and request the time of the destination machine [1] .
9	Flow Duration	int64	Duration of the flow in Microsecond [63] [64].
10	Total Fwd Packets	int64	Total packets in the forward direction [63] [64].
11	Total Backward Packets	int64	Total packets in the backward direction [63] [64].
12	Total Length of Fwd Packets	float64	Total size of packet in forward direction [63] [64].
13	Total Length of Bwd Packets	float64	Total size of packet in backward direction [63] [64].
14	Fwd Packet Length Max	float64	Maximum size of packet in forward direction [63] [64].
15	Fwd Packet Length Min	float64	Minimum size of packet in forward direction [63] [64].
16	Fwd Packet Length Mean	float64	Mean size of packet in forward direction [63] [64].
17	Fwd Packet Length Std	float64	Standard deviation size of packet in forward direction [63] [64].
18	Bwd Packet Length Max	float64	Maximum size of packet in backward direction [63] [64].
19	Bwd Packet Length Min	float64	Minimum size of packet in backward direction [63] [64].
20	Bwd Packet Length Mean	float64	Mean size of packet in backward direction [63] [64].
21	Bwd Packet Length Std	float64	Standard deviation size of packet in backward direction [63] [64].
22	Flow Bytes/s	float64	Number of flow bytes per second [63] [64].
23	Flow Packets/s	float64	Number of flow packets per second [63] [64].

24	Flow IAT Mean	float64	Mean time between two packets sent in the flow [63] [64].
25	Flow IAT Std	float64	Standard deviation time between two packets sent in the flow [63] [64].
26	Flow IAT Max	float64	Maximum time between two packets sent in the flow [63] [64].
27	Flow IAT Min	float64	Minimum time between two packets sent in the flow [63] [64].
28	Fwd IAT Total	float64	Total time between two packets sent in the forward direction [63] [64].
29	Fwd IAT Mean	float64	Mean time between two packets sent in the forward direction [63] [64].
30	Fwd IAT Std	float64	Standard deviation time between two packets sent in the forward direction [63] [64].
31	Fwd IAT Max	float64	Maximum time between two packets sent in the forward direction [63] [64].
32	Fwd IAT Min	float64	Minimum time between two packets sent in the forward direction [63] [64].
33	Bwd IAT Total	float64	Total time between two packets sent in the backward direction [63] [64].
34	Bwd IAT Mean	float64	Mean time between two packets sent in the backward direction [63] [64].
35	Bwd IAT Std	float64	Standard deviation time between two packets sent in the backward direction [63] [64].
36	Bwd IAT Max	float64	Maximum time between two packets sent in the backward direction [63] [64].
37	Bwd IAT Min	float64	Minimum time between two packets sent in the backward direction [63] [64].
38	Fwd PSH Flags	int64	Number of times the PSH flag was set in packets travelling

			in the forward direction (0 for UDP) [63] [64].
39	Bwd PSH Flags	int64	Number of times the PSH flag was set in packets travelling in the backward direction (0 for UDP) [63] [64].
40	Fwd URG Flags	int64	Number of times the URG flag was set in packets travelling in the forward direction (0 for UDP) [63] [64].
41	Bwd URG Flags	int64	Number of times the URG flag was set in packets travelling in the backward direction (0 for UDP) [63] [64].
42	Fwd Header Length	int64	Total bytes used for headers in the forward direction [63] [64].
43	Bwd Header Length	int64	Total bytes used for headers in the backward direction [63] [64].
44	Fwd Packets/s	float64	Number of forward packets per second [63] [64].
45	Bwd Packets/s	float64	Number of backward packets per second [63] [64].
46	Min Packet Length	float64	Minimum length of a packet [63] [64].
47	Max Packet Length	float64	Maximum length of a packet [63] [64].
48	Packet Length Mean	float64	Mean length of a packet
49	Packet Length Std	float64	Standard deviation length of a packet [63] [64].
50	Packet Length Variance	float64	Variance length of a packet [63] [64].
51	FIN Flag Count	int64	Number of packets with FIN [63] [64].
52	SYN Flag Count	int64	Number of packets with SYN [63] [64].
53	RST Flag Count	int64	Number of packets with RST [63] [64].
54	PSH Flag Count	int64	Number of packets with PUSH [63] [64].
55	ACK Flag Count	int64	Number of packets with ACK [63] [64].

56	URG Flag Count	int64	Number of packets with URG [63] [64].
57	CWE Flag Count	int64	Number of packets with CWR [63] [64].
58	ECE Flag Count	int64	Number of packets with ECE [63] [64].
59	Down/Up Ratio	float64	Download and upload ratio [63] [64].
60	Average Packet Size	float64	Average size of packet [63] [64].
61	Avg Fwd Segment Size	float64	Average size observed in the forward direction [63] [64].
62	Avg Bwd Segment Size	float64	Average size observed in the backward direction [63] [64].
63	Fwd Header Length.1	int64	Total bytes used for headers in the forward direction [63] [64].
64	Fwd Avg Bytes/Bulk	int64	Average number of bytes bulk rate in the forward direction [63] [64].
65	Fwd Avg Packets/Bulk	int64	Average number of packets bulk rate in the forward direction [63] [64].
66	Fwd Avg Bulk Rate	int64	Average number of bulk rate in the forward direction [63] [64].
67	Bwd Avg Bytes/Bulk	int64	Average number of bytes bulk rate in the backward direction [63] [64].
68	Bwd Avg Packets/Bulk	int64	Average number of packets bulk rate in the backward direction [63] [64].
69	Bwd Avg Bulk Rate	int64	Average number of bulk rate in the backward direction [63] [64].
70	Subflow Fwd Packets	int64	The average number of packets in a sub flow in the forward direction [63] [64].
71	Subflow Fwd Bytes	int64	The average number of bytes in a sub flow in the forward direction [63] [64].
72	Subflow Bwd Packets	int64	The average number of packets in a sub flow in the backward direction [63] [64].

73	Subflow Bwd Bytes	int64	The average number of bytes in a sub flow in the backward direction [63] [64].
74	Init_Win_bytes_forward	int64	The total number of bytes sent in initial window in the forward direction [63] [64].
75	Init_Win_bytes_backward	int64	The total number of bytes sent in initial window in the backward direction [63] [64].
76	act_data_pkt_fwd	int64	Count of packets with at least 1 byte of TCP data payload in the forward direction [63] [64].
77	min_seg_size_forward	int64	Minimum segment size observed in the forward direction [63] [64].
78	Active Mean	float64	Mean time a flow was active before becoming idle [63] [64].
79	Active Std	float64	Standard deviation time a flow was active before becoming idle [63] [64].
80	Active Max	float64	Maximum time a flow was active before becoming idle [63] [64].
81	Active Min	float64	Minimum time a flow was active before becoming idle [63] [64].
82	Idle Mean	float64	Mean time a flow was idle before becoming active [63] [64].
83	Idle Std	float64	Standard deviation time a flow was idle before becoming active [63] [64].
84	Idle Max	float64	Maximum time a flow was idle before becoming active [63] [64].
85	Idle Min	float64	Minimum time a flow was idle before becoming active [63] [64].
86	SimillarHTTP	object	Similar to HyperText Transfer Protocol (HTTP)
87	Inbound	int64	Traffic originates from outside the network [65].
88	Label	object	Target variable for prediction.

The CIC-DDoS2019 Dataset table and its feature number, name, data types, and description are displayed above. The feature names in the CIC-DDoS2019 Dataset represent DDoS attacks, and the data consists of three data types: one, which is an integer 64; two, which is a float 64; and three, which is an object. In the Data Types, there are six objects, thirty-eight integers, and forty-five floats: therefore, this totals 88 feature columns. Int64 is used for whole numbers; usually, these numbers are whole, decimal, and include 64 bits representing integers - floats using 64 bits of storage. Multiple bits exist, including 4, 8, 16, 32, 64, etc. The Object data type is utilized to store names in the object.

In Machine Learning, bits typically represent the numbers used, and using lower bits can reduce the memory required for the training and testing stage. For instance, 32 bits will save more memory than 64 bits, and integers will save more time than floating-point integers. It is essential to consider memory reduction because if bit usage is not minimized, a more extended period will take hours for training and testing due to increased memory usage and computational requirements.

Feature names represent the Distributed Denial of Service (DDoS) attacks. There are eighty-eight feature names, the last of which is the label column. Each column represents a DDoS attack.

All these feature names describe characteristics typically associated with a DDoS attack. For example, IP stands for Internet Protocol (IP). Next, we have Source Port, Destination Port, Protocol, Timestamp, Forward Packets, Backward Packets, Forward Packet Length, Backward Packet Length, Bytes, IAT Count, URG Count, ECE Count, RST Count, Bytes/Bulk, Active, Idle, Inbound, SimiliarHTTP, and many other feature

names and each of these feature names defines how a DDoS attack occurs. In Backward Packets and Forward Packets, packets are transmitted in a unidirectional network traffic flow. Next, we have the forward and backward packet lengths, for example, the size measured in bytes of the packet lengths. Then, we have the Counts: IAT stands for Inter-Arrival Time, URG stands for Urgent Pointer, ECE stands for ECN-Echo, ECN means Explicit Congestion Notification, and RST stands for Reset.

Results

As mentioned in Chapter 2, multiple machine-learning algorithms and neural networks were employed in this paper. This paper will present various results, specifically on accuracy and graphs. Additionally, the hyperparameters will be detailed to explain what leads to better accuracy and overfitting.

Firstly, we will start with traditional machine learning algorithms. The first traditional machine learning algorithm employed was the Decision Tree. Within the Decision Tree classifier, three techniques were utilized: without Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) is, general testing, training, and hyperparameters. The second technique uses Principal Component Analysis, not Synthetic Minority Over-Sampling Technique (SMOTE), whereas the third technique uses Principal Component Analysis and Synthetic Minority Over-Sampling Technique (SMOTE). Each of the three different techniques resulted in different accuracy. SMOTE is a good practice for imbalanced datasets, as our dataset is imbalanced. So, when a dataset presents an imbalance between its classes, that is, an uneven distribution of observations among them, the classification task is inherently more challenging [66]. Moreover, the problem becomes challenging in imbalanced datasets

because multi-class datasets were modified so that the union of one or more classes became the positive class, and the union of one or more of the remaining classes was labeled as the negative class [67]. By using the Synthetic Minority Over-Sampling Technique (SMOTE) with our imbalanced dataset, the problem becomes less complex because SMOTE can result in an overgeneralization of the minority class and several extensions have been proposed to address this issue [66].

Furthermore, the hypermeters will be discussed shortly to show what were used and what accuracy it led to. Also, Chapter 3 mentioned the train test splitting and the number of features used in the CIC-DDoS2019 dataset. The Decision Tree classifier with its hyperparameters is shown in Table 3. In contrast, only the machine learning tools were used to train our model, which is an early stage in the development process.

Table 3

Decision Tree Classifier and their Hyperparameters without Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) in CIC-DDoS2019 Dataset

Name	**Used**	**Description**
random_state	42	Controls both the randomness of the bootstrapping of the samples used when building trees and the sampling of the features to consider when looking for the best split at each node [68].
max_depth	10	The maximum depth of the tree [68].
min_samples_split	5	The minimum number of samples required to split an internal node [68].
min_samples_leaf	15	The minimum number of samples required to be at a leaf node [68].
max_features	10	The number of features to consider when looking for the best split [68].

ccp_alpha	0.01	Complexity parameter used for Minimal Cost-Complexity Pruning [68].
splitter	best	The strategy used to choose the split at each node [68].

Table 3 shows the machine learning tools used to train our model. Thus, the same hyperparameters were used three times to test and train, and all three times led to different accuracies, different cross-validation scores and mean scores, different AUC (Area Under the Curve) of a ROC (Receiver Operating Characteristic) curve and their accuracy, confusion matrix, and other performance metric accuracies. Generally, in cross-validation, learning the parameters of a prediction function and testing it on the same data is a methodological mistake: a model that would just repeat the labels of the samples that it has just seen would have a perfect score but would fail to predict anything useful on yet-unseen data [69].

In the following tables, we will briefly describe their accuracy, cross-validation score, cross-validation mean score, AUC (Area Under the Curve) of an ROC (Receiver Operating Characteristic) curve accuracy score, with PCA and without PCA, and without SMOTE and with SMOTE.

See below in Tables 4, 5, and 6.

Table 4

Decision Tree Classifier Technique 1 without PCA and SMOTE

Decision Tree: Accuracy Score	**Decision Tree: Cross-Validation Score**	**Decision Tree: Cross-Validation Mean Score**	**Decision Tree: ROC-AUC Score**
89.2 %	[0.84424286 0.86924286 0.83965714 0.93904286	87.9%	99.4%

	0.94812783 0.84004057 0.84105487 0.92078458 0.90462721 0.84115487]		

Table 5

Decision Tree Classifier Technique 2 with PCA and without SMOTE

Decision Tree: Accuracy Score	**Decision Tree: Cross-Validation Score**	**Decision Tree: Cross-Validation Mean Score**	**Decision Tree: ROC-AUC Score**
83.53 %	[0.73035714 0.78827143 0.81878571 0.8553 0.80339719 0.79622566 0.84869784 0.8652695 0.88455549 0.7939542]	81.85%	98.1%

Table 6

Decision Tree Classifier Technique 3 with PCA and SMOTE

Decision Tree: Accuracy Score	**Decision Tree: Cross-Validation Score**	**Decision Tree: Cross-Validation Mean Score**	**Decision Tree: ROC-AUC Score**
80.58 %	[0.711 0.79948571 0.80284286 0.81905714 0.80776868 0.84422635 0.80924013 0.84742639 0.82949756 0.79488278]	80.65%	97.9%

In Tables 4, 5, and 6, we see three different techniques, each representing different accuracies, cross-validation, and ROC-AUC scores. Typically, regular testing and training in the first technique perform best as opposed to techniques two and three. However, it's important to note that an ImblearnPipeline was used in our second and third techniques. The ImblearnPipeline has been utilized in most imbalanced learning techniques. In the ImblearnPipeline, a SimpleImputer was utilized with mean; our scaler has been set to MinMaxScaler, PCA with PCA = 82, and our Decision Tree classifier, as stated in Table 3, with its hyperparameters.

Moreover, we will show the AUC (Area Under the Curve) of a ROC (Receiver Operating Characteristic) curve below in Figures 30, 31, and 32 with the three different techniques.

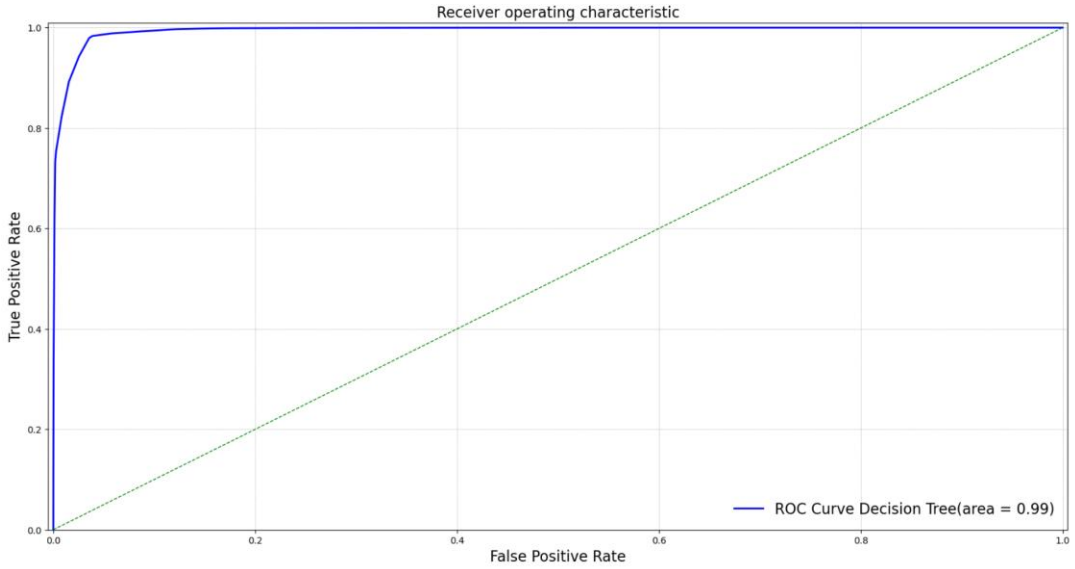


Figure 30. Decision Tree Classifier Technique 1 with AUC-ROC Curve without PCA and SMOTE.

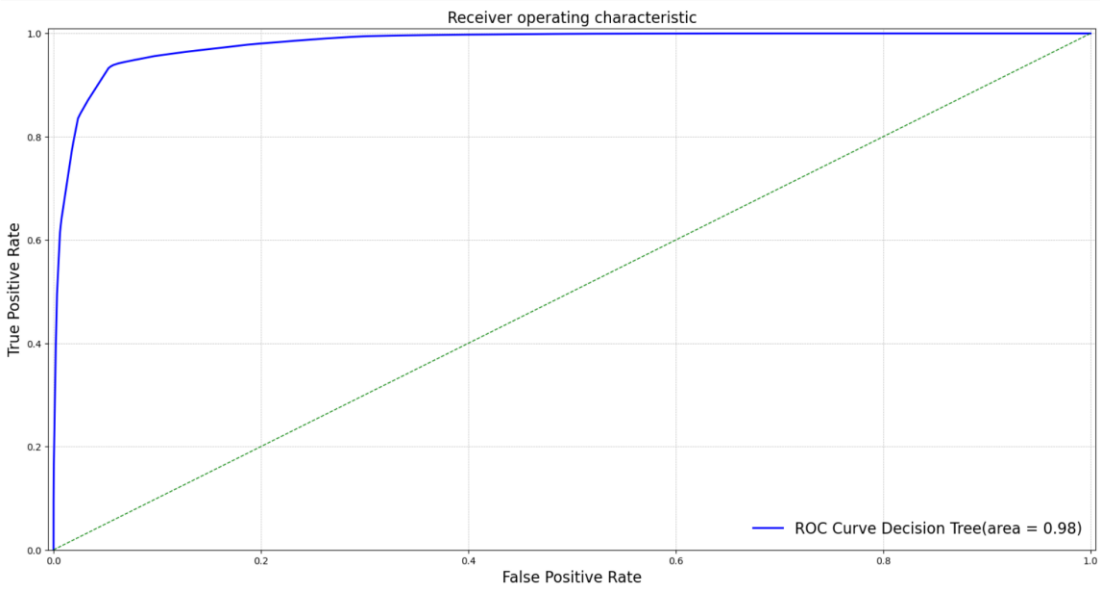


Figure 31. Decision Tree Classifier Technique 2 with AUC-ROC Curve with PCA and without SMOTE.

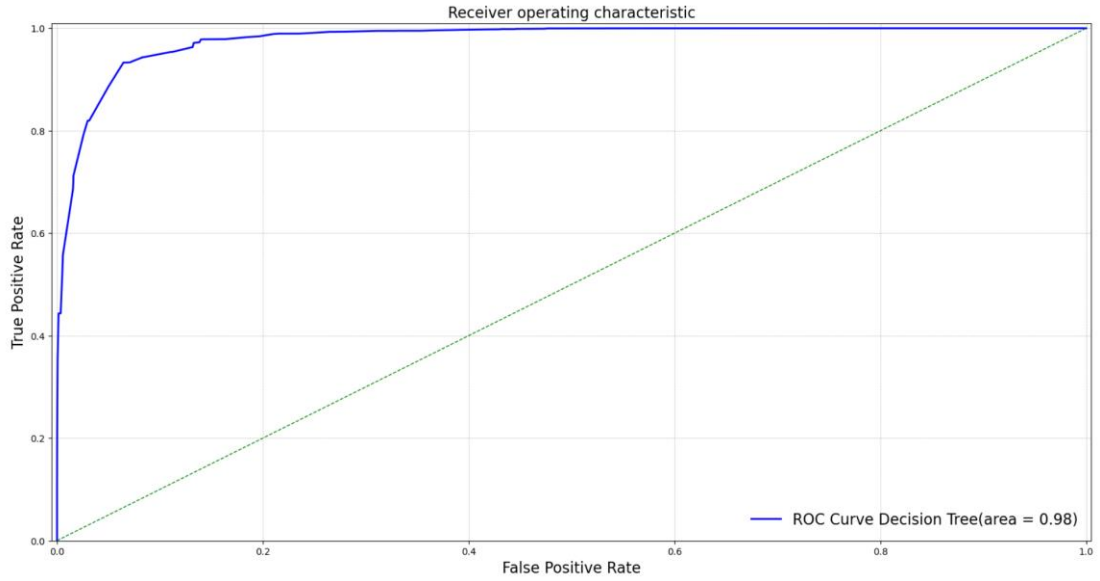


Figure 32. Decision Tree Classifier Technique 3 with AUC-ROC Curve with PCA and with SMOTE.

Figures 30, 31, and 32 depict a Decision Tree classifier with three different techniques using SMOTE and PCA and vice-versa. With Figure 32, we get a better ROC curve as we can see the zig-zag lines, meaning that technique 3 most likely performs

better with SMOTE and PCA and performs a beautiful ROC curve. The area does not reach 1.0, so our ROC curve has overfitted. However, its 0.98 shows a good percentage overall.

In the following figures, we will show the Confusion Matrix and the Performance Metrics to show what leads to high and low accuracies in the three different techniques.

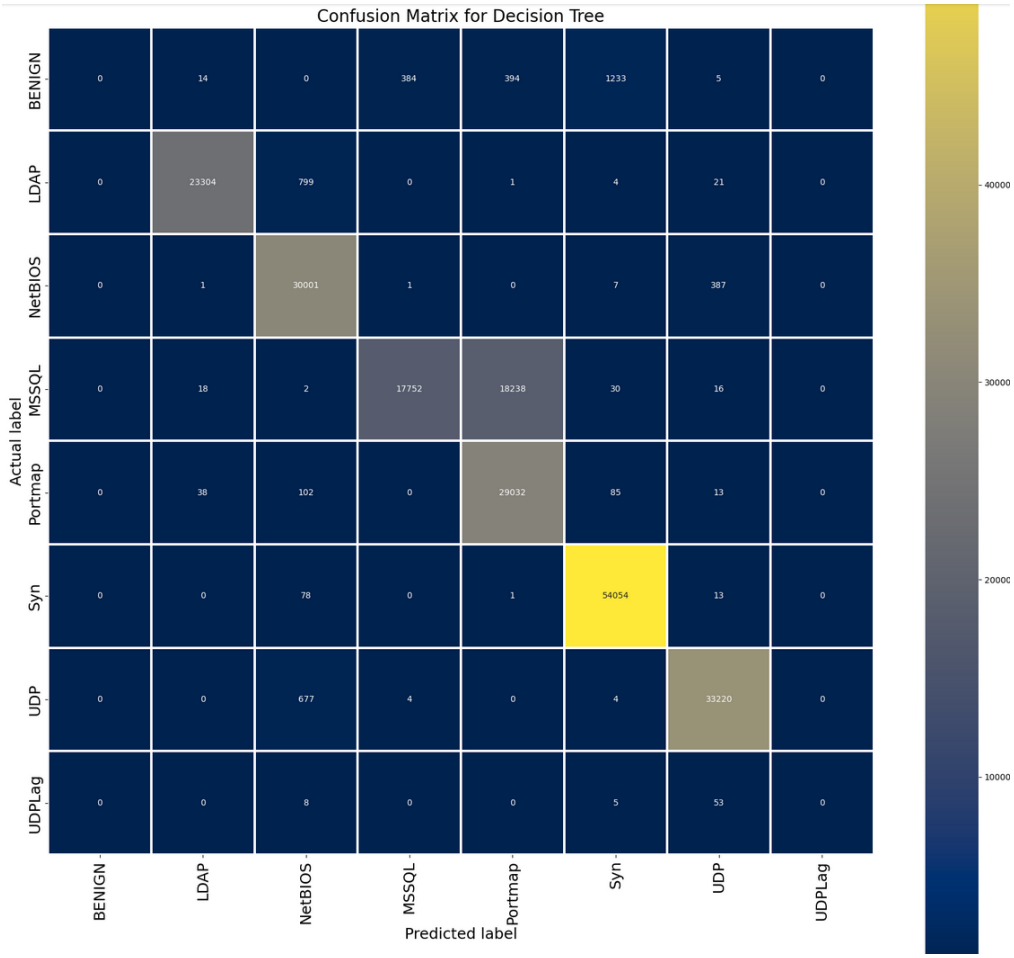


Figure 33. Decision Tree Classifier Technique 1 with Confusion Matrix without PCA and SMOTE.

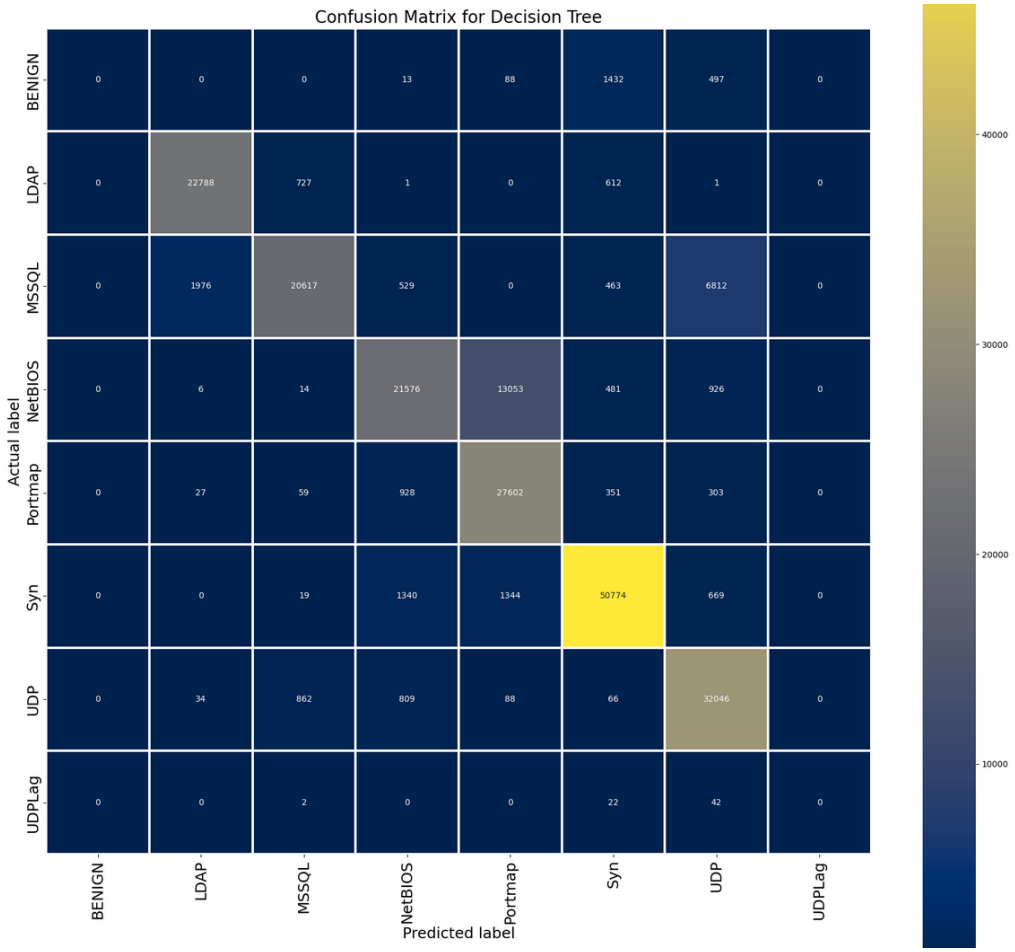


Figure 34. Decision Tree Classifier Technique 2 with Confusion Matrix with PCA and without SMOTE.

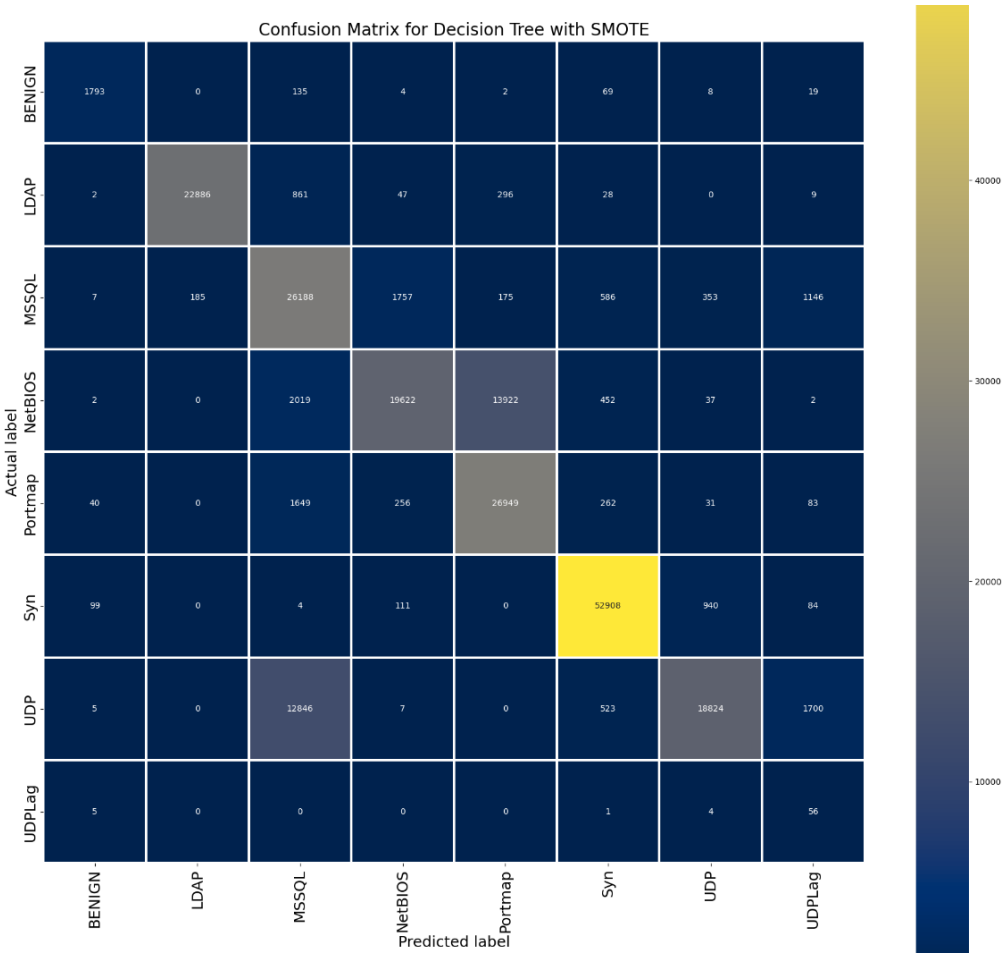


Figure 35. Decision Tree Classifier Technique 3 with Confusion Matrix with PCA and with SMOTE.

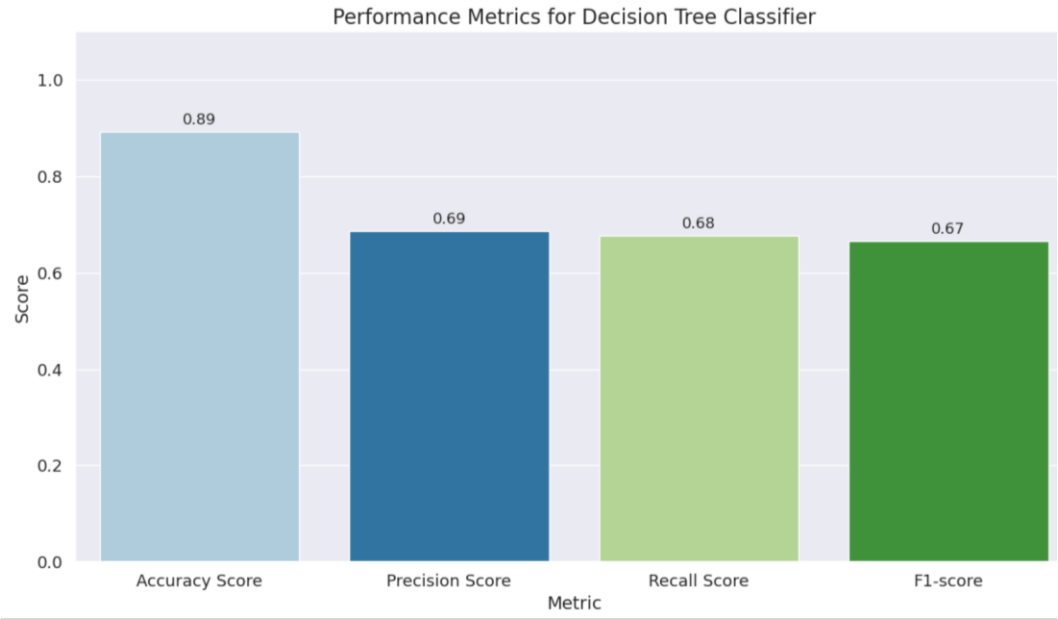


Figure 36. Decision Tree Classifier Technique 1 with Performance Metrics and F1-Score without PCA and SMOTE.

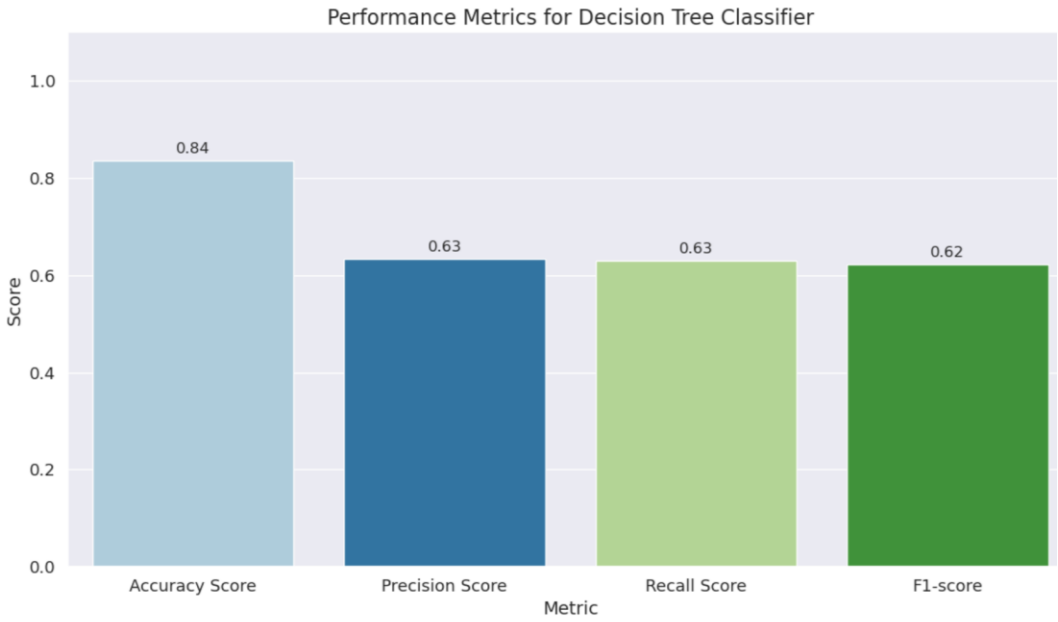


Figure 37. Decision Tree Classifier Technique 2 with Performance Metrics and F1-Score with PCA and without SMOTE.

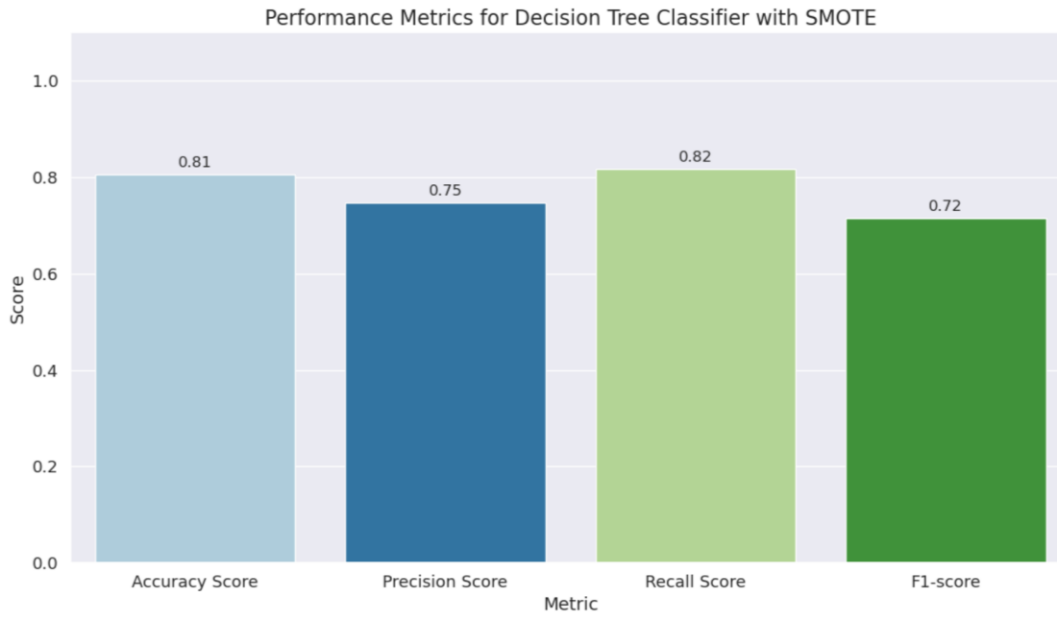


Figure 38. Decision Tree Classifier Technique 3 with Performance Metrics and F1-Score with PCA and SMOTE.

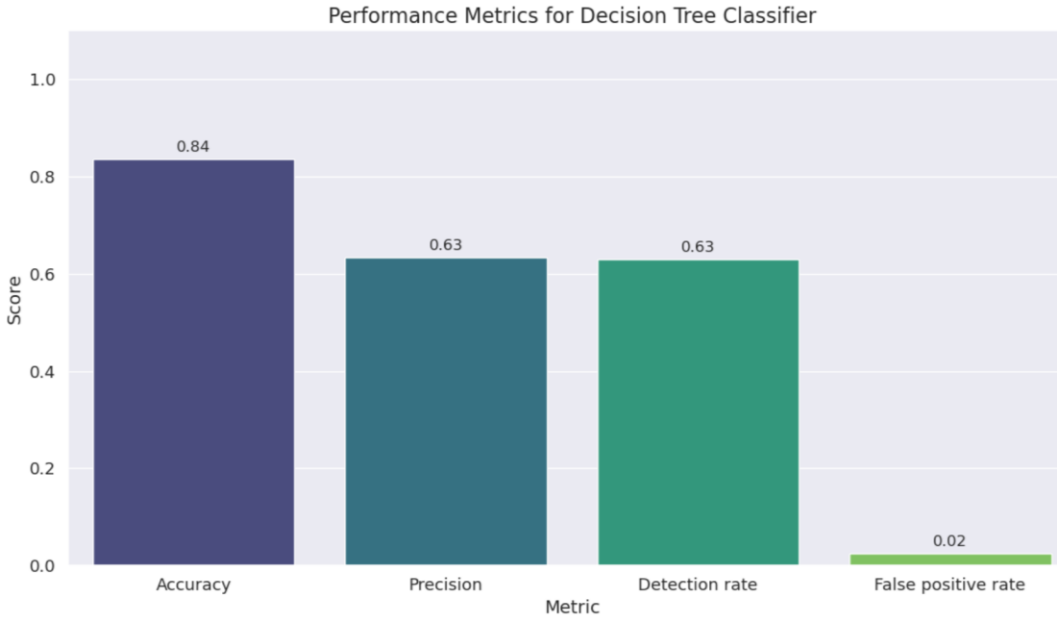


Figure 39. Decision Tree Classifier Technique 1 with Performance Metrics and False Positive Rate without PCA and SMOTE.

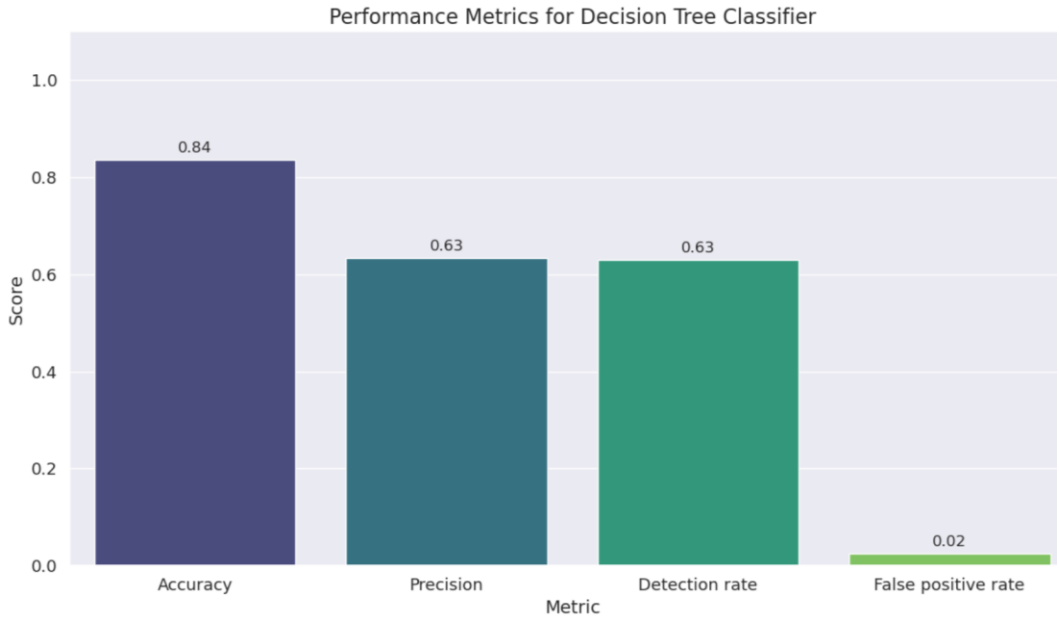


Figure 40. Decision Tree Classifier Technique 2 with Performance Metrics and False Positive Rate with PCA and without SMOTE.

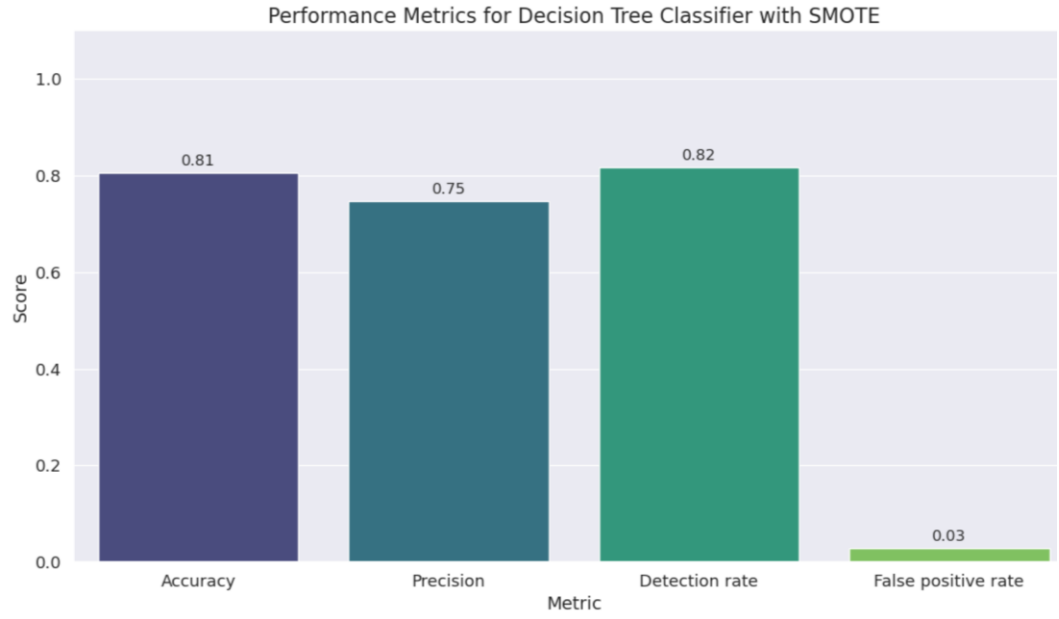


Figure 41. Decision Tree Classifier Technique 3 with Performance Metrics and False Positive Rate with PCA and SMOTE.

Figures 33-35 show a multi-class classification confusion matrix using the three techniques. In addition, the labels are displayed on the left-hand side and underneath the confusion matrix. There are eight labels and benign, which presents non-attack traffic, and the rest are Distributed Denial of Service (DDoS attacks). The seven labels are LDAP, MSSQL, NetBIOS, Portmap, Syn, UDP, and UDP-Lag. We can also see many different numbers and colors in the box. Generally, the confusion matrix has True Positive, True Negative, False Positive, and False Negative.

The following six figures, 36-41, show a performance metric with an F1-score and another performance metric with a false positive rate. In Figure 36, the Decision Tree accuracy score is 89.22%, the precision for the Decision Tree is 68.65%, the recall for the Decision Tree is 67.69%, and the F1-score for the Decision Tree is 66.58%. In Figure 37, the Decision Tree accuracy score is 83.53%, the precision for the Decision Tree is 63.32%, the recall for the Decision Tree is 63.09%, and the F1-score for the Decision Tree is 62.25%. Figure 38, the Decision Tree accuracy score is 80.58%, the precision for the Decision Tree is 74.70%, the Recall for the Decision Tree is 81.74%, and the F1-score for the Decision Tree is 71.51%. Figure 39, the accuracy for the Decision Tree is 89.22%, the precision for the Decision Tree is 68.65%, the detection rate for the Decision Tree is 67.69%, and the false positive rate is 1.58%. In Figure 40, the accuracy for the Decision Tree is 83.53%, the precision for the Decision Tree is 63.32%, the detection rate for the Decision Tree is 63.09%, and the false positive rate is 2.45%. Lastly, the accuracy for the Decision Tree is 80.58%, the precision for the Decision Tree is 74.70%, the detection rate for the Decision Tree is 81.74%, and the false positive rate is 2.83% in Figure 41.

The second traditional machine learning algorithm utilized was CatBoost, followed by the third being Support Vector Machine, the fourth being K-Nearest Neighbor, and the fifth being Random Forest. In these machine learning algorithms, we will not use the three techniques as we did in Decision Tree. Instead, we will only opt for one technique rather than multiple techniques and within the use of PCA and/or SMOTE.

CatBoost utilizes gradient boosting on decision trees to mitigate the model's overfitting, generalizing to unseen data. Within the CatBoost classifier, we will describe the machine learning tools, specifically the hyperparameters.

Table 7

CatBoost Classifier and their Hyperparameters with Principal Component Analysis (PCA)

Name	**Used**	**Description**
iterations	100	CatBoost builds number of trees which refers to the number of iterations [70].
learning_rate	0.1	Used for reducing the gradient step [71]. The learning rate used for training [72].
depth	5	Depth of the trees [71].
loss_function	MultiClass	The metric to use in training [73].
eval_metric	MultiClass	Calculate the specified metric on raw approximated values of the formula and label values [72].
random_seed	42	Used for training and setting the seed for the random number generator [72]. The random seed used for training [72].

In the CatBoost classifier and its hypermeters, the loss_function and eval_metric were set to multiclass because we are using multiclass classification; therefore, it is crucial to use the specific parameter for the machine learning algorithm. Also, a learning rate of 0.1 was used to determine the step size during the gradient descent optimization. Again, ImblearnPipeline was used, and a SimpleImputer with mean was utilized; the scaler was set to MinMaxScaler, PCA with PCA = 82.

CatBoost classifier with PCA will be shown below, along with the accuracy score, the cross-validation score, and the ROC-AUC score. See below in Table 8.

Table 8

CatBoost Classifier with Principal Component Analysis (PCA) in CIC-DDoS2019 Dataset

CatBoost: Accuracy Score	**CatBoost: Cross-Validation Score**	**CatBoost: Cross-Validation Mean Score**	**CatBoost: ROC-AUC Score**
92.7 %	[0.82378571 0.86318571 0.94924286 0.9479 0.9481564 0.94977071 0.95095644 0.95042786 0.94807069 0.95108502]	92.8%	99.8%

As shown above, in Table 8, the CatBoost accuracy score, cross-validation mean score, and ROC-AUC score were above 90%, indicating outstanding performance. This is one of the reasons why CatBoost performs exceptionally well.

In addition to Table 8, we will display a multi-class classification ROC-AUC curve. In this ROC-AUC curve, we will see the labels mentioned in the Decision Tree that there are eight labels, and we also have a micro-average in the ROC-AUC curve.

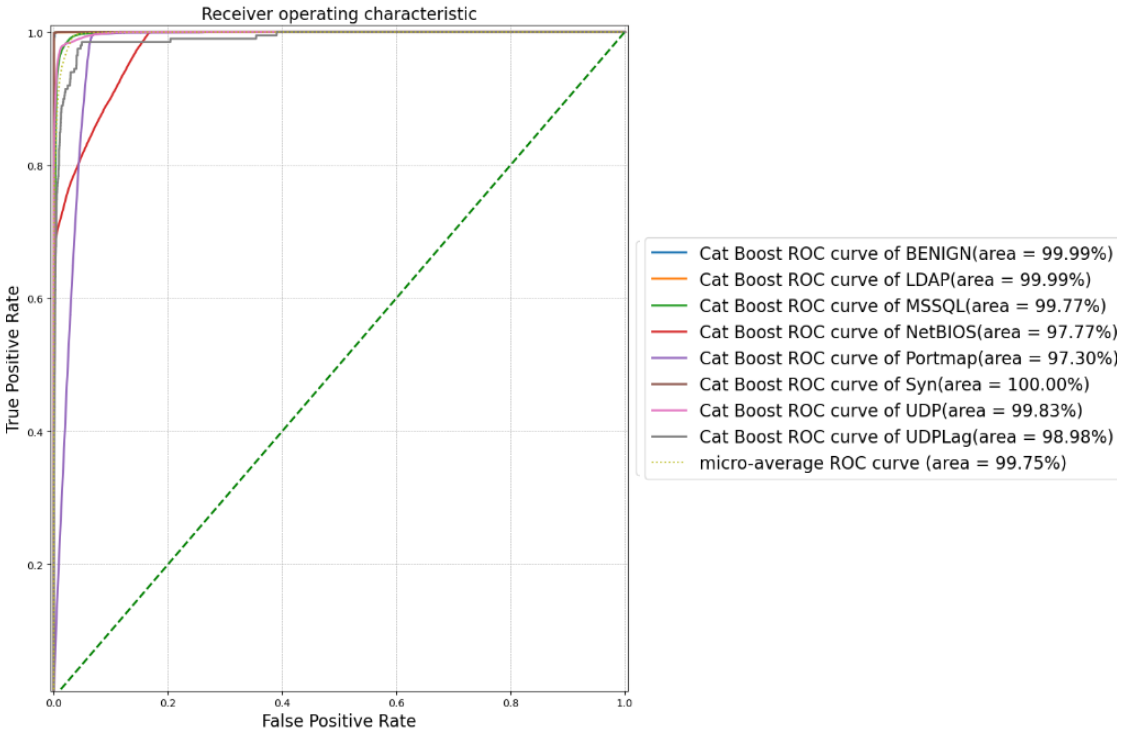


Figure 42. ROC-AUC Curve of a Multi-Class Classification in CatBoost.

Figure 42 shows a multi-class classification ROC-AUC curve. In addition, the percentage scores of each label are demonstrated with its color. The labels specified range from 97% - 100%. The graph shows a 100.00% ROC curve, which indicates overfitting, which is not a good fit and/or practice. Overfitting usually occurs when the model fits the training data too closely as this is not best practice. It also results in poor performance on new and unseen data.

The confusion matrix and the performance metrics will be shown below in Figures 43, 44, and 45.

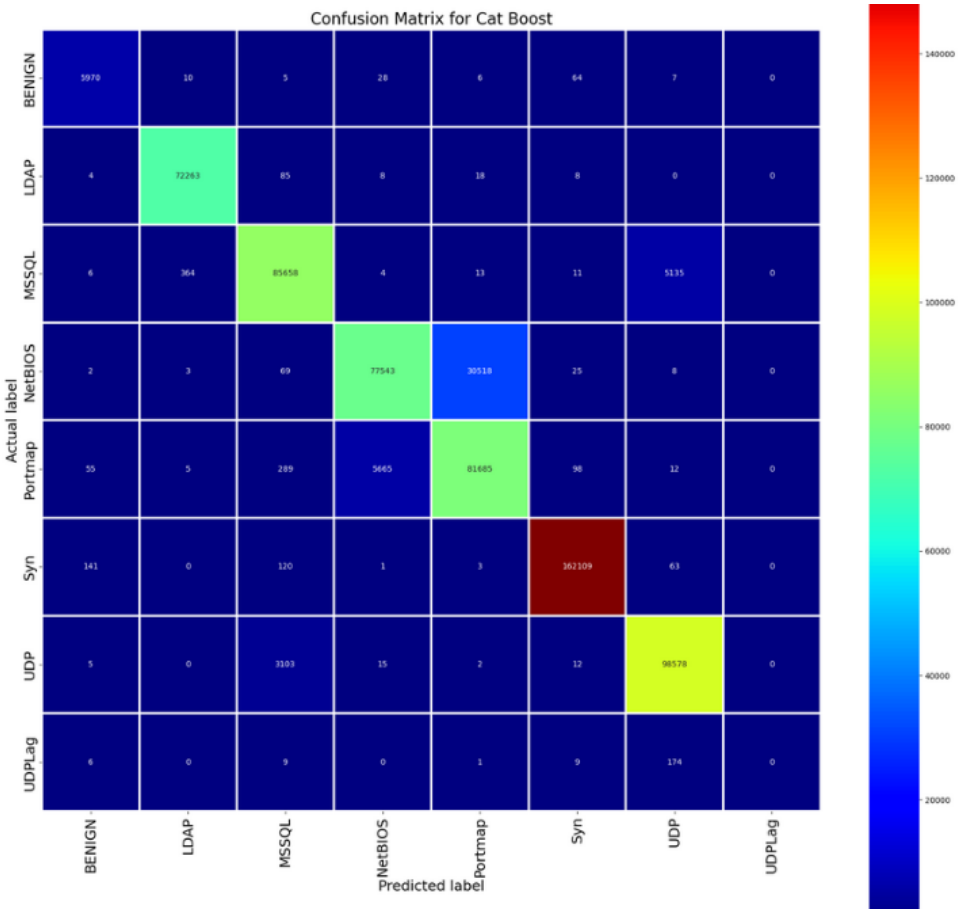


Figure 43. CatBoost Classifier with Confusion Matrix.

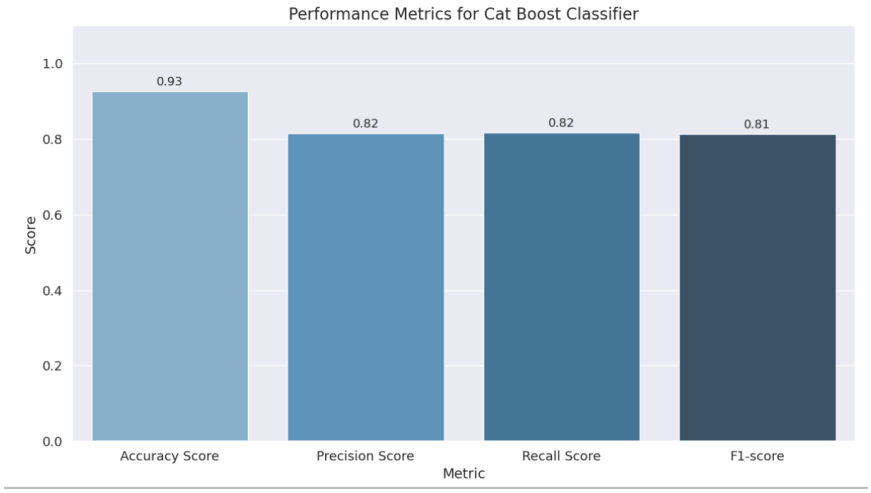


Figure 44. CatBoost Classifier with Performance Metrics and F1-Score.

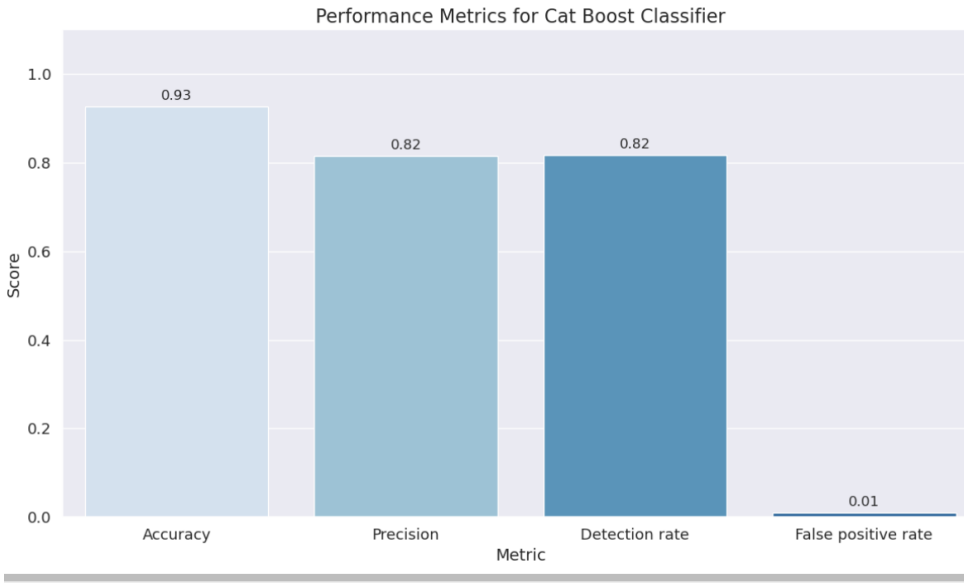


Figure 45. CatBoost Classifier with Performance Metrics and False Positive Rate.

Figure 43 shows a CatBoost Classifier with a confusion matrix, whereas, in Figures 44 and 45, we have performance metrics and F1-score and false positive rate. In the confusion matrix, we can see many true positive indicating positive classes. Figure 45 shows that the false positive rate is 0.01, meaning that normal traffic is classified as an attack. The red box with Syn is the highest true positive in the confusion matrix. We can also see benign performing true positive as benign representing non-attack traffic.

Support Vector Machine utilizes the hyperplane, along with the kernel trick, to classify the data points by separating the negative and positive classes. In Table 9, Support Vector Machine Classifiers and their Hyperparameters with Principal Component Analysis (PCA) and SMOTE will be displayed.

Table 9

Support Vector Machine Classifier and their Hyperparameters with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE)

Name	**Used**	**Description**
C	1.0	Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive [74].
kernel	rbf	Specifies the kernel type to be used in the algorithm [74].
gamma	0.0005	Kernel coefficient for 'rbf', 'poly' and 'sigmoid' [74].
random_state	42	Controls the pseudo random number generation for shuffling the data for probability estimates [74].
probability	True	Whether to enable probability estimates [74].

As mentioned in Chapter 2, the Support Vector Machine takes a very long time to run. And because of that, cross-validation was not used. Generally, even if we used it, it would take forever to run because we needed a mighty PC and a CPU-GPU.

Table 10

Support Vector Machine Classifier with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE)

Support Vector Machine: Accuracy Score	**Support Vector Machine: ROC-AUC Score**
78.9 %	98.9%

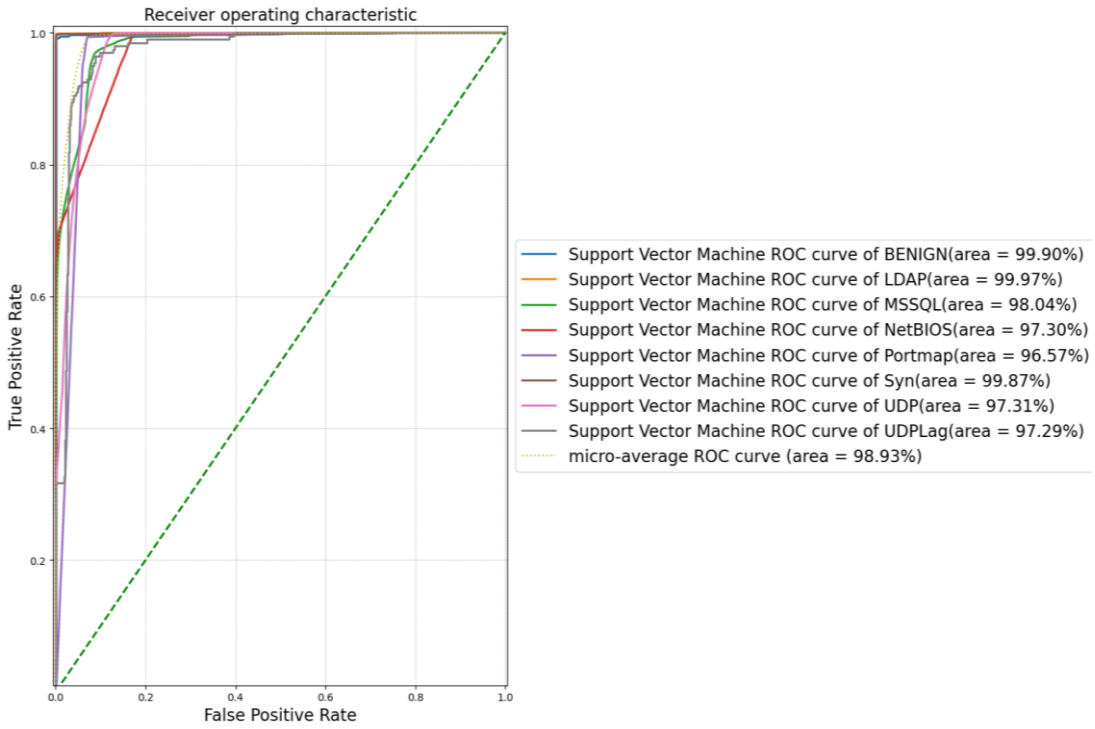


Figure 46. ROC-AUC Curve of a Multi-Class Classification in Support Vector Machine.

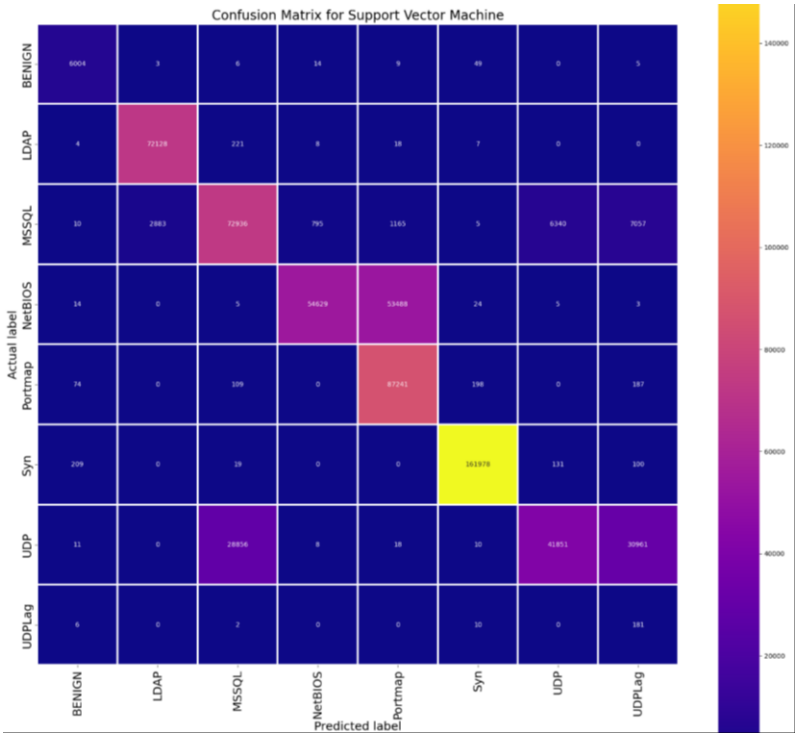


Figure 47. Support Vector Machine Classifier with Confusion Matrix.

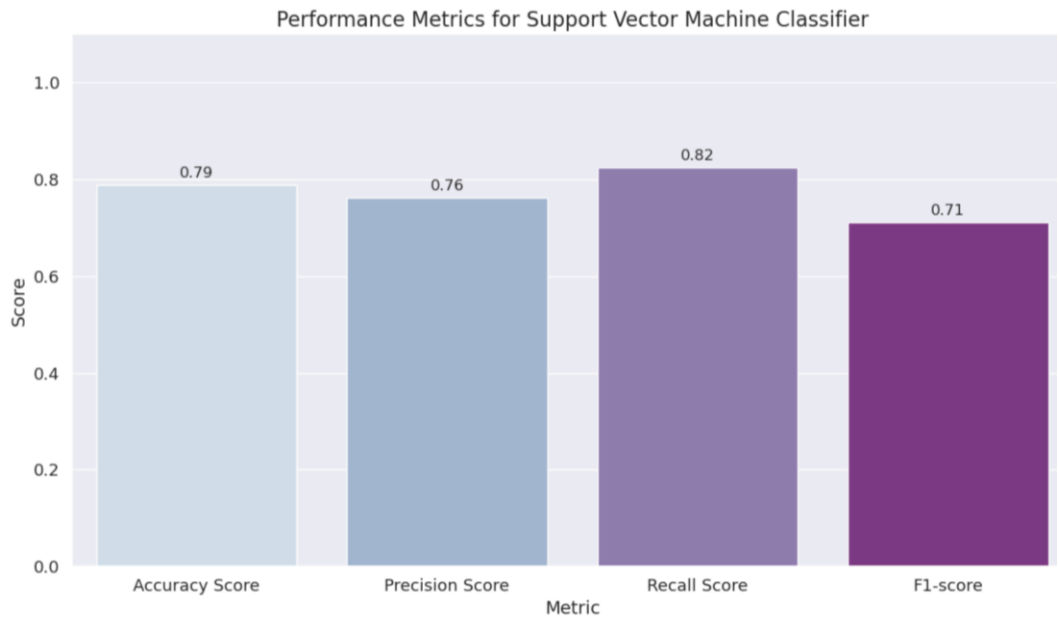


Figure 48. Support Vector Machine Classifier with Performance Metrics and F1-Score.

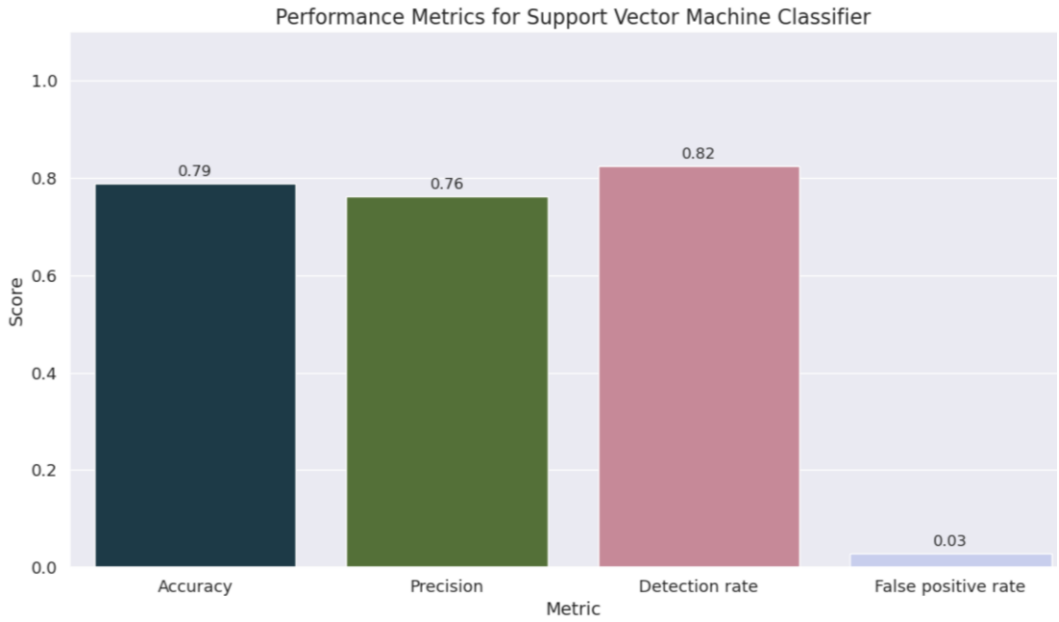


Figure 49. Support Vector Machine Classifier with Performance Metrics and False Positive Rate.

The accuracy score for the Support Vector Machine was 79%, meaning that the accuracy did perform well, maybe not the best, but it did perform over 75%, which is

good practice. The ROC-AUC curve was in the high 90s, and the confusion matrix performed excellent. As we can see, not only did it predict true positives, but 0 means that no predictions were predicted. However, Figures 48 and 49 show that recall score and detection rate have the highest percentage.

The following traditional machine learning algorithm is the K-Nearest Neighbor (KNN). K-Nearest Neighbor (KNN) is used for classification and regression and is easy to implement. In this algorithm, we will show the hyperparameters along with their descriptions to show the meaning of what each name in the K-Nearest Neighbor does. A KNN of 5 was used, along with SMOTE with a random state of 42 and PCA set to 82. See below for the KNN hyperparameters.

Table 11

K-Nearest Neighbor Classifier and their Hyperparameters with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE)

Name	**Used**	**Description**
n_neighbors	5	Number of neighbors to use [75].
weights	uniform	Uniform weights. All points in each neighborhood are weighted equally [75].
algorithm	brute	Brute will use a brute-force search [75].
n_jobs	-1	The number of parallel jobs to run for neighbors' search [75].

Table 12

K-Nearest Neighbor Classifier with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE)

K-Nearest Neighbor: Accuracy Score	**K-Nearest Neighbor: Cross-Validation Score**	**K-Nearest Neighbor: Cross-Validation Mean Score**	**K-Nearest Neighbor: ROC-AUC Score**
92.0 %	[0.83312857 0.8686 0.939 0.93862857 0.93767054 0.9379134 0.93787054 0.93818483 0.93785626 0.9310133]	92.0%	98.4%

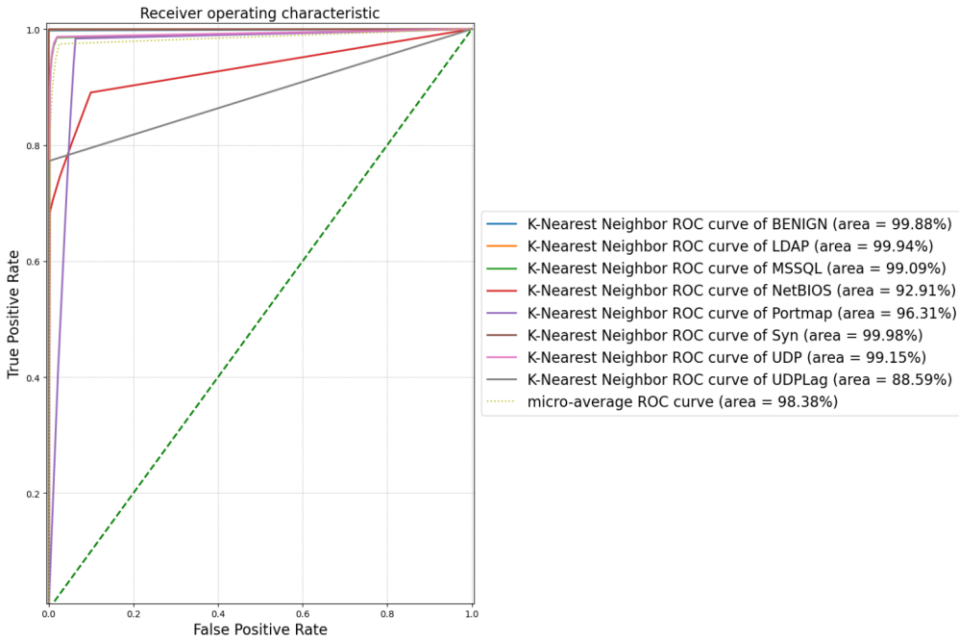


Figure 50. ROC-AUC Curve of a Multi-Class Classification in K-Nearest Neighbor.

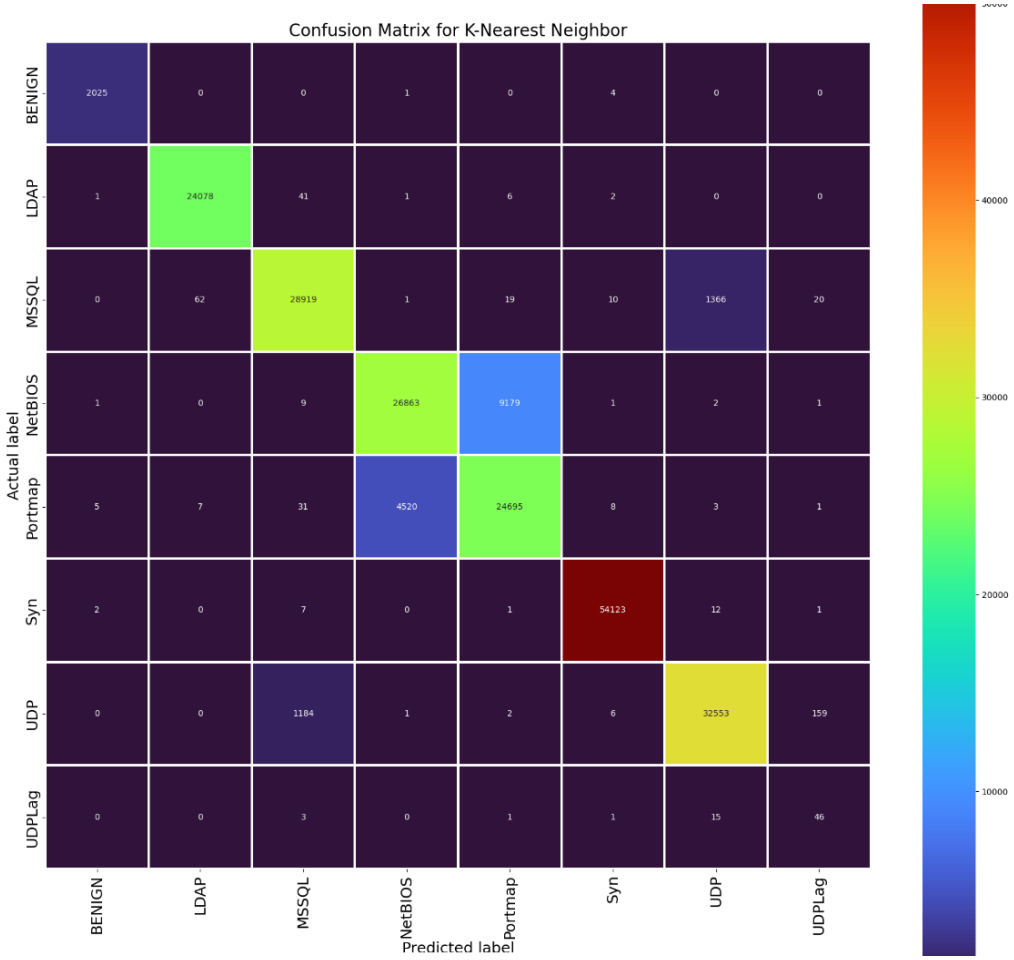


Figure 51. K-Nearest Neighbor Classifier with Confusion Matrix.

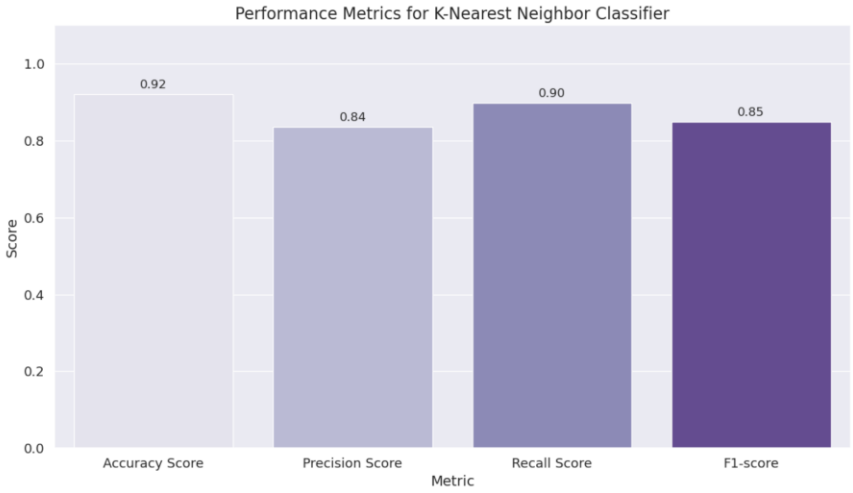


Figure 52. K-Nearest Neighbor Classifier with Performance Metrics and F1-Score.

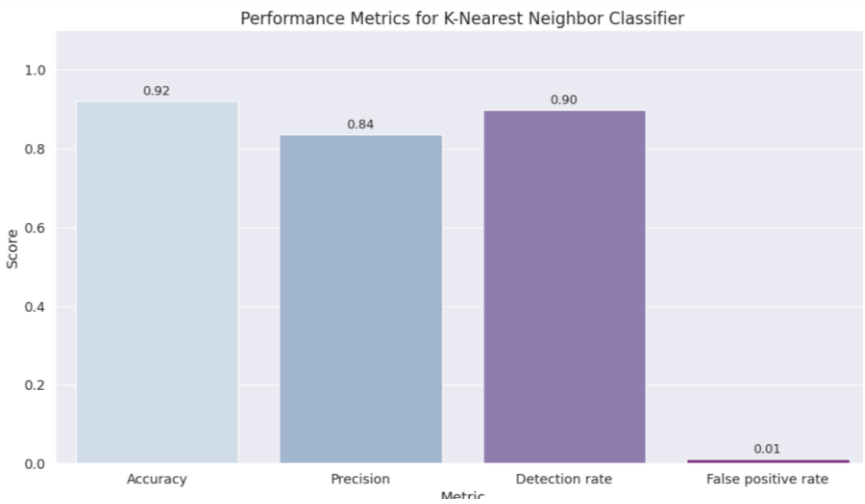


Figure 53. K-Nearest Neighbor Classifier with Performance Metrics and False Positive Rate.

Figures 50-53 display a confusion matrix, ROC-AUC curve, and the Performance Metrics and False Positive Rate and F1-Score. As shown above, the detection rate and recall score have the second-highest scores. The false positive rate is 0.1 %. Also, in the confusion matrix, the TP are all classified.

The last traditional machine learning algorithm is the Random Forest. Random Forest uses ensemble learning methods to vote for the most popular class. In the Random Forest, the PCA was used with PCA being set to 82, and the scale used was the MinMaxScaler and SimpleImputer with mean and within an ImblearnPipeline. In Table 13, we will display the Random Forest classifier and their hyperparameters where only two were used and with Principal Component Analysis (PCA).

Table 13

Random Forest Classifier and their Hyperparameters with Principal Component Analysis (PCA)

Name	**Used**	**Description**
bootstrap	True	Whether bootstrap samples are used when building trees [76].
random_state	42	Controls both the randomness of the bootstrapping of the samples used when building trees and the sampling of the features to consider when looking for the best split at each node [76].

Table 14

Random Forest Classifier with Principal Component Analysis (PCA)

Random Forest: Accuracy Score	**Random Forest: Cross-Validation Score**	**Random Forest: Cross-Validation Mean Score**	**Random Forest: ROC-AUC Score**
93.5 %	[0.84814286 0.88362857 0.95267143 0.95155714 0.95117073 0.95102787 0.95117073 0.95224217 0.95235646 0.94282775	93.4%	99.8%

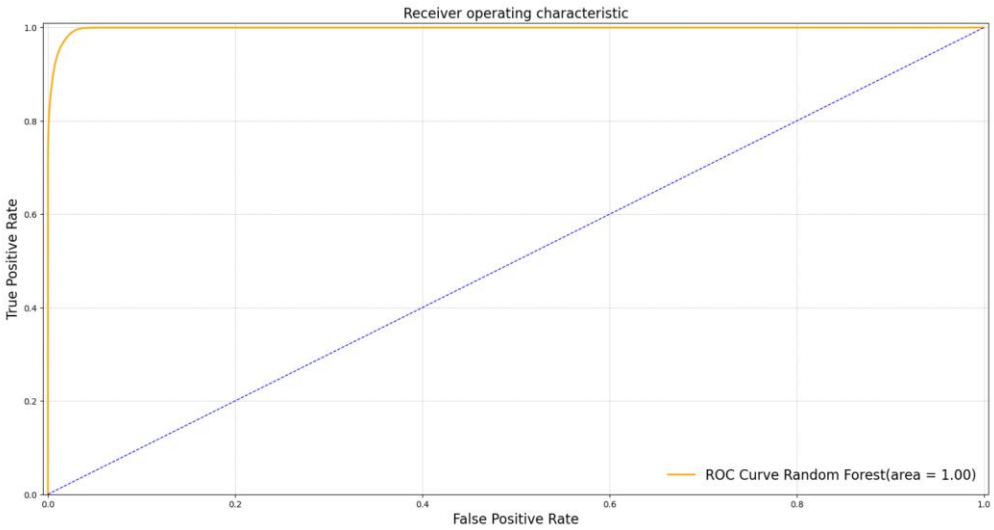


Figure 54. Random Forest Classifier with AUC-ROC Curve and with PCA.

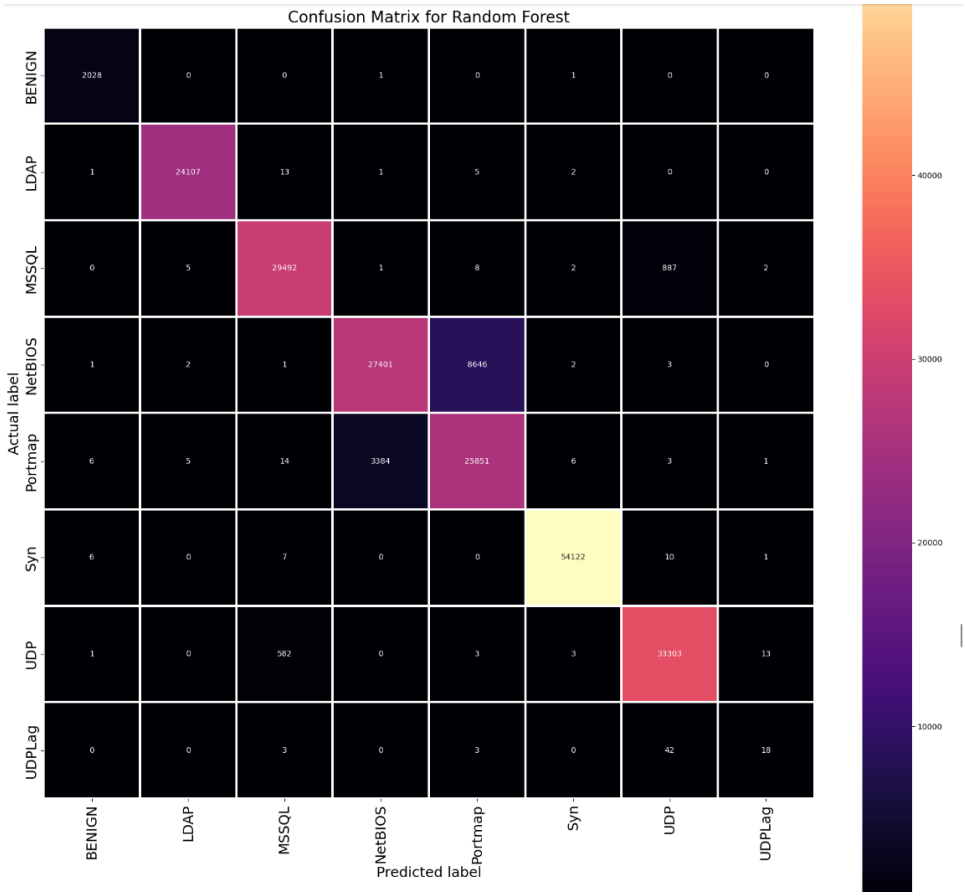


Figure 55. Random Forest Classifier with Confusion Matrix.

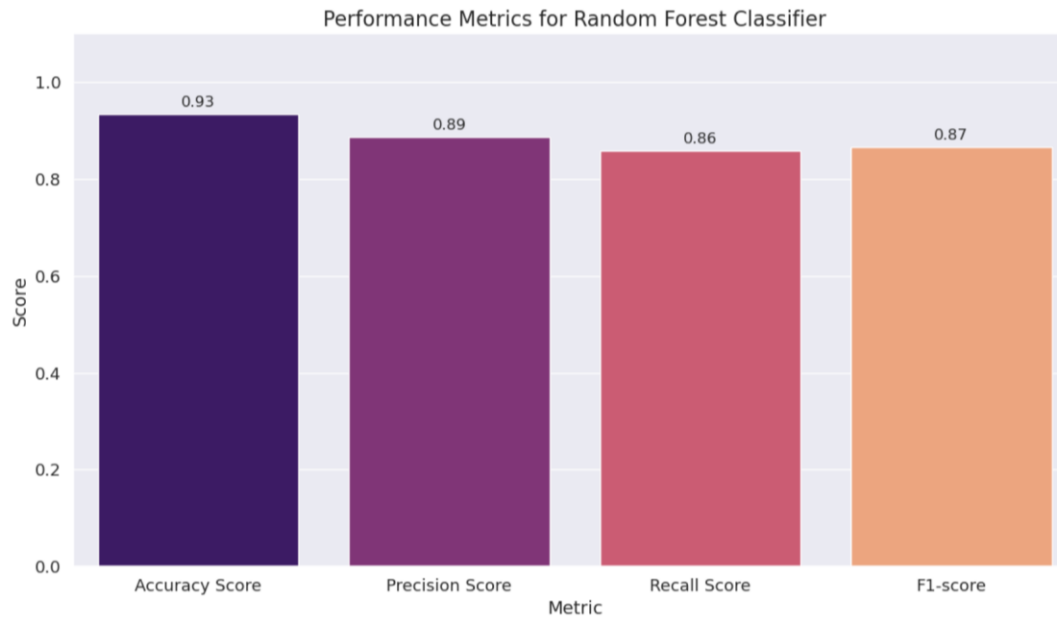


Figure 56. Random Forest Classifier with Performance Metrics and F1-Score.

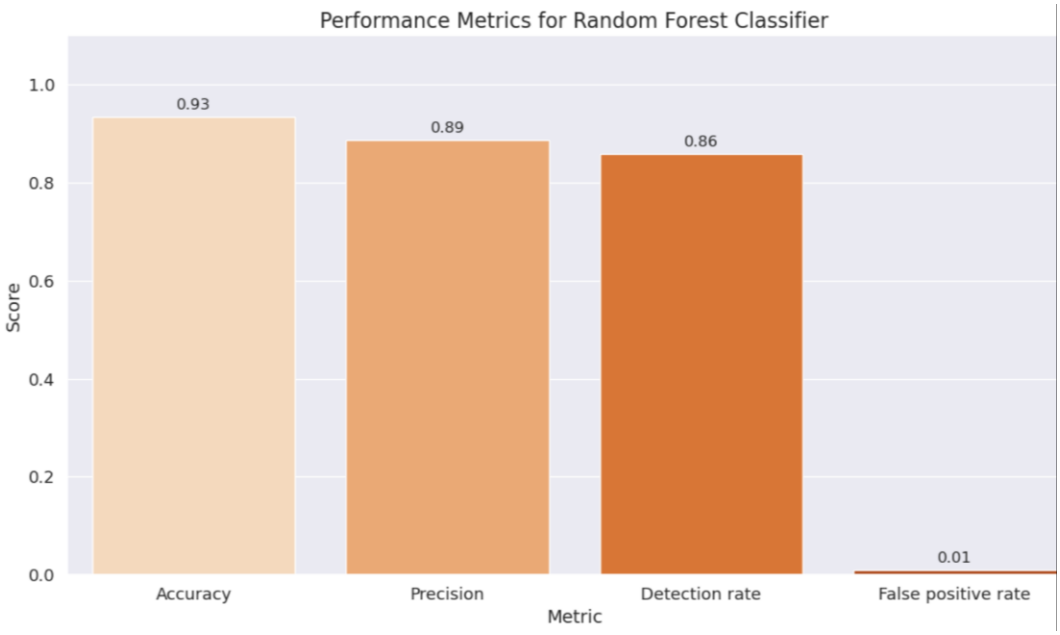


Figure 57. Random Forest Classifier with Performance Metrics and False Positive Rate.

In Figure 54, the Random Forest classifier with the AUC-ROC curve and with PCA shows a 1.00 in the graph, which depicts an overfitting AUC-ROC curve. We can also see that the line touches the 1.0 on the very high top. Figure 55 shows a confusion

matrix and Figures 56 and 57 show an F1-score and false positive rate of the Random Forest classifier.

The following three neural networks, the Convolutional Neural Network, Recurrent Neural Network, and Long Short-Term Memory, will be described shortly. The first neural network, the Convolutional Neural Network (CNN), also known as ConvNets, is a type of network design for deep learning algorithms, methods, and neural networks. A recurrent neural network (ANN) is an artificial neural network (ANN) deployed for supervised and unsupervised learning, and long short-term memory is a type of recurrent neural network.

In the following neural network, we will display a model summary of the Convolutional Neural Network (CNN), the Recurrent Neural Network, and Long Short-Term Memory. In addition to the model summary, the hyperparameters will be described, including their description and what was used. The AUC-ROC curve, confusion matrix, performance metrics, and F1-score and false positive rates will be portrayed. In addition, the model accuracy and loss for all three neural networks, the Convolutional Neural Network (CNN), the Recurrent Neural Network, and Long Short-Term Memory, will be depicted below in the following figures. Also, a test loss and test accuracy will be displayed.

Firstly, we will display a model summary of CNN, as shown below.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1d (Conv1D)             (None, 76, 512)           4096

 batch_normalization (BatchN  (None, 76, 512)          2048
 ormalization)

 max_pooling1d (MaxPooling1D  (None, 38, 512)          0
 )

 conv1d_1 (Conv1D)           (None, 32, 256)           917760

 batch_normalization_1 (Batc  (None, 32, 256)          1024
 hNormalization)

 max_pooling1d_1 (MaxPooling  (None, 16, 256)          0
 1D)

 conv1d_2 (Conv1D)           (None, 10, 128)           229504

 batch_normalization_2 (Batc  (None, 10, 128)          512
 hNormalization)

 flatten (Flatten)           (None, 1280)              0

 dropout (Dropout)           (None, 1280)              0

 dense (Dense)               (None, 8)                 10248

=================================================================
Total params: 1,165,192
Trainable params: 1,163,400
Non-trainable params: 1,792
```

Figure 58. Model Summary of the Convolutional Neural Network (CNN).

Figure 58 shows the model summary of the CNN, as shown above. The Model: "sequential" is the beginning of creating a sequential model for the CNN architecture. In CNN, we used Convolutional 1D one-dimensional data because our data is time series, and not images. Therefore, we are using non image data. Also, the type is Conv1D because we are using Convolutional 1D dimension and not 2D (two dimension) or 3D (three dimension).

The Output Shape: (None, 76, 512) indicates the batch size is unspecified, with 76 steps in the output sequence, each with 512 units. The second is Output Shape: (None, 76, 512), which maintains the shape batch-normalization. The third, Output Shape: (None, 38, 512), assumes a pool size of 2, halving the sequence length. The next, Output

Shape: (None, 32, 256), the output is reduced in sequence length. This CNN model has 1,165,192 total params, 1,163,400 trainable params, and 1,792 non-trainable params.

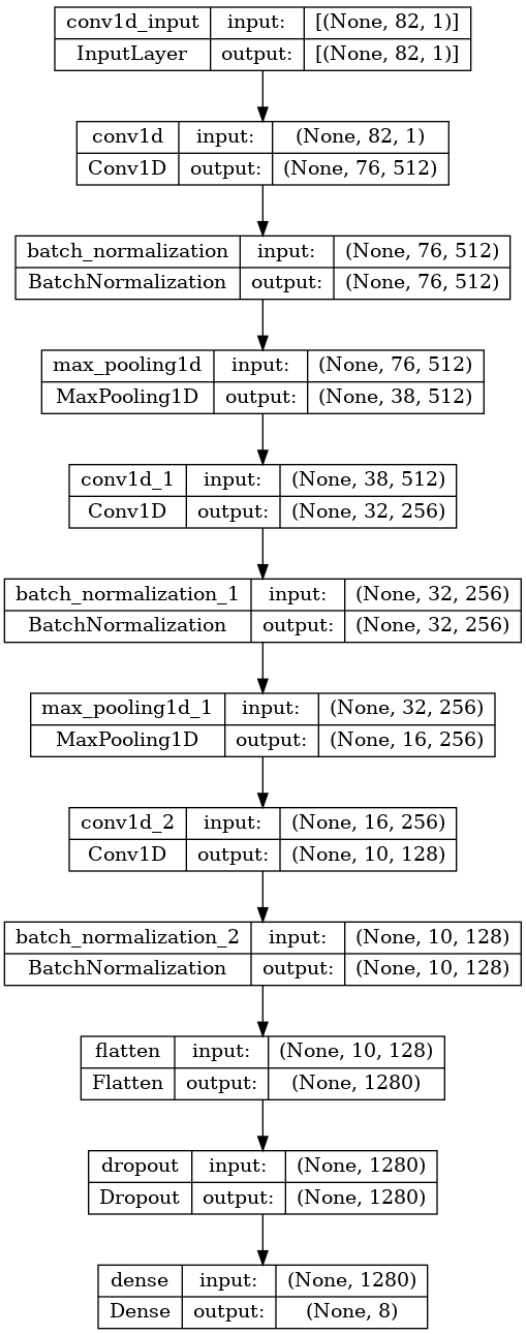


Figure 59. Convolutional Neural Network (CNN) Model.

In the above Figure 59, we can see a Convolutional Neural Network (CNN) model. A 1D convolutional layer with 512 units and a kernel size of seven was employed. Activation was set to relu (Rectified Linear Unit) as relu is used as the activation function, the input shape was assigned to (82, 1), and lastly, the kernel regularizer=L1L2 was set to l1=0.01 and l2=0.01. The kernel regularizer applied both L1 and L2 regularization to the kernel weights. A batch normalization and a Max-pooling layer with a pool size of 2 were utilized. A flatten, dropout, dense was also employed.

Within the CNN, model accuracy and model loss were depicted. In addition to the CNN, an optimizer set to adam, loss set to sparse categorical cross-entropy, and metrics set to accuracy complied with the CNN model. To train our model, 20 epochs were run with a batch size of 32 and verbose to 1. A Convolutional Neural Network (CNN) model accuracy and loss is shown in Figure 60. The CNN model accuracy was 93.10%, and the loss was 0.1689.

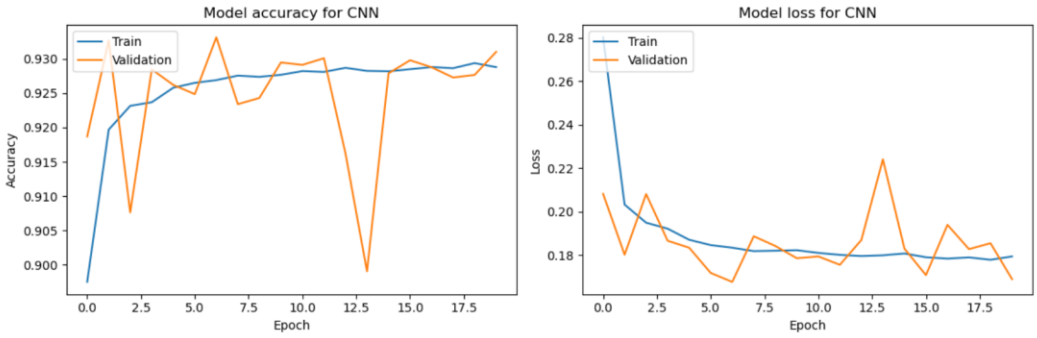


Figure 60. Convolutional Neural Network (CNN) Model Accuracy and Loss.

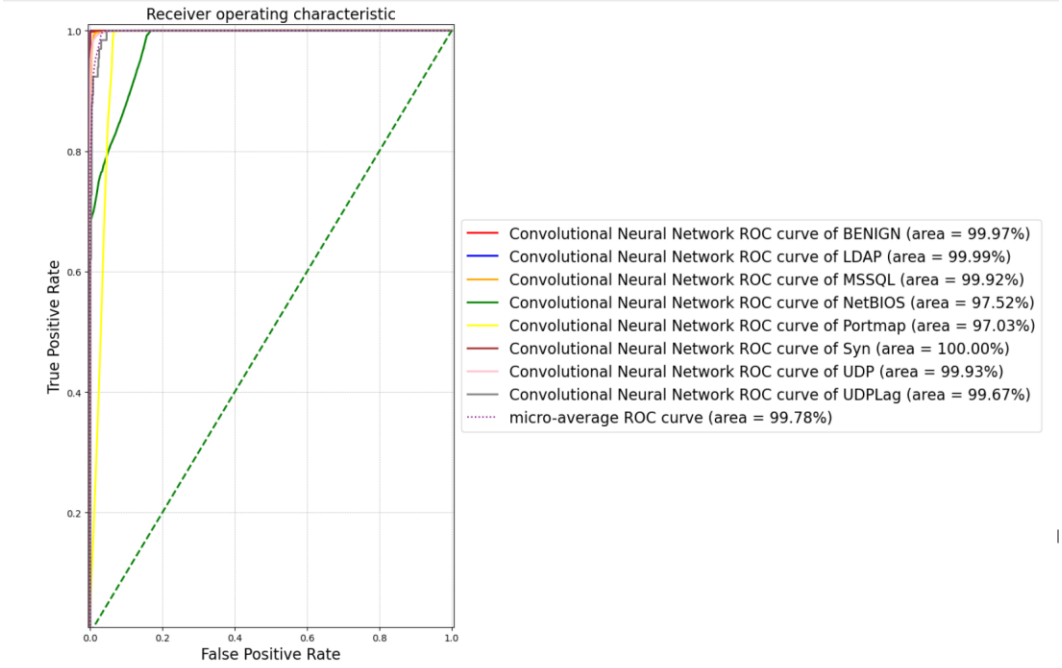


Figure 61. ROC-AUC Curve of a Multi-Class Classification in Convolutional Neural Network (CNN).

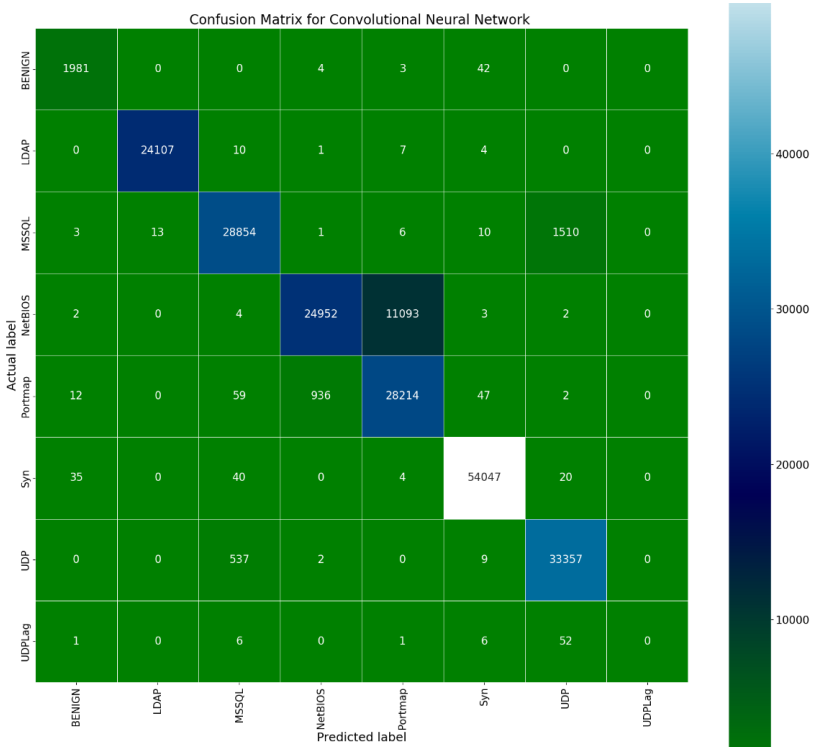


Figure 62. Convolutional Neural Network (CNN) with Confusion Matrix.

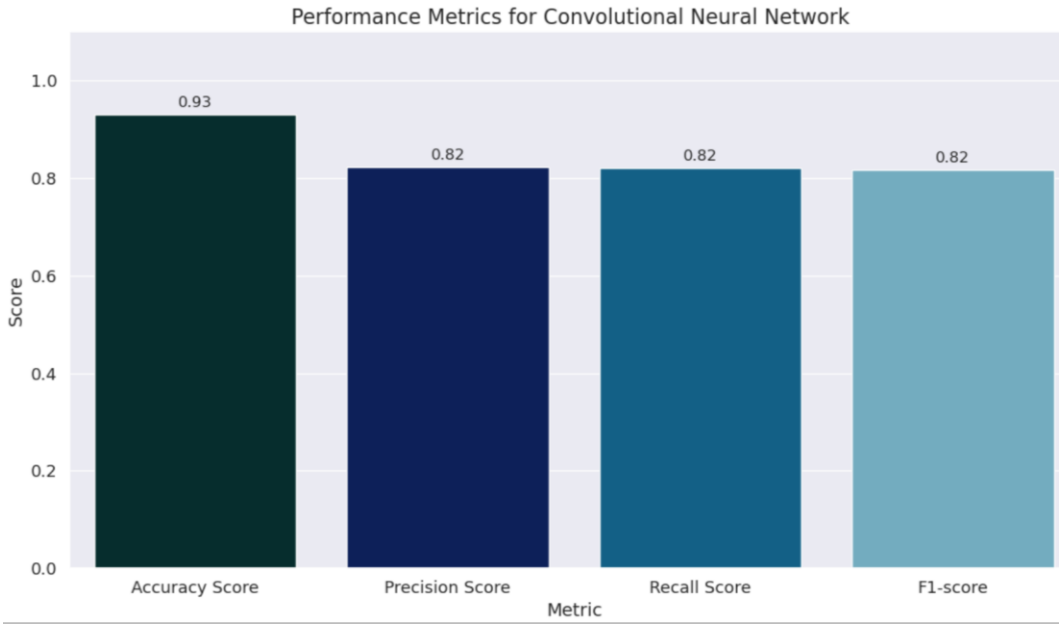


Figure 63. Convolutional Neural Network with Performance Metrics and F1-Score.

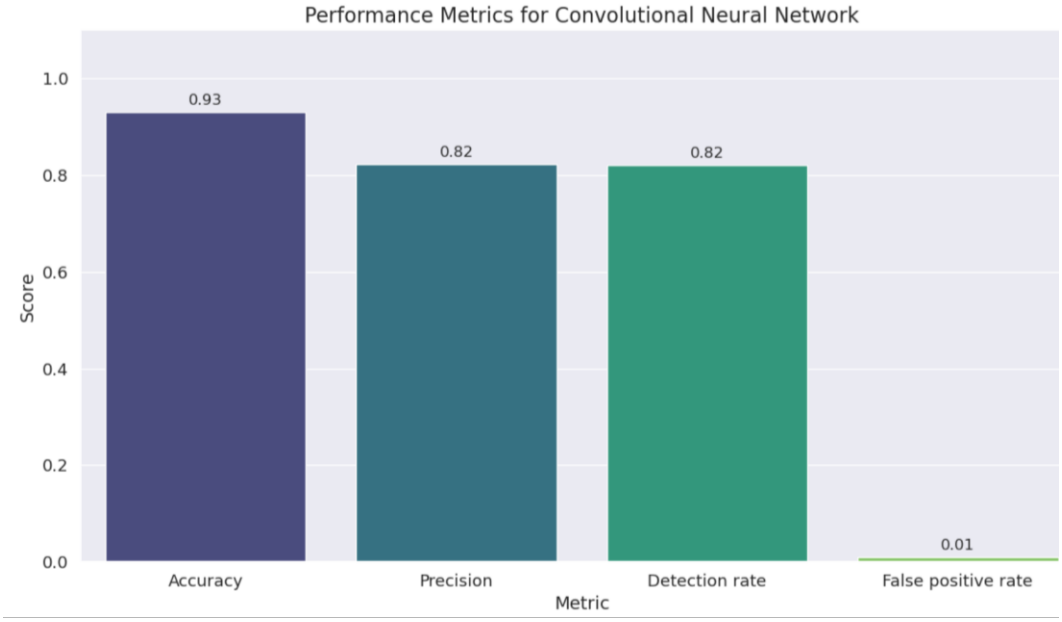


Figure 64. Convolutional Neural Network with Performance Metrics and False Positive Rate in CIC-DDoS2019 Dataset.

The above four Figures 61, 62, 63, and 64 show the confusion matrix, the ROC-AUC curve, and the Performance metrics with the F1-score and false positive rate. In the confusion matrix, we see the UDP-Lag has all 0's, and in the ROC-AUC curve, Syn has a

100.0%, meaning it is overfitted. Also, the CNN performed well, with an accuracy of 93.1%.

The next following neural network, the Recurrent Neural Network (RNN), is shown below in Figure 65. Like the CNN model summary, the RNN also has a sequential, but we use SimpleRNN instead. SimpleRNN stands for Simple Recurrent Neural Network, and SimpleRNN is used in sequential data.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 simple_rnn (SimpleRNN)      (None, 128)               16640

 dense (Dense)               (None, 64)                8256

 dropout (Dropout)           (None, 64)                0

 dense_1 (Dense)             (None, 64)                4160

 dropout_1 (Dropout)         (None, 64)                0

 dense_2 (Dense)             (None, 8)                 520

=================================================================
Total params: 29,576
Trainable params: 29,576
Non-trainable params: 0
```

Figure 65. Model Summary of the Recurrent Neural Network (RNN).

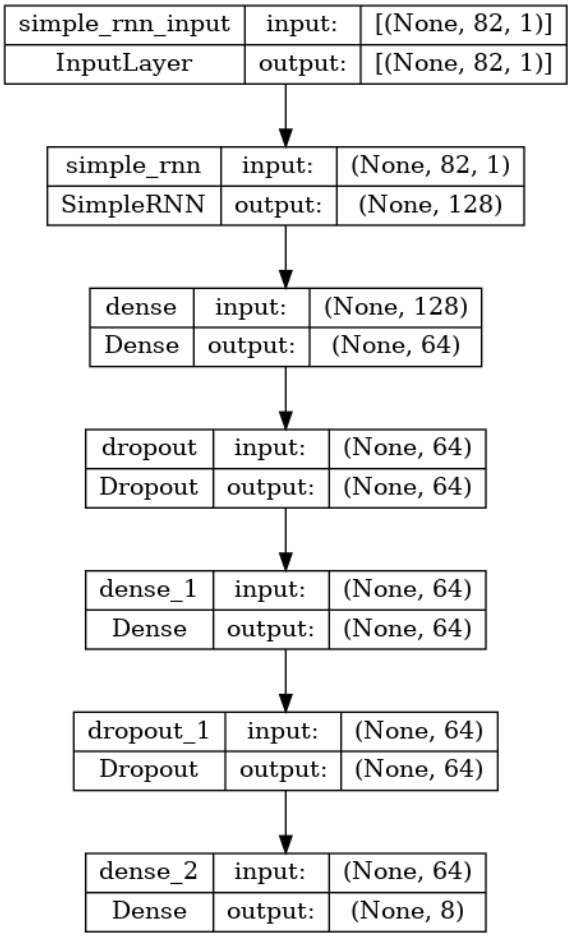


Figure 66. Recurrent Neural Network (RNN) Model.

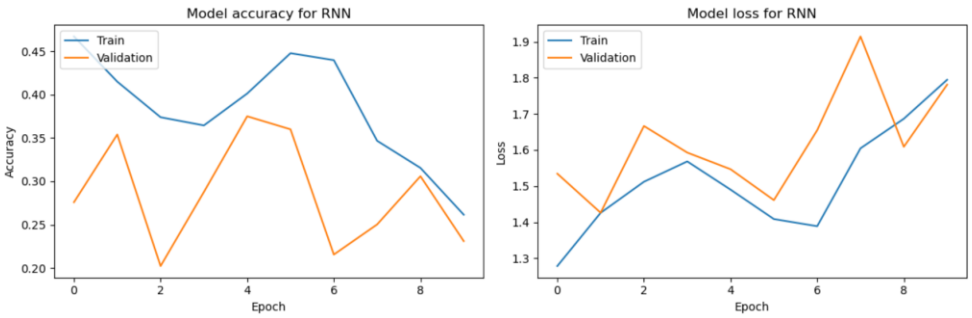


Figure 67. Recurrent Neural Network (RNN) Model Accuracy and Loss.

Like the CNN, we use the SimpleRNN layer with 128 units, an activation of tanh where tanh refers to the hyperbolic tangent activation function, and the input shape is (None, 82, 1). A dropout, dense, was also employed. We also see a SimpleRNN, which indicates that a SimpleRNN is being used for this neural network. The RNN model has 29, 576 total params, 29, 576 trainable params, and 0 non-trainable params. The loss was 1.7812, and the accuracy was 0.2310. In addition to the loss and accuracy, an optimizer with Adam and a learning rate of 0.001 was applied, the loss was set to sparse categorical cross-entropy, and metrics were set to accuracy. In other words, epochs were set to 10 and batch size to 32.

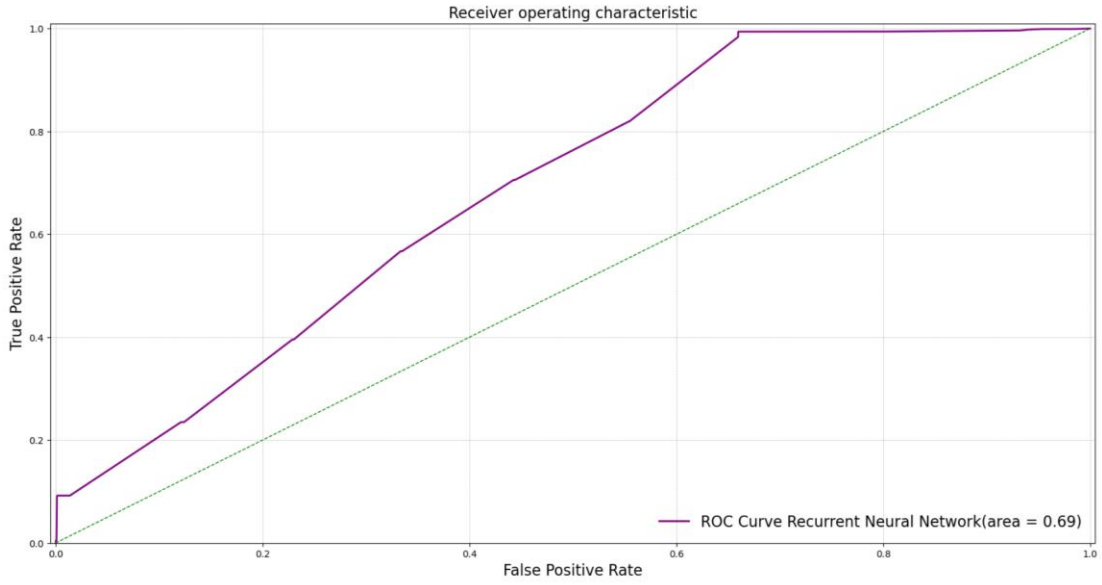


Figure 68. ROC-AUC Curve of a Recurrent Neural Network (RNN).

Figure 68 shows a ROC-AUC curve of the Recurrent Neural Network. As we can see, the line starts at the bottom left-hand side and goes straight up in a curvy line. However, this line does not go in the True Positive Rate direction, meaning that the model may be unable to predict the True Positive Rates. The Area Under the Curve was 69.1%.

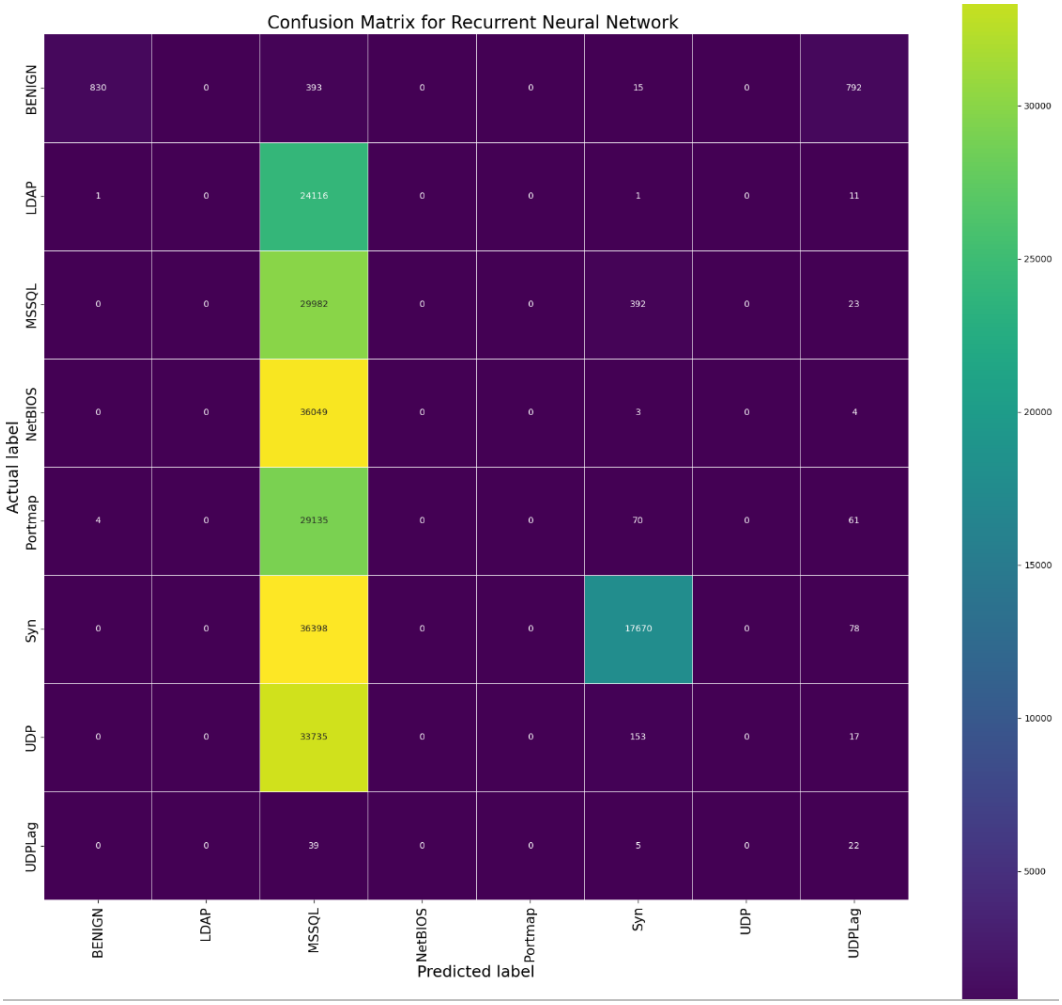


Figure 69. Recurrent Neural Network (RNN) of a Confusion Matrix.

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.41	0.58	2030
1	0.00	0.00	0.00	24129
2	0.16	0.99	0.27	30397
3	0.00	0.00	0.00	36056
4	0.00	0.00	0.00	29270
5	0.97	0.33	0.49	54146
6	0.00	0.00	0.00	33905
7	0.02	0.33	0.04	66
accuracy			0.23	209999
macro avg	0.27	0.26	0.17	209999
weighted avg	0.28	0.23	0.17	209999

Figure 70. Recurrent Neural Network Classification Report.

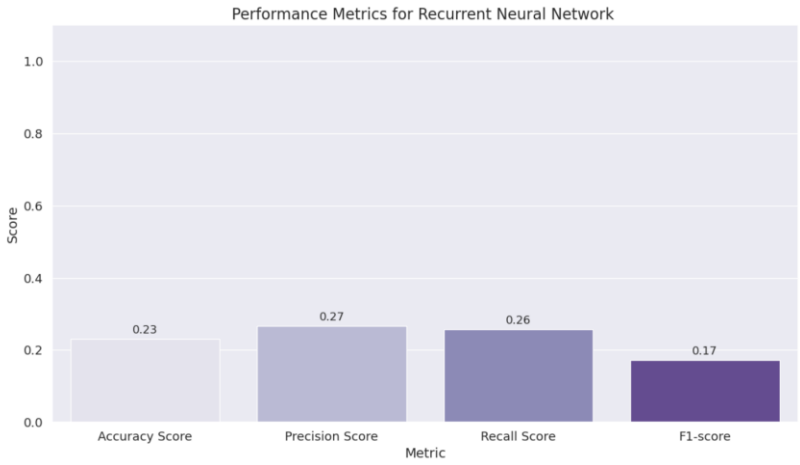


Figure 71. Recurrent Neural Network with Performance Metrics and F1-Score.

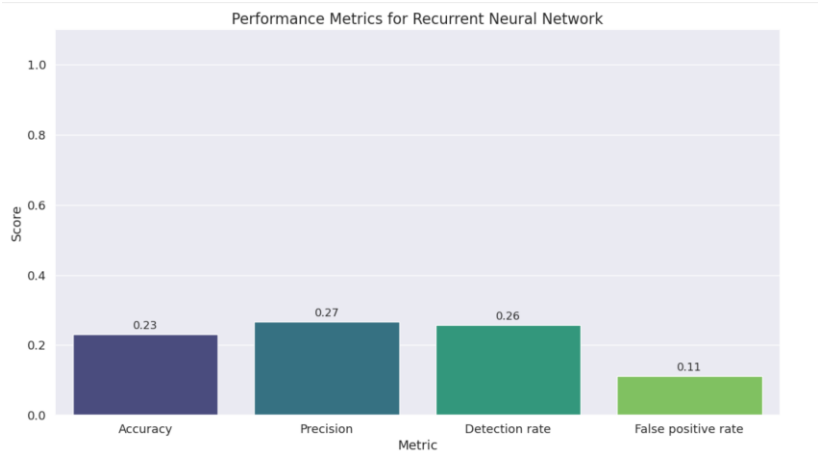


Figure 72. Recurrent Neural Network with Performance Metrics and False Positive Rate in APA-DDoS.

In Fig 69, a confusion matrix shows few True Positive Rates. Many of the boxes are 0. Only the MSQL column and Syn represent high numbers. The rest of the columns show low numbers in identifying True Positive, False Positive, True Negative, and False Negative. Figure 70 shows a classification report. According to the classification report, much support was needed based on the labels and the imbalanced dataset.

Moreover, Figures 71 and 72 illustrate the performance metrics, false positive rate, and accuracy. This RNN had the lowest accuracy score, with 23.1, meaning it did

not perform well. The imbalanced dataset may have affected the model performance, leading to low accuracy. However, the RNN had the lowest AUC and did not improve.

The last neural network, the Long Short-Term Memory, did perform better than the Recurrent Neural Network. Below, we will describe the following Long Short-Term Memory model summary.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 256)               264192

 dense (Dense)               (None, 128)               32896

 flatten (Flatten)           (None, 128)               0

 dropout (Dropout)           (None, 128)               0

 dense_1 (Dense)             (None, 64)                8256

 dropout_1 (Dropout)         (None, 64)                0

 dense_2 (Dense)             (None, 8)                 520

=================================================================
Total params: 305,864
Trainable params: 305,864
Non-trainable params: 0
```

Figure 73. Model Summary of the Long Short-Term Memory (LSTM).

In the above two neural networks, the Convolutional Neural Network and Recurrent Neural Network, Figure 73 and Long Short-Term Memory use a “Sequential” model. The type is LSTM because we use Long Short-Term Memory, which is why the name is displayed. The Output Shape (None, 256) shows the LSTM layer where each timestep has 256 units. A dense layer of 128 neurons was used. Then, we flattened the output to 1D to connect it to a dense layer; however, no parameters were used as indicated 0. Another dense layer, a dropout layer, and the fully connected layers, which depict 520 parameters, were used. In this Long Short-Term Memory model summary, the

total params were 305, 864, the trainable params were 305, 864, and the non-trainable params were 0. Therefore, in this Long Short-Term Memory, the non-trainable was 0, meaning they were not trainable; however, in the CNN, we did see a non-trainable param that was not 0.

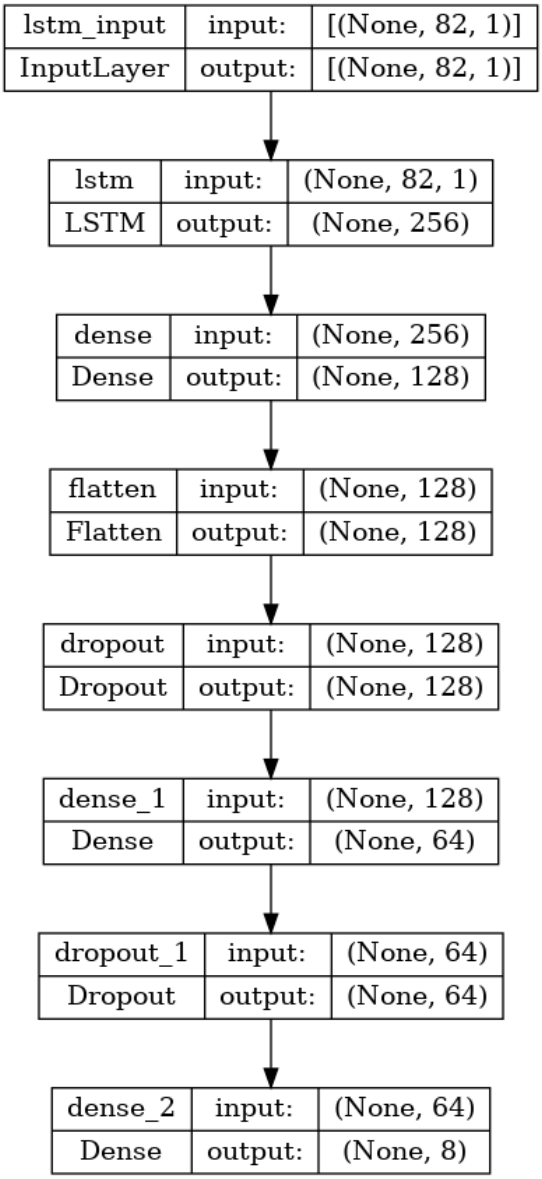


Figure 74. Long Short-Term Memory (LSTM) Model.

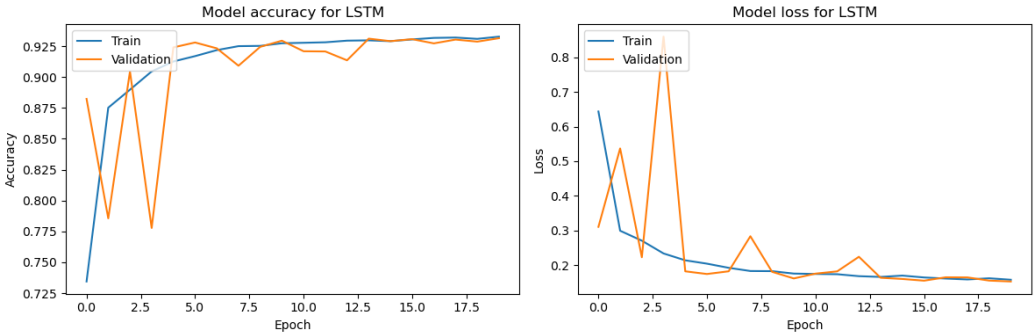


Figure 75. Long Short-Term Memory (LSTM) Model Accuracy and Loss.

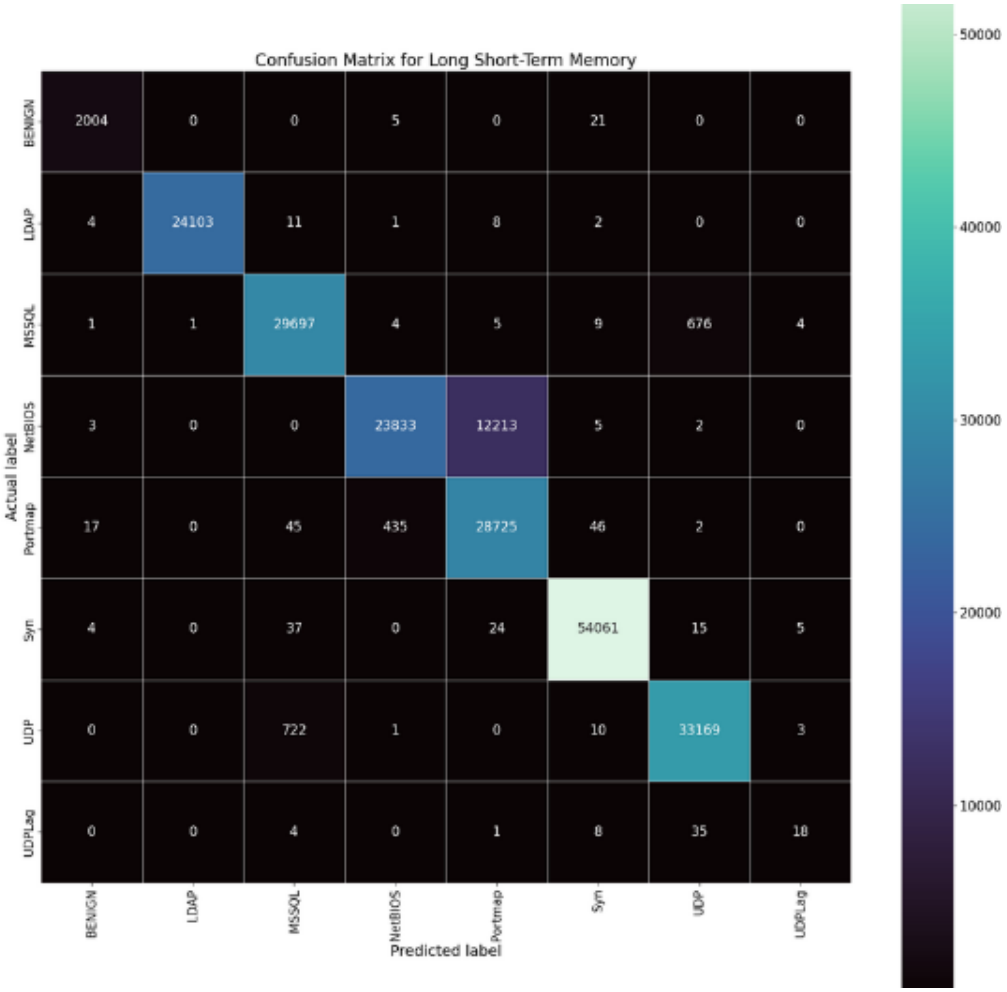


Figure 76. Long Short-Term Memory (LSTM) of a Confusion Matrix.

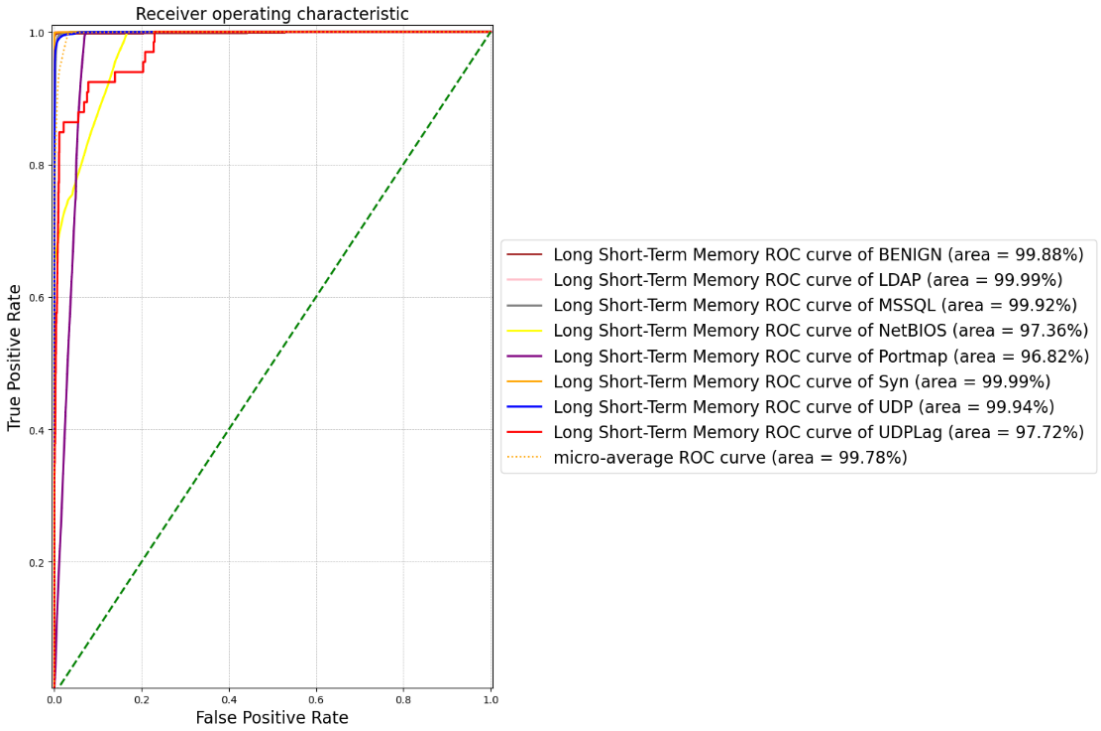


Figure 77. ROC-AUC Curve of a Multi-Class Classification in Long Short-Term Memory.

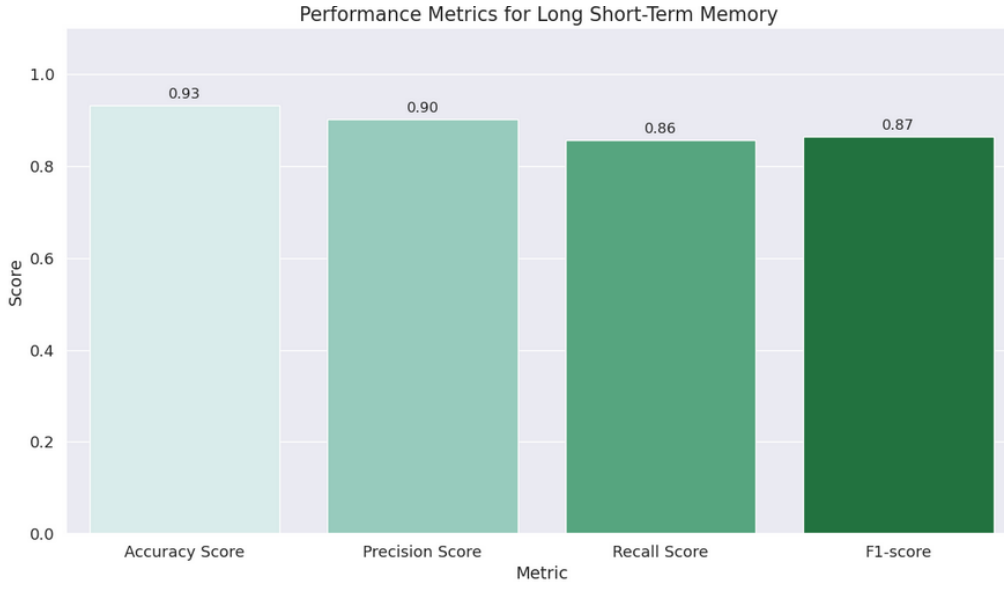


Figure 78. Long Short-Term Memory with Performance Metrics and F1-score.

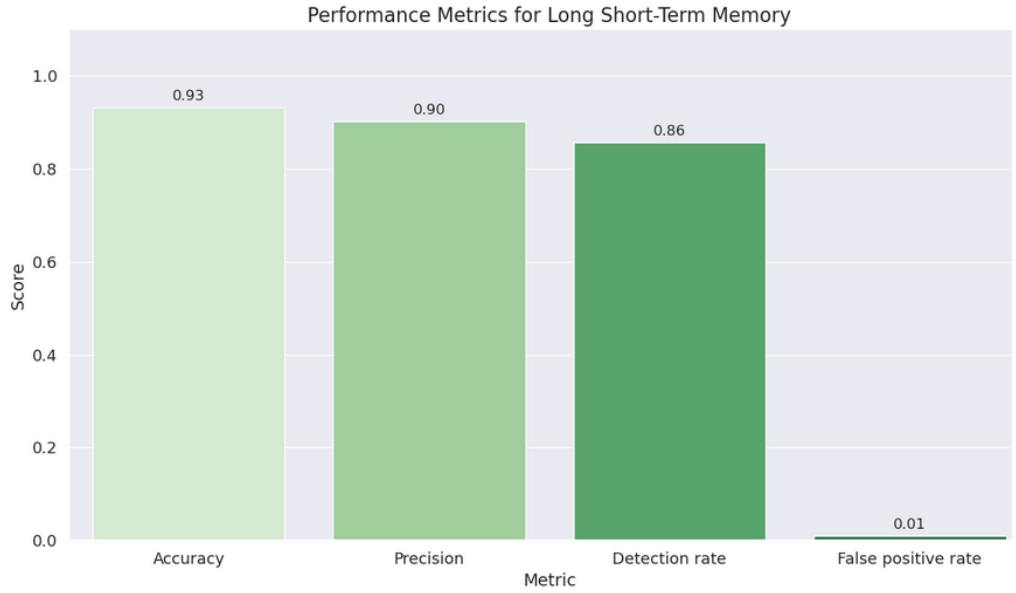


Figure 79. Long Short-Term Memory with Performance Metrics and False Positive Rate in CIC-DDoS2019 Dataset.

In Figure 74, a model is shown. Usually, the lstm_input shows (None, 82, 1), followed by the LSTM input and output and the dense, flattened dropout. In addition, a learning rate of 0.001 was used, loss was set to sparse categorical cross-entropy, and the metric was set to accuracy. Moreover, the epochs were set to twenty, and the batch size was 256. In Figure 75, the loss was 0.1523, and the accuracy was 93.15%. The confusion matrix in Figure 76 shows many True Positives but also shows 0. The multi-class classification in Figure 77 is depicted, and it shows high percentages of the ROC-AUC curve. The last two figures, 78 and 79, illustrate the performance metrics with F1-score and false positive rate. In Figure 78, the accuracy score is 93.15%, the precision is 90.23%, the recall is 85.69%, and the F1-score is 86.51%. The last figure, Figure 79, shows the accuracy at 93.15%, the precision at 90.23%, the detection rate at 85.69%, and the false positive rate at 1.00%.

CHAPTER 5

APA-DDoS Dataset and their Results

Chapter 5 will discuss the APA-DDoS Dataset, their observational results, and the hyperparameters associated with each machine learning algorithm and neural network. Additionally, machine learning algorithms are developed in this dataset for Intrusion Detection Systems (IDS) to classify attacks as normal or attack traffic.

The APA-DDoS Dataset is used to evaluate Intrusion Detection Systems (IDS) in this paper, and this dataset is imbalanced, which we will describe shortly. Given the sophistication of the attacks and a variety of normal traffic patterns, machine learning is becoming mainstream in IDS [77]. This APA-DDoS Dataset includes two Packet Capture (PCAP) files and one Comma-Separated Value (CSV file). The PCAP file names are ACK.pcap and PUSH-ACK.pcap, and the CSV file has a name associated with APA-DDoS-Dataset.csv. These three files represent the APA-DDoS Dataset. The dataset's common attack patterns included ACK and PUSH-ACK flooding DDoS attacks; IDS developers can use the developed dataset to increase the detection ratio using detection modules [77].

To begin with, the APA-DDoS Dataset consists of 151, 200 rows, and 23 columns. The last column is our label column. The number of features is twenty-three, and the number of samples is 151, 200. This APA-DDoS dataset is an imbalanced dataset.

Table 15 will show the APA-DDoS Dataset feature number, feature names, and description of the features; see below.

Table 15

APA-DDoS Dataset Feature Number, Feature Names, and Description of the Features

Feature Number (#)	**Feature Name**	**Data Type**	**Description**
0	ip.src	object	The source address of the internet protocol [78].
1	ip.dst	object	The destination address of the internet protocol [78].
2	tcp.srcport	int64	The source port of the transmission control protocol [79].
3	tcp.dstport	int64	The destination port of the transmission control protocol [79].
4	ip.proto	int64	The protocol field of the internet protocol header [78].
5	frame.len	int64	Frame length on the wire [80].
6	tcp.flags.syn	int64	Synchronization of the transmission control protocol flags [79].
7	tcp.flags.reset	int64	Reset of the transmission control protocol flags [79].
8	tcp.flags.push	int64	Push of the transmission control protocol flags [79].
9	tcp.flags.ack	int64	Acknowledgement of the transmission control protocol flags [79].
10	ip.flags.mf	int64	More Fragments of the internet protocol flags [78].
11	ip.flags.df	int64	Don’t Fragments of the internet protocol flags [78].
12	ip.flags.rb	int64	Reserved Bit of the internet protocol flags [78].
13	tcp.seq	int64	Sequence Number within transmission control protocol [79].
14	tcp.ack	int64	Acknowledgement of the transmission control protocol [79].
15	frame.time	object	Arrival Time [80].
16	Packets	int64	A certain number of data bytes, wrapped or encapsulated in headers and/or trailers that contain information about where the packet came from, where it’s going [61].
17	Bytes	int64	Eight bits and Base-8 numbering system used to identify a section of a dotted decimal IP address [61].

18	Tx Packets	int64	Transmitted Packets
19	Tx Bytes	int64	Transmitted Bytes
20	Rx Packets	int64	Received Packets
21	Rx Bytes	int64	Received Bytes
22	Label	object	Target variable for prediction

The APA-DDoS dataset table and its feature number, feature names, and data types are displayed above. The feature names in the APA-DDoS Dataset represent DDoS attacks, and the data consists of two data types: one, which is an integer sixty-four, and two, which is an object. In the Data Types, there are nineteen columns of Int64 and four objects; therefore, this totals twenty-three feature columns.

Feature names represent the Distributed Denial of Service (DDoS) attacks. There are twenty-three feature names, the last of which is the label column. Each column represents a DDoS attack. All these feature names describe characteristics typically associated with a DDoS attack. For example, IP stands for Internet Protocol (IP), TCP stands for Transmission Control Protocol (TCP), frame, Tx stands for transmitted, and Rx stands for received, flags, time, packets, and bytes. The description describes the feature name of the APA-DDoS Dataset.

Results

In this paper, the same machine-learning algorithms and neural networks applied in Chapter 4 were used in Chapter 5. This subsection will also present the accuracy, graphs, results, their hyperparameters associated with each machine learning algorithm, and what leads to overfitting and whatnot.

In Table 16, a hyperparameter with PCA is shown below, and the accuracy score, the AUC-ROC score, confusion matrix is seen below. This traditional ML shows that the model did not overfit.

Table 16

Decision Tree Classifier and their Hyperparameters with Principal Component Analysis (PCA) in APA-DDoS Dataset

Name	**Used**	**Description**
random_state	5	Controls both the randomness of the bootstrapping of the samples used when building trees and the sampling of the features to consider when looking for the best split at each node [68].
max_depth	5	The maximum depth of the tree [68].
min_samples_split	10	The minimum number of samples required to split an internal node [68].
min_samples_leaf	10	The minimum number of samples required to be at a leaf node [68].
max_features	5	The number of features to consider when looking for the best split [68].
ccp_alpha	0.05	Complexity parameter used for Minimal Cost-Complexity Pruning [68].
splitter	best	The strategy used to choose the split at each node [68].
criterion	gini	The function to measure the quality of a split [68].

Table 17

Decision Tree Classifier with Principal Component Analysis (PCA)

Decision Tree: Accuracy Score	**Decision Tree: Cross-Validation Score**	**Decision Tree: Cross-Validation Mean Score**	**Decision Tree: ROC-AUC Score**
94.4 %	[0.98617725 0.99371693 0.93981481	97.4%	98.7%

	0.97116402 0.99272487 0.97757937 0.98935185 0.98630952 0.93921958 0.96276455]		

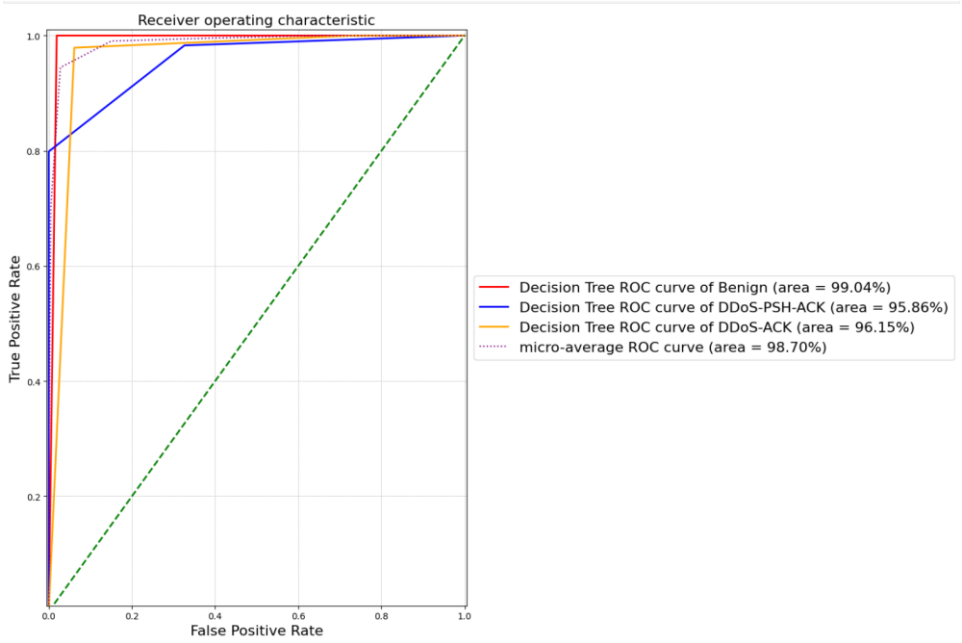


Figure 80. ROC-AUC Curve of a Multi-Class Classification Decision Tree.

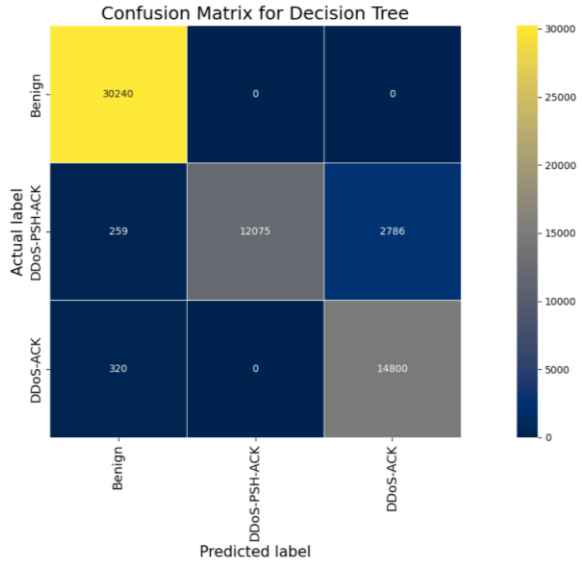


Figure 81. Decision Tree of a Confusion Matrix in APA-DDoS Dataset.

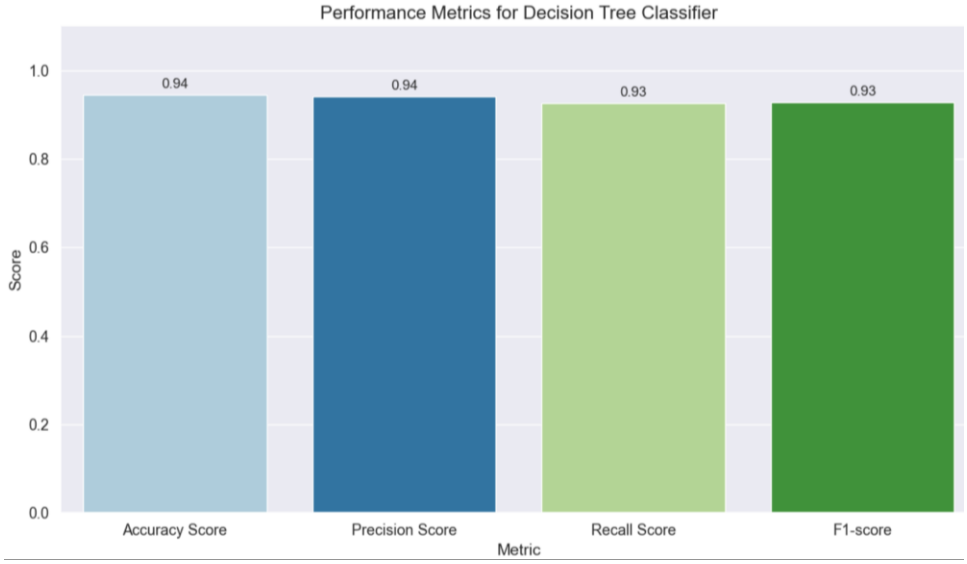


Figure 82. Decision Tree with Performance Metrics and F1-Score in APA-DDoS Dataset.

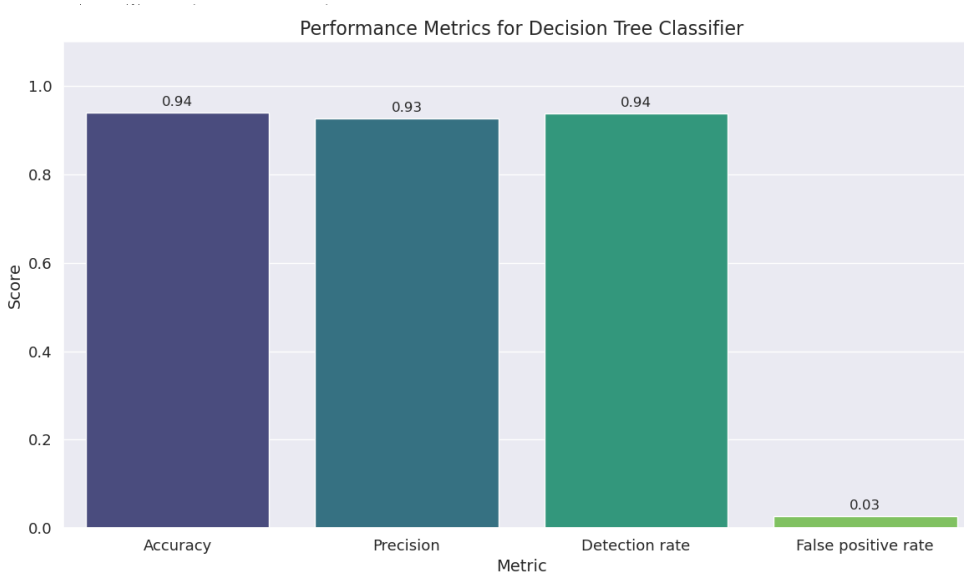


Figure 83. Decision Tree with Performance Metrics and False Positive Rate in APA-DDoS Dataset.

The same hyperparameter used in the CIC-DDoS2019 dataset was also used in the APA-DDoS dataset, only the iteration was changed to 20 for the number of trees and the PCA was set to fourteen, an ImblearnPipeline was used with a SimpleImputer to mean. In addition, we will describe the CatBoost classifier and their accuracy and cross-validation score in Table 18, see below.

Table 18

CatBoost Classifier with Principal Component Analysis (PCA) in APA-DDoS Dataset

CatBoost: Accuracy Score	**CatBoost: Cross-Validation Score**	**CatBoost: Cross-Validation Mean Score**	**CatBoost: ROC-AUC Score**
92.9 %	[0.98955026 1. 1. 0.99966931 0.99973545 1. 1. 1. 1. 1.]	99.9%	98.7%

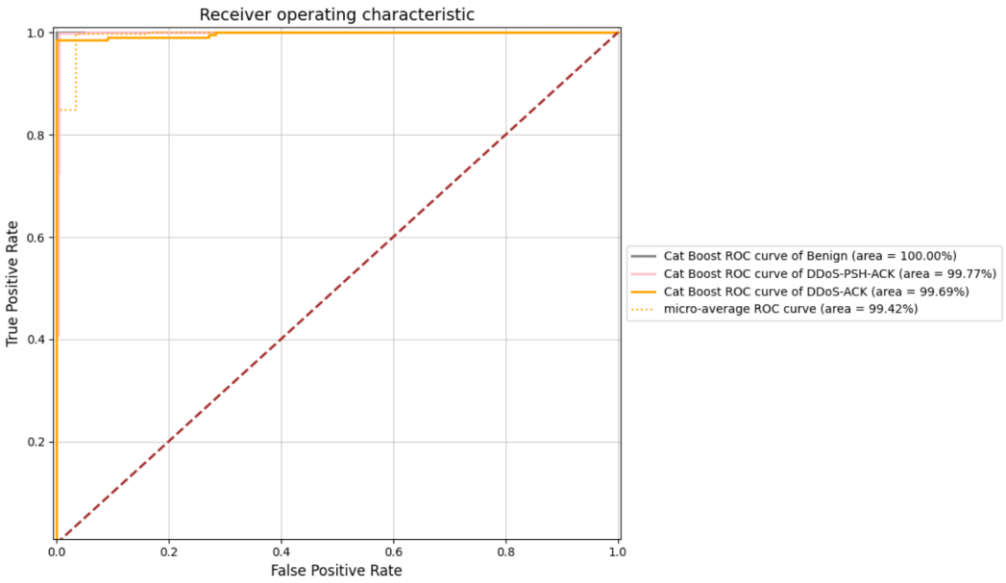


Figure 84. ROC-AUC Curve of a Multi-Class Classification CatBoost.

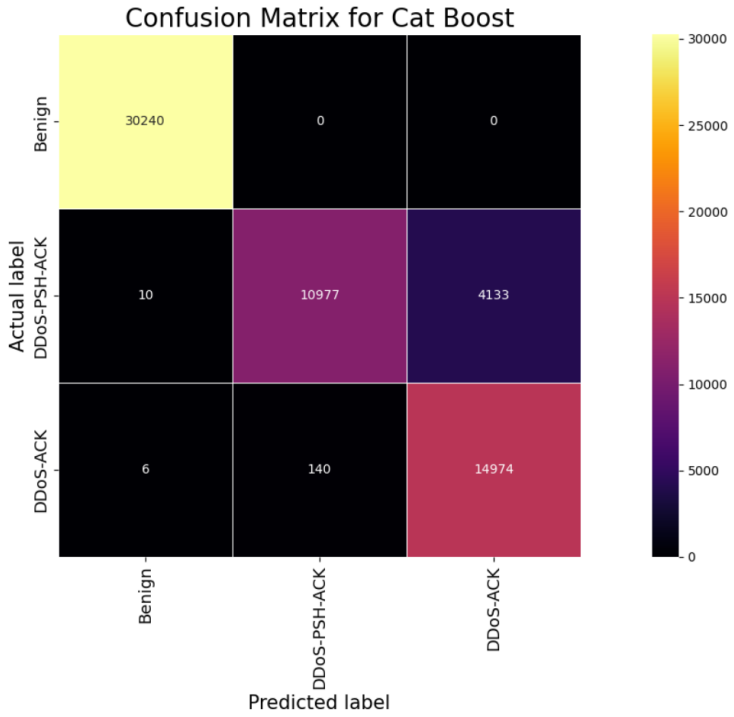


Figure 85. Confusion Matrix of a Multi-Class Classification CatBoost.



Figure 86. Performance Metrics and F1-Score with CatBoost.



Figure 87. Performance Metrics and False Positive Rate with CatBoost.

Figures 84-87 show performance metrics, a confusion matrix, and a ROC-AUC curve. In these figures, we can see that the scores are at a high 90 percent, which shows that the machine learning algorithm performs great without any exceptions and has not overfitted. In the confusion matrix, we see the true positives have performed without 0, which shows True Positives in the APA-DDoS dataset.

The only difference in the hyperparameters in SVM was that it uses a random state with five, according to Chapter 4. Also, it uses SMOTE with a random state of five and PCA with fourteen and uses ImblearnPipeline with SimpleImputer set to mean. However, in Table 19, the accuracy score, cross-validation mean score, and ROC-AUC score seen below all show 100%, which implies that the Support Vector Machine algorithm has overfitted because, generally, the APA-DDoS dataset is a clean, small-sized Distributed Denial of Service (DDoS) dataset causing this algorithm which its dataset to overfit. Even though the parameter was set to C = 1.0, where C is the regularization parameter of the Support Vector Machine classifier, our model has overfitted, reaching 100% accuracy.

Table 19

Principal Component Analysis (PCA) and Synthetic Minority Oversampling (SMOTE) with Support Vector Machine in APA-DDoS Dataset

Support Vector Machine: Accuracy Score	**CatBoost: Cross-Validation Score**	**CatBoost: Cross-Validation Mean Score**	**CatBoost: ROC-AUC Score**
100.0 %	[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]	100.0%	100.0%

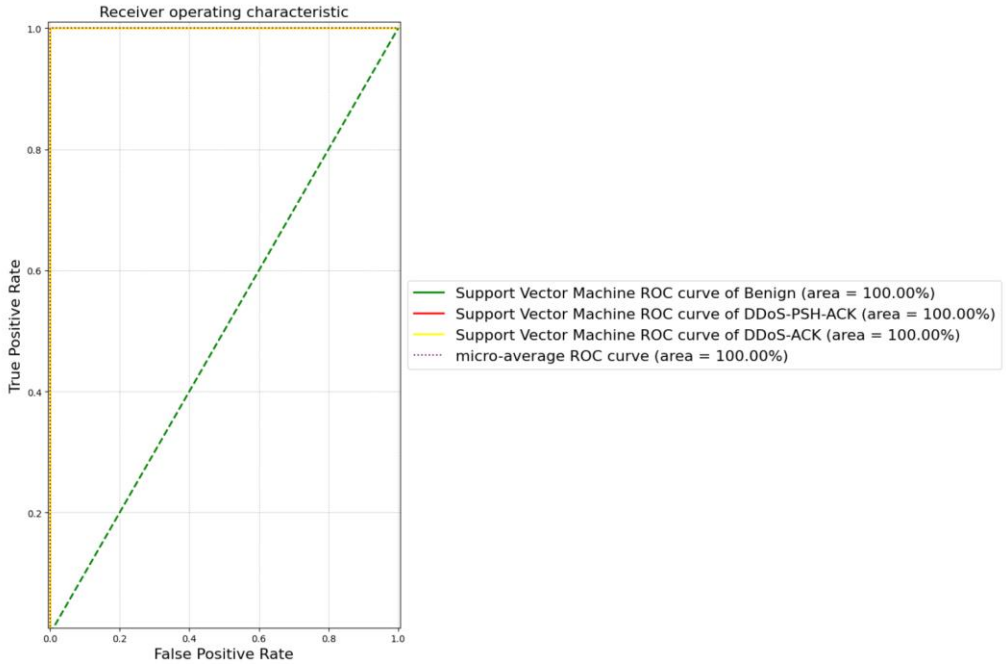


Figure 88. APA-DDoS Dataset Multi-Class Classification ROC-AUC Curve in Support Vector Machine.

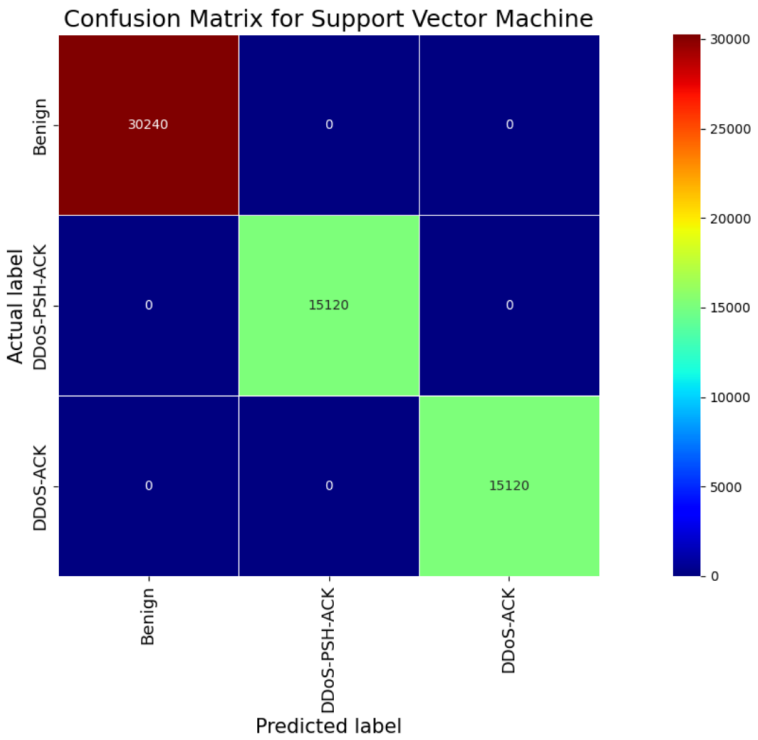


Figure 89. APA-DDoS Dataset Multi-Class Classification Confusion Matrix in SVM.

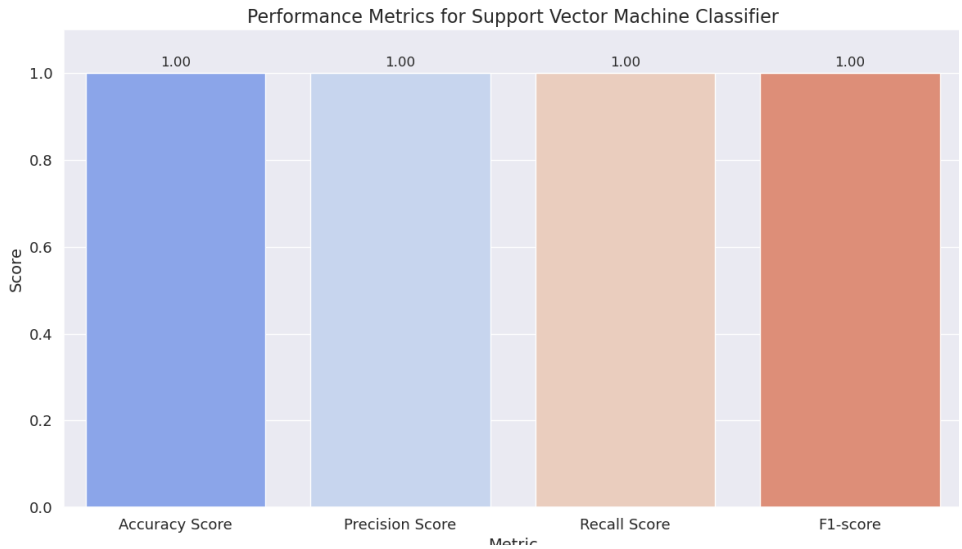


Figure 90. Support Vector Machine Classifier with Performance Metrics and F1-Score in APA-DDoS Dataset.

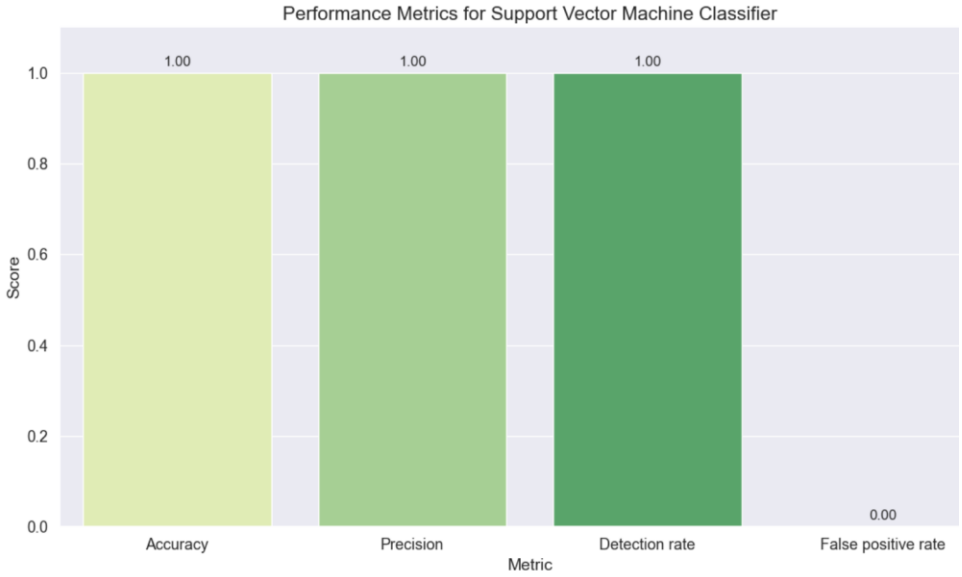


Figure 91. Support Vector Machine Classifier with Performance Metrics and False Positive Rate in APA-DDoS Dataset.

Generally, in the above figures, Figures 88-91, we can see 100% accuracy, and the graphs all reach 1.0. The confusion matrix shows a matrix that only portrays three boxes with three different numbers. In contrast, all other numbers in the confusion matrix depict 0. This is an example of a cleaned dataset where all numbers lead to 100%.

K-Nearest Neighbor will be the last traditional machine-learning algorithm we will discuss. We just saw in the Support Vector Machine that the APA-DDoS dataset is cleaned. However, in K-Nearest Neighbor, the same thing happens. Again, we are noticing 100.0% accuracy.

Table 20

Hyperparameters with Principal Component Analysis (PCA) and Synthetic Minority Over-Sampling Technique (SMOTE) in K-Nearest Neighbor Classifier with APA-DDoS Dataset

Name	**Used**	**Description**
n_neighbors	1000	Number of neighbors to use [75].
weights	uniform	Uniform weights. All points in each neighborhood are weighted equally [75].
algorithm	brute	Brute will use a brute-force search [75].
n_jobs	-1	The number of parallel jobs to run for neighbors' search [75].
leaf_size	30	Leaf size passed to BallTree or KDTree [75].

metric	minkowski	Metric to use for distance computation [75].
p	2	Power parameter for the Minkowski metric [75].

Table 21

Principal Component Analysis (PCA) and Synthetic Minority Oversampling (SMOTE) with K-Nearest Neighbor Accuracy and Cross-Validation Mean Score and ROC-AUC Score with APA-DDoS Dataset

K-Nearest Neighbor: Accuracy Score	**K-Nearest Neighbor: Cross-Validation Score**	**K-Nearest Neighbor: Cross-Validation Mean Score**	**K-Nearest Neighbor: ROC-AUC Score**
100.0 %	[0.99867725 0.99927249 0.99993386 0.99887566 0.9968254 0.99589947 1. 0.99887566 0.99920635 1.]	99.9%	100.0%

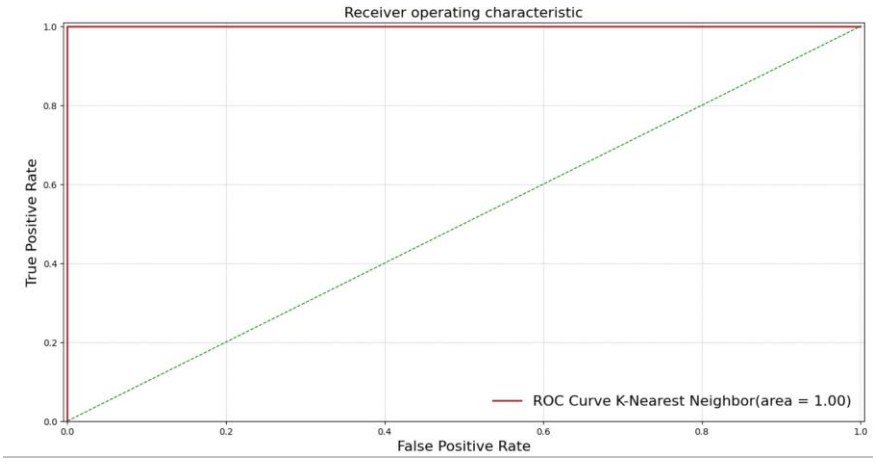


Figure 92. APA-DDoS Dataset ROC-AUC Curve in K-Nearest Neighbor.

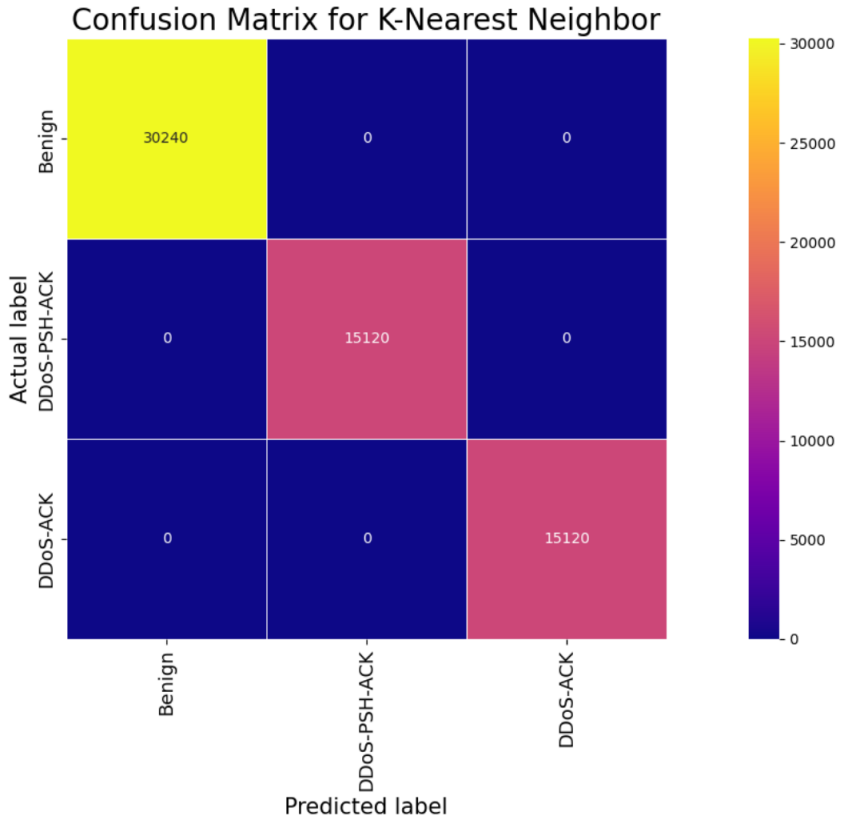


Figure 93. APA-DDoS Dataset Confusion Matrix in K-Nearest Neighbor.

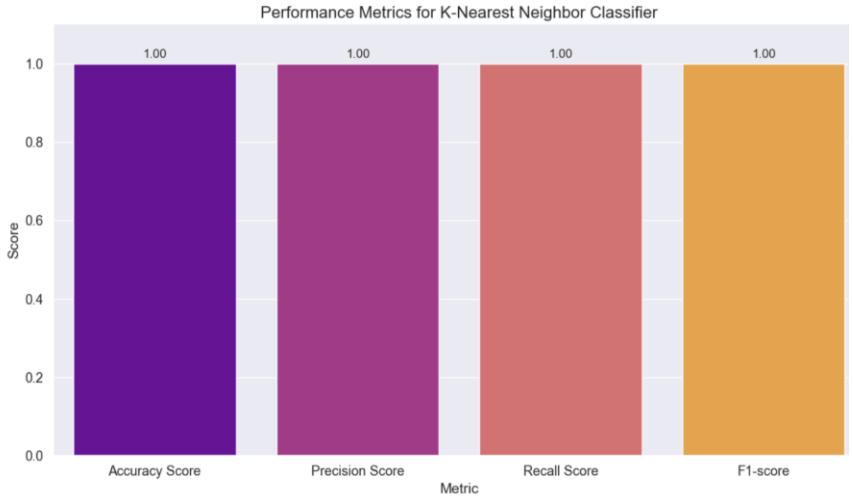


Figure 94. K-Nearest Neighbor with Performance Metrics and F1-Score in APA-DDoS Dataset.

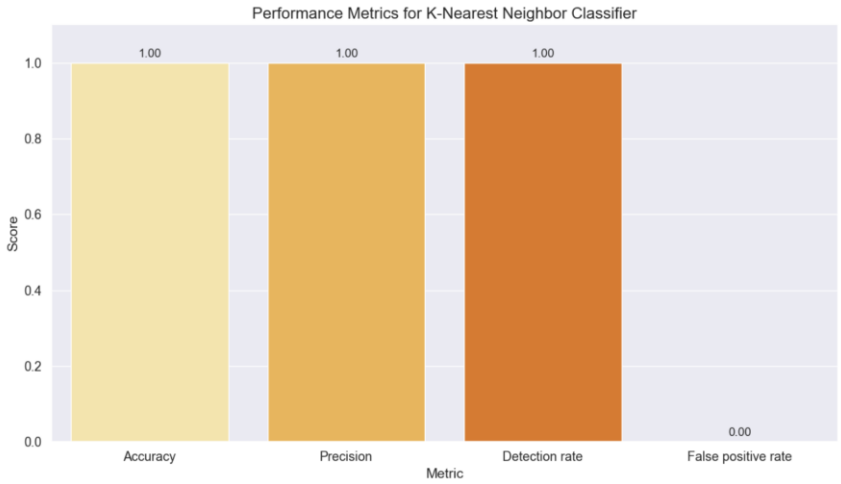


Figure 95. K-Nearest Neighbor with Performance Metrics and False Positive Rate in APA-DDoS Dataset.

Figures 92-95 also show that the confusion matrix, ROC-AUC curve, and performance metrics all have overfitted, and we see 1.00, which is 100%. Again, this shows us that this is a clean dataset.

The following three neural networks and the Convolution Neural Network (CNN), the Recurrent Neural Network (RNN), and the Long-Short Term Memory (LSTM) in the APA-DDoS Dataset will be described, as well as their accuracy, ROC-AUC score, confusion matrix, and the performance metrics of the CNN, the RNN, and the LSTM.

The following figures will show the first neural network, the Convolution Neural Network (CNN), with the model summary, ROC-AUC curve, confusion matrix, and performance metrics. As seen in Figure 96, the model is sequential and uses Conv1D because the dataset is time series data. In addition, the convolutional layers have thirty-two filters, a kernel size of seven, an activation ReLu, and a kernel regularizer with l1

and l2 where l1=0.01 and l2=0.01. In addition, a max pooling layer with pool size two was used. A convolutional layer with 64 filters, a kernel size of 7, and ReLu activation was used. Also, the flatten, dropout layer and output layer were used in the CNN. In this model for the CNN, the total parameters were 73, 283, the trainable parameters were 73, 283, and the no-trainable parameters were 0. An optimizer set to adam, loss set to sparse categorical cross-entropy, and metrics set to accuracy were used, and thirty rounds of epoch and a batch size of 128 were used.

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1d_3 (Conv1D)           (None, 14, 32)            256

 max_pooling1d_2 (MaxPooling  (None, 7, 32)            0
 1D)

 conv1d_4 (Conv1D)           (None, 7, 64)             14400

 max_pooling1d_3 (MaxPooling  (None, 3, 64)            0
 1D)

 conv1d_5 (Conv1D)           (None, 3, 128)            57472

 flatten_1 (Flatten)         (None, 384)               0

 dropout_1 (Dropout)         (None, 384)               0

 dense_1 (Dense)             (None, 3)                 1155

=================================================================
Total params: 73,283
Trainable params: 73,283
Non-trainable params: 0
```

Figure 96. Model Summary of Convolutional Neural Network (CNN) in the APA-DDoS Dataset.

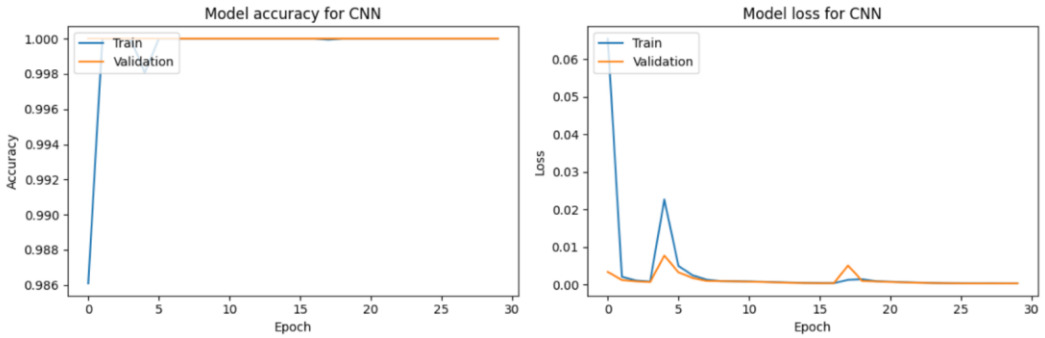


Figure 97. Model Accuracy and Loss of Convolutional Neural Network (CNN) in the APA-DDoS Dataset.

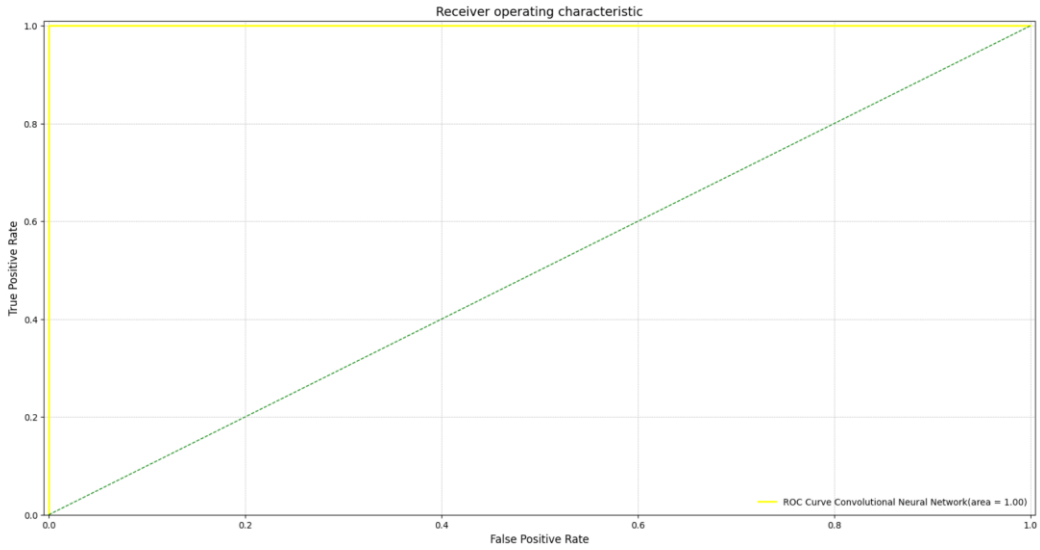


Figure 98. ROC-AUC Curve of Convolutional Neural Network (CNN) in the APA-DDoS Dataset.

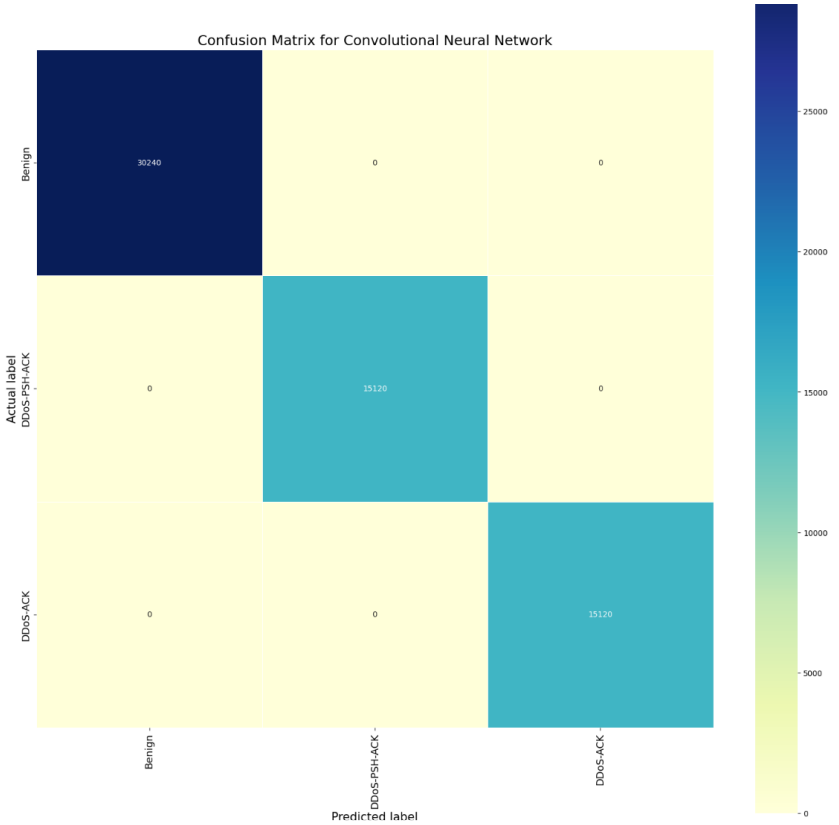


Figure 99. Multi-Class Classification Confusion Matrix of Convolutional Neural Network (CNN) in the APA-DDoS Dataset.

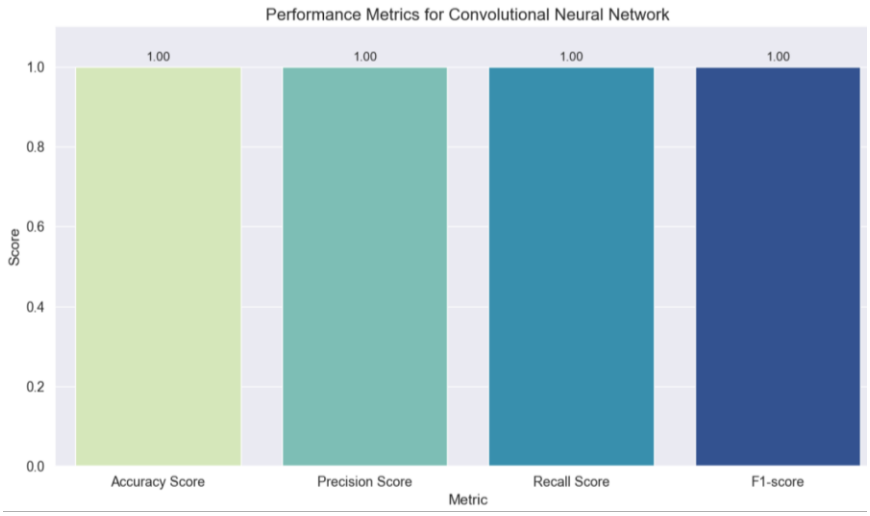


Figure 100. Convolutional Neural Networks with Performance Metrics and F1-Score in APA-DDoS Dataset.

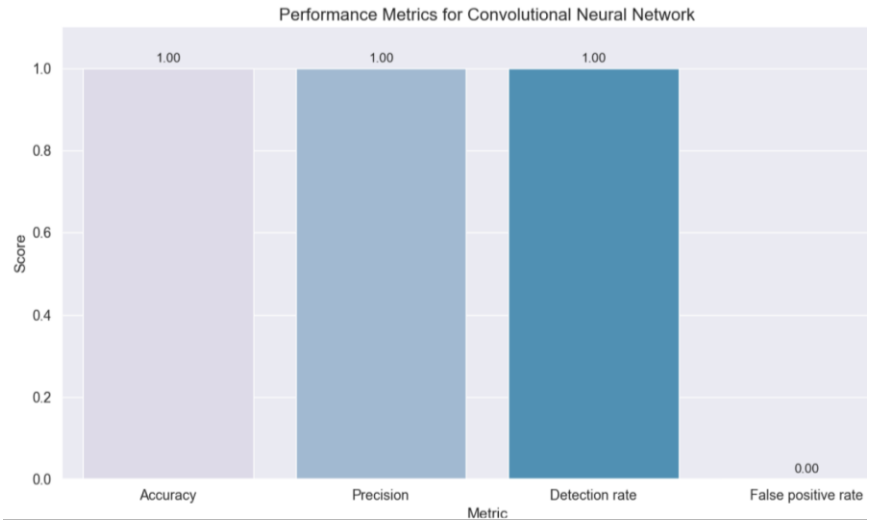


Figure 101. Convolutional Neural Networks with Performance Metrics and False Positive Rate in APA-DDoS Dataset.

In the CNN of our APA-DDoS dataset, our neural network has been overfitted again. Therefore, it has not only reached 100% accuracy and 1.0 in the ROC-AUC curve, but also it shows a perfect model that is unusual. Overfitting usually occurs when the model starts to learn the noise in the training data, and once that happens, our model becomes unpredictable.

Our second neural network in the APA-DDoS dataset, the Recurrent Neural Network, again performs overfitting. As we just saw in the Convolutional Neural Network, our neural network model had overfitted the same and has been applied to the Recurrent Neural Network. Generally, in Figure 102, the Recurrent Neural Network model summary, no matter what kernel size and activation filters were used. Still, it has overfitting, even if using the kernel regularly, it still overfitted. Therefore, in the below figures, Figures 103-107, our RNN model has been overfitted. We can also see in Figure 103 the model accuracy and loss, perfect model accuracy and loss with 100.0% accuracy

and 0.0 loss. However, in this RNN model, total params and trainable params were 91, 203 while non-trainable params were 0.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 simple_rnn (SimpleRNN)      (None, 256)               66048

 dense (Dense)               (None, 64)                16448

 dense_1 (Dense)             (None, 128)               8320

 flatten (Flatten)           (None, 128)               0

 dropout (Dropout)           (None, 128)               0

 dense_2 (Dense)             (None, 3)                 387

=================================================================
Total params: 91,203
Trainable params: 91,203
Non-trainable params: 0
_________________________________________________________________
```

Figure 102. Model Summary of Recurrent Neural Network (RNN) in the APA-DDoS Dataset.

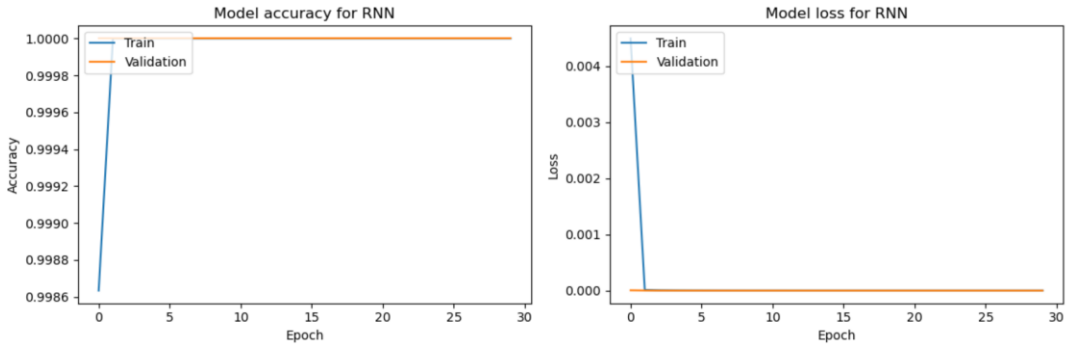


Figure 103. Model Accuracy and Loss of Recurrent Neural Network (RNN) in the APA-DDoS Dataset.

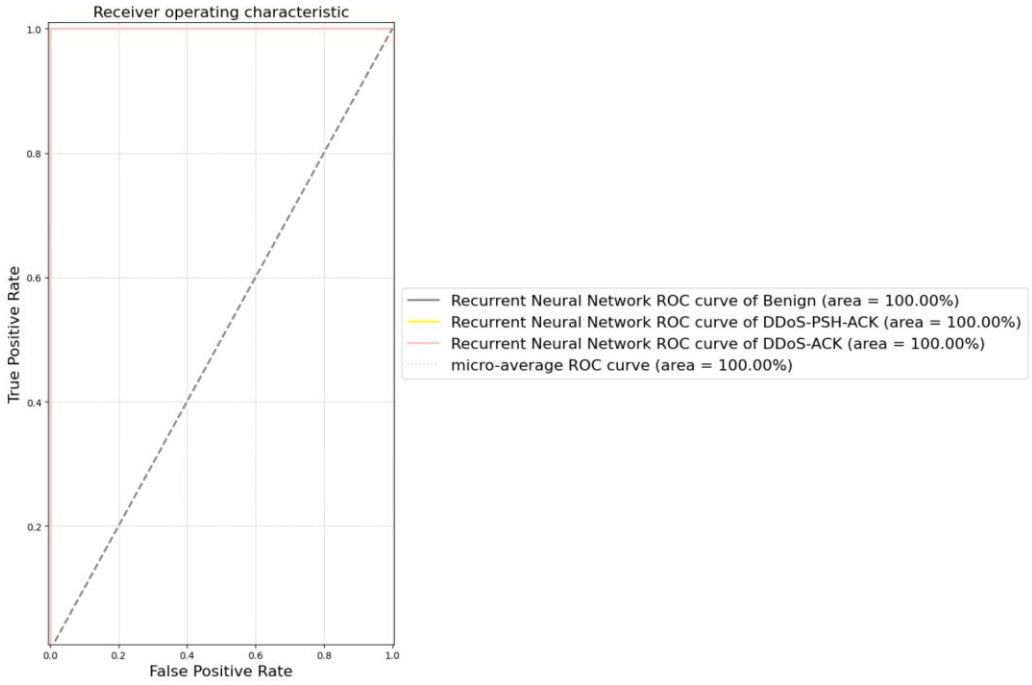


Figure 104. ROC-AUC Curve of Recurrent Neural Network (RNN) in the APA-DDoS Dataset.

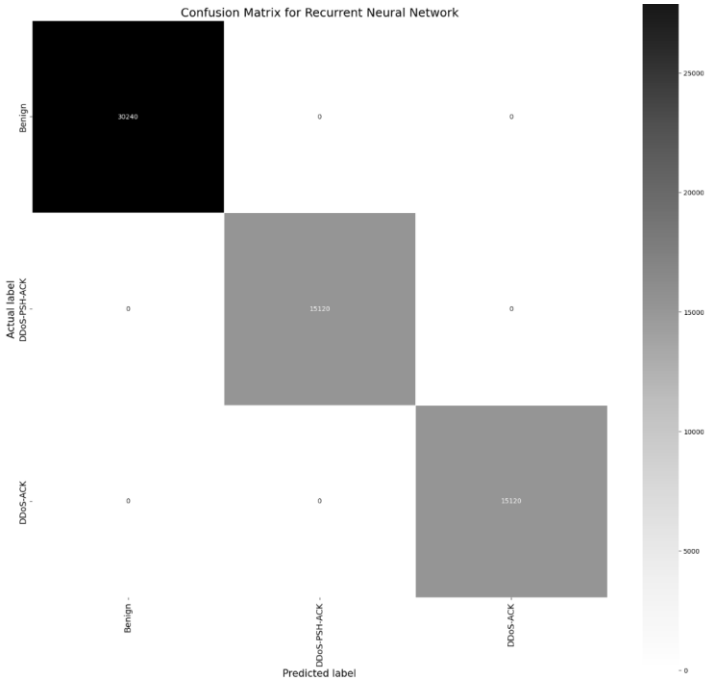


Figure 105. Confusion Matrix of Recurrent Neural Network (CNN) in the APA-DDoS Dataset.

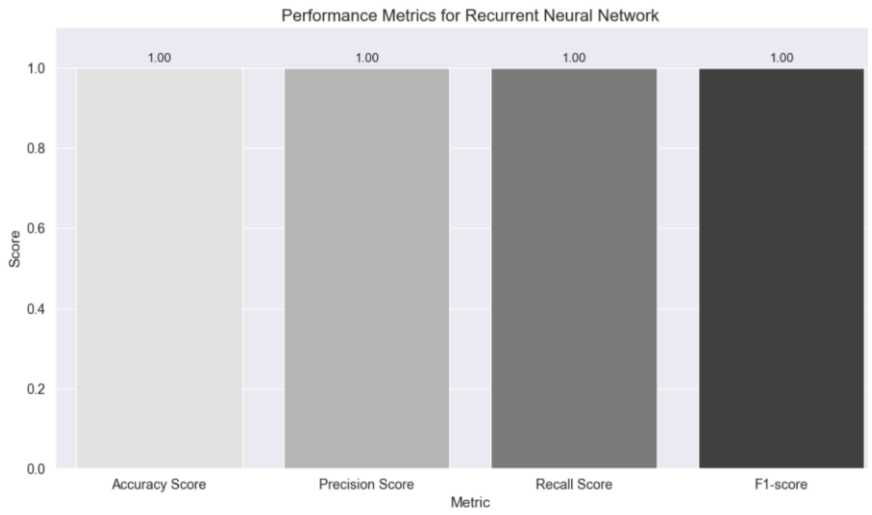


Figure 106. Recurrent Neural Networks with Performance Metrics and F1-Score in APA-DDoS Dataset.

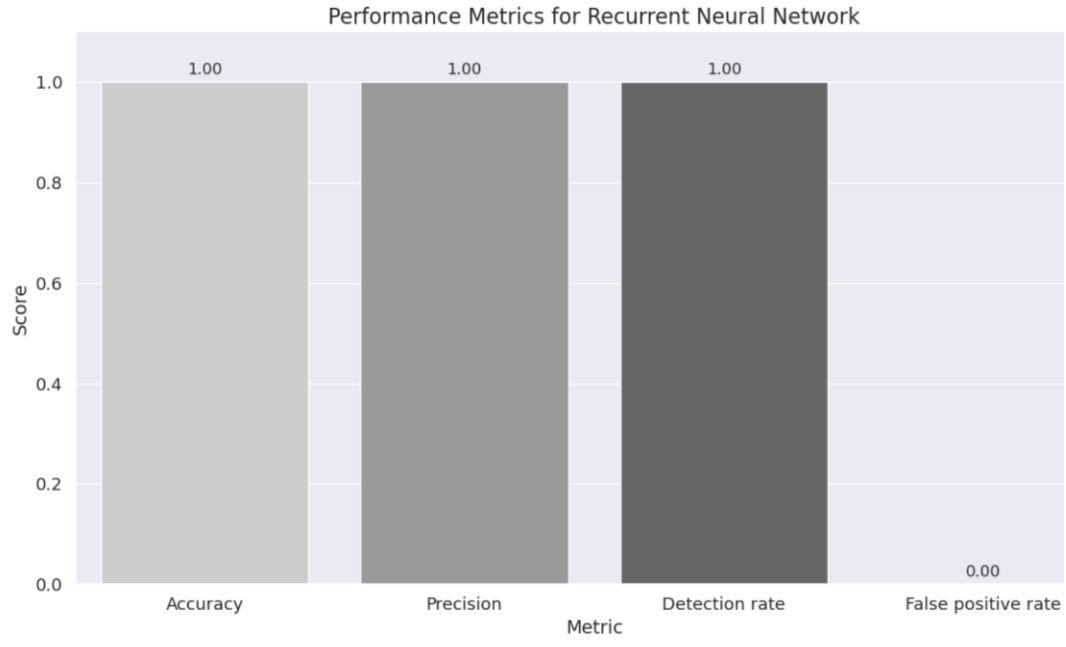


Figure 107. Recurrent Neural Networks with Performance Metrics and False Positive Rate in APA-DDoS Dataset.

The last neural network model, the Long-Short Term Memory, the model did not overfit. In this LSTM model for the APA-DDoS dataset, the total params and trainable params were 18, 894, 851, and non-trainable params were 0. Therefore, eighteen million params were used and dense, dropout, and dense again. The type is LSTM because we are using the Long-Short Term Memory. We used an activation of tanh for our LSTM model and a kernel regularizer. Thirty epochs were tested with a batch size of 128. In Figure 109, our model accuracy is 0.5, and our loss is 1.0520. In addition, our confusion matrix shows all 0s, and one column, the benign column, was represented with numbers showing that our model might not have performed commonly. Therefore, this might have affected the performance of our LSTM model. In Figure 110, our ROC-AUC score was 62.5%. However, it's important to note that in our last figure, Figure 113, the false positive rate is 33.33%, meaning that this prediction is not good as it is nonattack traffic.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 2048)              16793600

 dense (Dense)               (None, 1024)              2098176

 dropout (Dropout)           (None, 1024)              0

 dense_1 (Dense)             (None, 3)                 3075

=================================================================
Total params: 18,894,851
Trainable params: 18,894,851
Non-trainable params: 0
_________________________________________________________________
```

Figure 108. Model Summary of Long Short-Term Memory (LSTM) in the APA-DDoS Dataset.

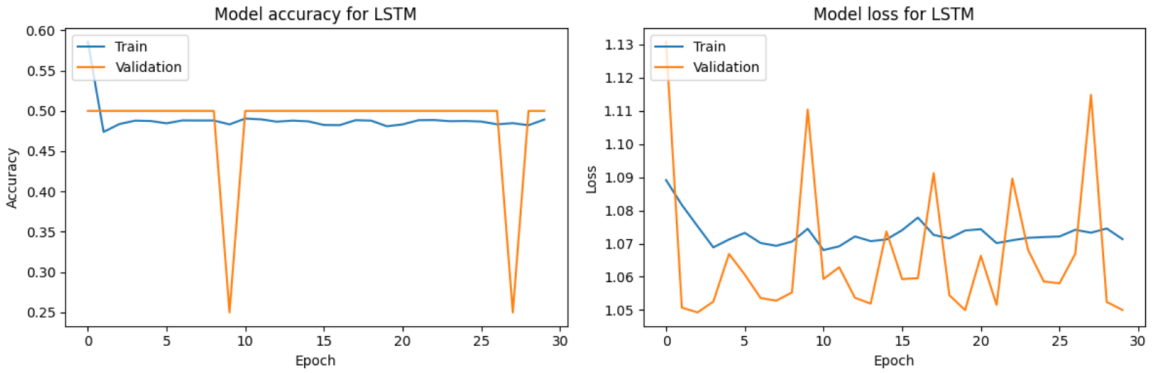


Figure 109. Long Short-Term Memory Model Accuracy and Loss in APA-DDoS Dataset.

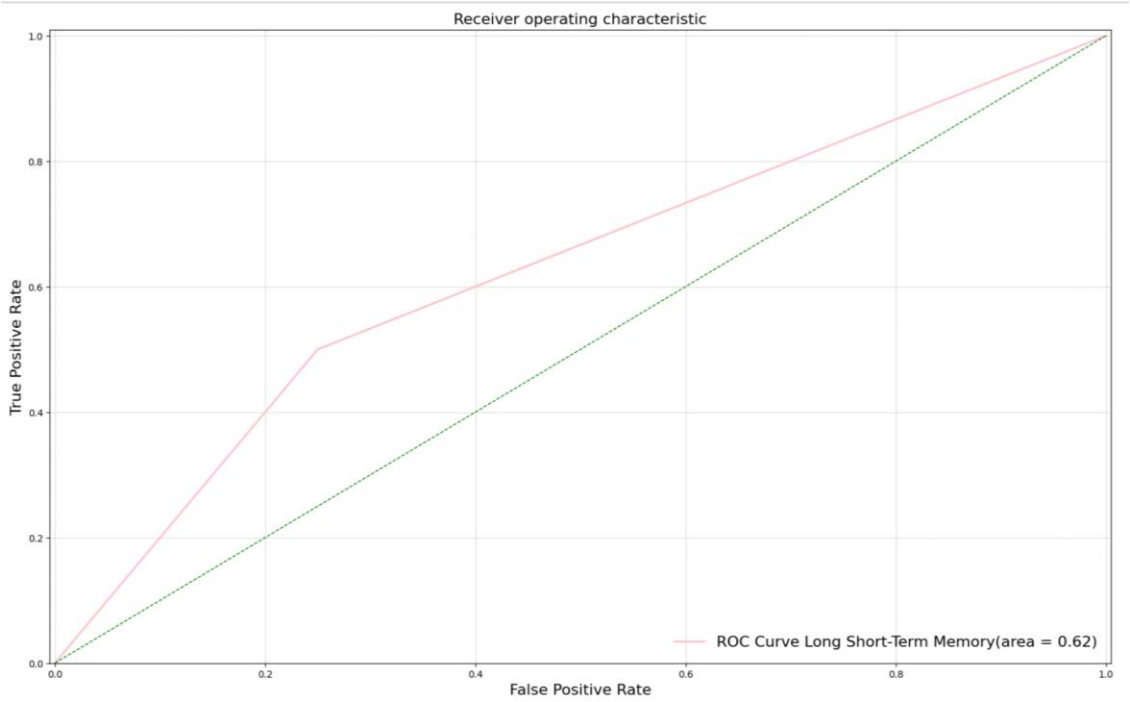


Figure 110. ROC-AUC Score Long Short-Term Memory in APA-DDoS Dataset.

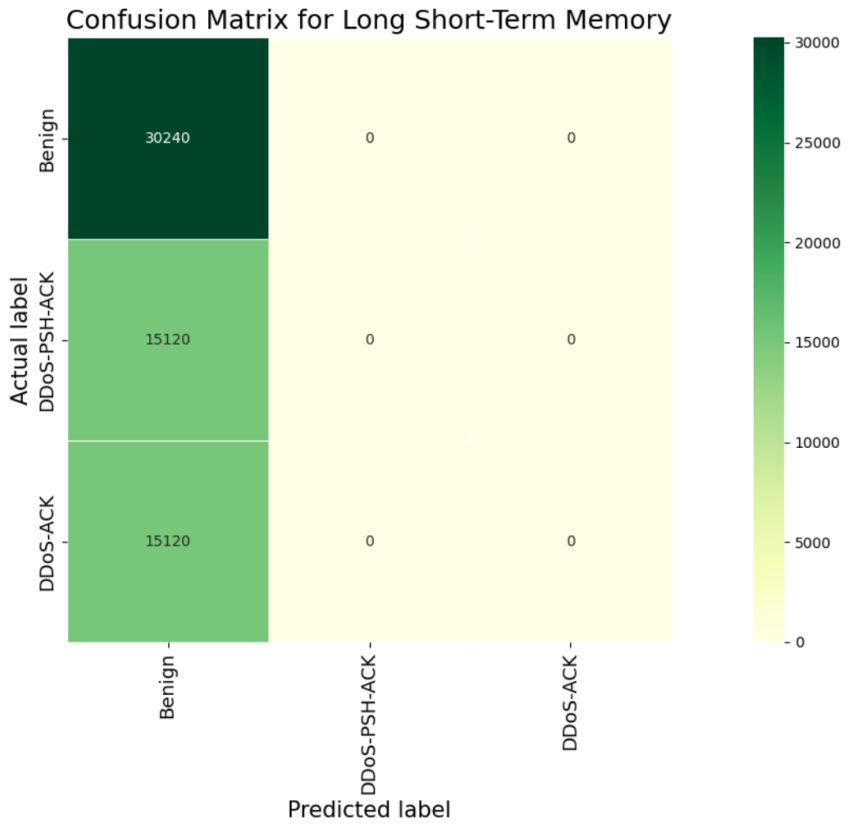


Figure 111. Confusion Matrix Long Short-Term Memory in APA-DDoS Dataset.

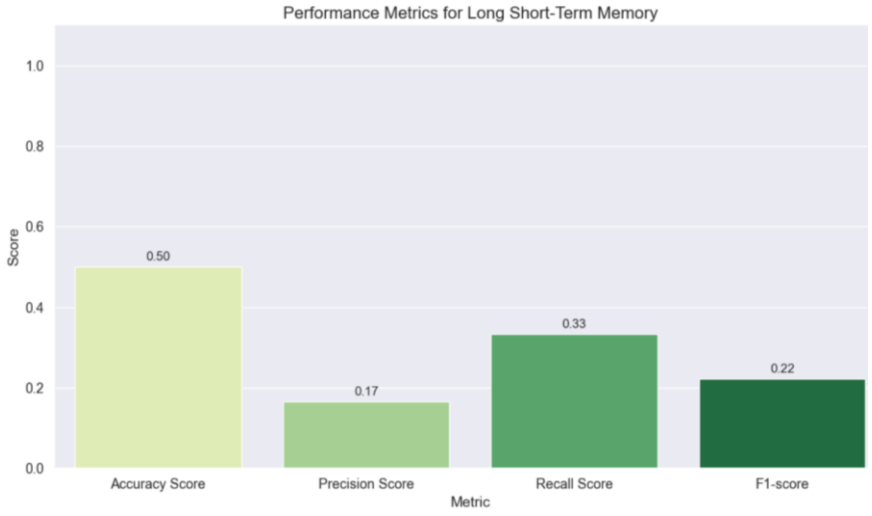


Figure 112. Long Short-Term Memory with Performance Metrics and F-1Score in APA-DDoS Dataset.

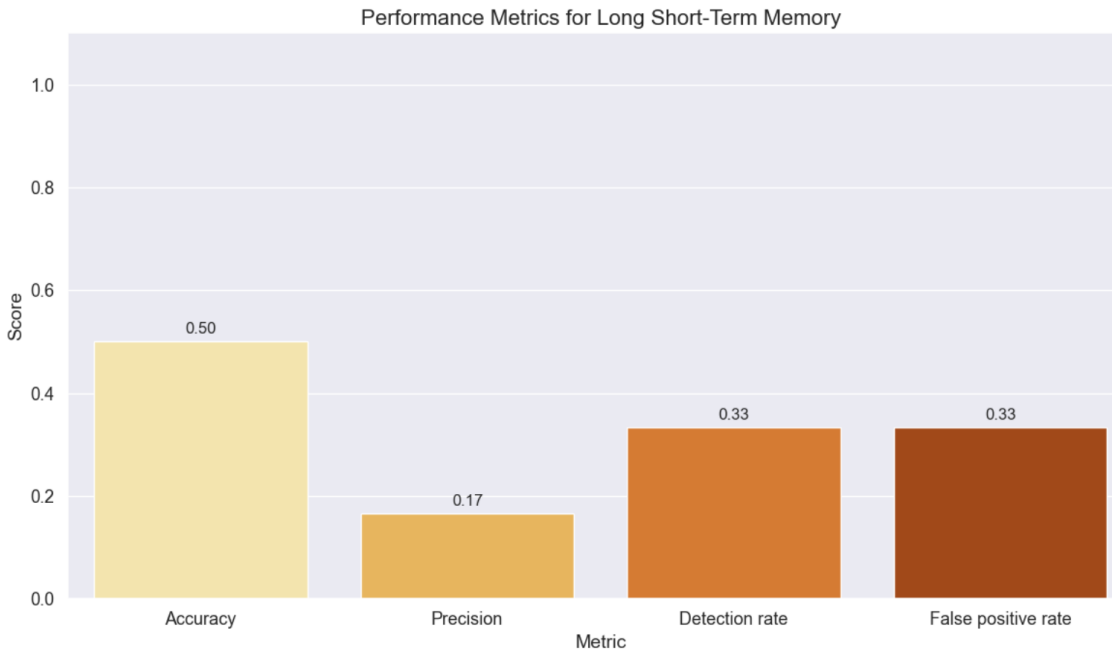


Figure 113. Long Short-Term Memory with Performance Metrics and False Positive Rate in APA-DDoS Dataset.

CHAPTER 6

Conclusion and Future Work

Chapter 6 will finalize the conclusion, the results, and future work.

The first dataset, the CIC-DDoS2019 dataset, the Random Forest, performed the best with 93.50% accuracy, the Convolutional Neural Network at 93.10%, and the lowest Recurrent Neural Network at 23.10%. The highest Area Under the Curve (AUC) were CatBoost, Random Forest, and Convolutional Neural Network with 99.8%, and the lowest Area Under the Curve (AUC was Recurrent Neural Network with 69.10%.

In the second dataset, the APA-DDoS, the Decision Tree classifier performed the best with 97.64% accuracy. Long Short-Term Memory had the lowest accuracy with 50.00% in the APA-DDoS dataset. For the AUC, the highest AUC was 99.61% with CatBoost, and the lowest AUC was Long Short Term-Memory.

Each machine learning algorithm led to different accuracy, leading to high and low accuracies based on the machine learning tools and, specifically, their hypermeters. In the Area Under the Curve of an ROC (Receiver Operating Characteristic) curve, the lines mostly led to a Truer Positive Rate and overfitting in some Machine Learning algorithms. Lastly, in the confusion matrix we saw many differences in the multi-class classification and overfitting in some confusion matrix that also led to perfect confusion matrix such as True Positives only.

Our future work will be to test and train more Machine Learning algorithms for CIC-DDoS2019 and APA-DDoS datasets. In addition, we plan to use Temporal

Convolutional Networks and Generative Adversarial Networks to test and train our model with these machine learning algorithms. Also, we plan to find any Machine Learning techniques to reduce overfitting in the APA-DDoS dataset and improve accuracy in the CIC-DDoS2019 dataset. Lastly, we will use binary classification for both datasets to train and test the data.

REFERENCES

[1] E. Skoudis and T. Liston, in *Counter Hack Reloaded: A Step-by-Step Guide to Computer Attacks and Effective Defenses 2nd Edition*, Upper Saddle River, New Jersey: Prentice Hall, 2005, pp. 50, 52, 534, 523.

[2] S.-P. Oriyano, in *CEH V9: Certified Ethical Hacker Version 9*, Alameda, California: Sybex, 2016, pp. 53, 170, 171, 182, 184, 318, 390.

[3] M. Jakobsson and Z. Ramzan, in *Crimeware: Understanding New Attacks and Defenses*, Addison-Wesley, 2008, p. 378.

[4] M. Kapko, "DDoS attack traffic surged in 2023, Cloudflare finds," 9 January 2024. [Online]. Available: https://www.cybersecuritydive.com/news/ddos-attacks-surge-cloudflare/704011/#:~:text=%E2%80%9COn%20average%2C%20our%20systems%20auto,25%25%20quarter%20over%20quarter.%E2%80%9D. [Accessed 30 March 2024].

[5] "Is It Legal To Perform DDoS Attack Tests?," [Online]. Available: https://www.nimbusddos.com/blog-article-is-it-legal-to-perform-ddos-attack-tests.htm#:~:text=The%20Law-,DDoS%20attacks%20are%20illegal.,to%20attacks%20launched%20without%20permission.. [Accessed 30 March 2024].

[6] C. Taylor, "Distributed Denial of Service (DDoS)," 26 December 2019. [Online]. Available: https://cyberhoot.com/cybrary/distributed-denial-of-service/. [Accessed 1 April 2024].

[7] NSFOCUS, "Why IPS and Firewalls Are Not Anti-DDoS Solutions?," 23 March 2023. [Online]. Available: https://nsfocusglobal.com/why-ips-and-firewalls-are-not-anti-ddos-solutions/#:~:text=Firewalls%20and%20IPS%20Cannot%20Distinguish,threat%20by%20firewalls%20and%20IPS.. [Accessed 30 March 2024].

[8] "Famous DDoS attacks | The largest DDoS attacks of all time," [Online]. Available: https://www.cloudflare.com/learning/ddos/famous-ddos-attacks/. [Accessed 30 March 2024].

[9] E. Carter, "Cisco Secure Intrusion Detection System," Indianapolis, IN, Cisco Press, 2001, pp. 27, 453.

[10] K. Baker, "The 12 Most Common Types of Malware," CrowdStrike, 28 February 2023. [Online]. Available: https://www.crowdstrike.com/cybersecurity-101/malware/types-of-malware/. [Accessed 7 May 2024].

[11] "What is cloud account hijacking?," Polymer, 28 June 2022. [Online]. Available: https://www.polymerhq.io/blog/insider-threat/what-is-cloud-account-hijacking/. [Accessed 7 May 2024].

[12] D. Walkowski, "What Is a Distributed Denial-of-Service Attack?," 5 June 2019. [Online]. Available: https://www.f5.com/labs/learning-center/what-is-a-distributed-denial-of-service-attack. [Accessed 1 April 2024].

[13] "Understanding Denial-of-Service Attacks," Cybersecurity and Infrastructure Security Agency, 01 February 2021. [Online]. Available: https://www.cisa.gov/news-events/news/understanding-denial-service-attacks. [Accessed 1 April 2024].

[14] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," Gaithersburg, U.S. Department of Commerce, Technology Administration, National Institute of Standards and Technology, 2007, pp. 2-1.

[15] "Sklearn Decision Tree | Decision Tree Classifier in Python," [Online]. Available: https://aihints.com/sklearn-decision-tree/. [Accessed 3 April 2024].

[16] D. Campbell, "Nonparametric Statistics. In: Volkmar, F.R. (eds) Encyclopedia of Autism Spectrum Disorders," New York, NY, Springer, 2013, p. 2035.

[17] J. Kozak, "Decision Tree and Ensemble Learning Based on Ant Colony Optimization," New York City, NY, Springer International Publishing, 2018, p. 3.

[18] I. Reeves, "Cybersecurity Game Theory in Incident Response," 1 September 2022. [Online]. Available: https://fractionalciso.com/cybersecurity-game-theory-in-incident-response/. [Accessed 2 April 2024].

[19] A. Tarasov and S. J. Delany, "Benchmarking classification models for emotion recognition in natural speech: A multi-corporal study," Santa Barbara, CA, USA, IEEE, 2011, pp. 841-846.

[20] A. Chakure, "What Is Decision Tree Classification?," 10 February 20222. [Online]. Available: https://builtin.com/data-science/classification-tree. [Accessed 2 April 2024].

[21] J. Markey, "Using Decision Tree Analysis for Intrusion Detection: A How-To Guide," SANS, 2011, p. 1; 5.

[22] "CatBoost," [Online]. Available: https://catboost.ai/en/docs/. [Accessed 3 April 2024].

[23] A. Oppermann, "What Is CatBoost?," 6 April 2023. [Online]. Available: https://builtin.com/machine-learning/catboost. [Accessed 3 April 2024].

[24] V. Ershov, "CatBoost Enables Fast Gradient Boosting on Decision Trees Using GPUs," 18 December 2018. [Online]. Available: https://catboost.ai/news/catboost-enables-fast-gradient-boosting-on-decision-trees-using-gpus. [Accessed 3 April 2024].

[25] L. Breiman, "RANDOM FORESTS," January 2001. [Online]. Available: https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf. [Accessed 4 April 2024].

[26] R. Curry, "The Complete Guide to Random Forests: Part 1," 16 December 2021. [Online]. Available: https://medium.com/@curryrowan/the-complete-guide-to-random-forests-part-1-427c9cba8336. [Accessed 4 April 2024].

[27] B. E. Boser, I. M. Guyon and V. N. Vapnik, "A training algorithm for optimal margin classifiers," in *D. Haussler, editor, Proceedings of the 5th Annual ACM Workshop on Computational Learning*, Pittsburgh, PA, ACM Press, 1992, pp. 144-152.

[28] B. Schlkopf and A. J. Smola, "Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond," Cambridge, MA, The MIT Press, 2001.

[29] X. Wu and V. Kumar, in *The Top Ten Algorithms in Data Mining*, Chapman and Hall/CRC, 2009, p. 37.

[30] C. Smith, "What is SVM model?," 27 December 2020. [Online]. Available: https://missionalcall.com/2020/12/27/what-is-svm-model/. [Accessed 11 April 2024].

[31] K. Ghanem, F. J. Aparicio-Navarro, K. G. Kyriakopoulos, S. Lambotharan and J. A. Chambers, "Support Vector Machine for Network Intrusion and Cyber-Attack Detection," 2017 . [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8233268. [Accessed 11 April 2024].

[32] J. Zornoza, "Complexity and Big O Notation: The Ultimate Guide," 14 December 2020. [Online]. Available: https://aigents.co/data-science-blog/publication/complexity-and-big-o-notation-the-ultimate-guide. [Accessed 11 April 2024].

[33] Datasciencelovers, "Support Vector machine-Theory," [Online]. Available: http://www.datasciencelovers.com/machine-learning/support-vector-machine-theory/. [Accessed 11 April 2024].

[34] J. Robert, "What is the KNN algorithm?," 19 November 2020. [Online]. Available: https://datascientest.com/knn. [Accessed 11 April 2024].

[35] Y. L. Cun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel, "Handwritten Digit Recognition with a Back-Propagation Network," Holmdel, N. J., AT&T Bell Laboratories, 1989 .

[36] R. Yamashita, M. Nishio, R. K. G. Do and K. Togashi, "Convolutional neural networks: an overview and application in radiology," 22 June 2018. [Online]. Available: https://doi.org/10.1007/s13244-018-0639-9. [Accessed 14 April 2024].

[37] A. S. Gillis, E. Burns and K. Brush, "deep learning," TechTarget, [Online]. Available: https://www.techtarget.com/searchenterpriseai/definition/deep-learning-deep-neural-network. [Accessed 14 April 2024].

[38] M. Memon, "ANN vs CNN vs RNN: Neural Networks Guide," Levity, 16 November 2022. [Online]. Available: https://levity.ai/blog/neural-networks-cnn-ann-rnn. [Accessed 14 April 2024].

[39] I. G. Wilson and M. E. Wilson, "What computers cannot do," Vertex, 1970, pp. 9-10.

[40] Tech and Tales, "Neural Networks 101: A Step-by-Step Guide to Getting Started," Medium, 13 October 2023. [Online]. Available: https://techntales.medium.com/neural-networks-101-a-step-by-step-guide-to-getting-started-1c66f6b2346b. [Accessed 14 April 2024].

[41] Z. Yi and K. Tan, "Convergence Analysis of Recurrent Neural Networks," New York, NY, Springer, 2004, p. 1.

[42] N. Donges and M. Urwin, "A Complete Guide to Recurrent Neural Networks (RNNs)," Built In, 28 February 2024. [Online]. Available: https://builtin.com/data-science/recurrent-neural-networks-and-lstm. [Accessed 14 April 2024].

[43] A. Graves, "Supervised Sequence Labelling with Recurrent Neural Networks," Berlin, Springer-Verlag Berlin Heidelberg, 2012, pp. 22, 25.

[44] J. Brownlee, "Long Short-Term Memory Networks With Python: Develop Sequence Prediction Models with Deep Learning," Jason Brownlee, 2017, p. 10.

[45] C. Olah, "Understanding LSTM Networks," GitHub, 27 August 2015. [Online]. Available: https://colah.github.io/posts/2015-08-Understanding-LSTMs/. [Accessed 16 April 2024].

[46] "1.1.1. What is EDA?," NIST, [Online]. Available: https://www.itl.nist.gov/div898/handbook/eda/section1/eda11.htm. [Accessed 5 April 2024].

[47] Z. Bobbitt, "How to Read a Correlation Matrix," 27 January 2020. [Online]. Available: https://www.statology.org/how-to-read-a-correlation-matrix/. [Accessed 12 April 2024].

[48] "Representation: Feature Engineering," Google , [Online]. Available: https://developers.google.com/machine-learning/crash-course/representation/feature-engineering. [Accessed 17 April 2024].

[49] J. Frost, "Principal Component Analysis Guide & Example," Statistics By Jim, [Online]. Available: https://statisticsbyjim.com/basics/principal-component-analysis/. [Accessed 18 April 2024].

[50] G. Tanner, "Linear Discriminant Analysis (LDA)," 9 November 2021. [Online]. Available: https://ml-explained.com/blog/linear-discriminant-analysis-explained. [Accessed 18 April 2024].

[51] "RobustScaler," Scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html#sklearn.preprocessing.RobustScaler. [Accessed 18 April 2024].

[52] "MinMaxScaler," Scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html. [Accessed 18 April 2024].

[53] M. Eland, "Interpreting Confusion Matrixes," Accessible AI, 2 July 2022. [Online]. Available: https://accessibleai.dev/post/interpreting_confusion_matrixes/. [Accessed 18 April 2024].

[54] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," Research Gate, March 2015. [Online]. Available: https://www.researchgate.net/publication/273155496_The_Precision-Recall_Plot_Is_More_Informative_than_the_ROC_Plot_When_Evaluating_Binary_Classifiers_on_Imbalanced_Datasets. [Accessed 6 May 2024].

[55] "Classification: ROC Curve and AUC," Google, [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc. [Accessed 6 May 2024].

[56] R. Draelos, "Measuring Performance: AUC (AUROC)," Glass Box, 23 February 2019. [Online]. Available: https://glassboxmedicine.com/2019/02/23/measuring-performance-auc-auroc/. [Accessed 6 May 2024].

[57] "What is a Reflection Amplification Attack?," Net Scout, [Online]. Available: https://www.netscout.com/what-is-ddos/what-is-reflection-amplification-attack. [Accessed 23 April 2024].

[58] I. Sharafaldin, A. H. Lashkari, S. Hakak and A. A. Ghorbani, "DDoS evaluation dataset (CIC-DDoS2019)," 2019. [Online]. Available: https://www.unb.ca/cic/datasets/ddos-2019.html. [Accessed 23 April 2024].

[59] "Applications - CICFlowMeter (formerly ISCXFlowMeter)," [Online]. Available: https://www.unb.ca/cic/research/applications.html. [Accessed 2024 April 2024].

[60] "What is a SSDP DDoS Attack?," Cloudflare, [Online]. Available: https://www.cloudflare.com/learning/ddos/ssdp-ddos-attack/. [Accessed 28 April 2024].

[61] T. Lammle, "CCNA: Cisco Certified Network Associate Study Guide, 5th Edition," Sybex, 2005, pp. 612, 641, 642, 657, 658.

[62] "14 Working with Flow Identifiers," Oracle, . [Online]. Available: https://docs.oracle.com/en/industries/communications/uim/7.5.1/uim-help/working-flow-identifiers1.html#GUID-3B34D4F3-4F5A-4905-9BC8-B8C9D2DE0D96. [Accessed 27 April 2024].

[63] G. D. Gil, A. H. Lashkari, M. Mamun and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic Using Time-Related Features," February 2016. [Online]. Available: DOI: 10.5220/0005740704070414. [Accessed 27 April 2024].

[64] A. H. Lashkari, "CICFlowMeter /ReadMe.txt," GitHub, 7 June 2024. [Online]. Available: https://github.com/ahlashkari/CICFlowMeter/blob/master/ReadMe.txt. [Accessed 27 April 2024].

[65] M. Cobb and K. Beaver, "The differences between inbound and outbound firewall rules," TechTarget, 7 March 2024. [Online]. Available:

https://www.techtarget.com/searchsecurity/answer/Comparing-firewalls-Differences-between-an-inbound-outbound-firewall. [Accessed 27 April 2024].

[66] S. Vluymans, "Dealing with Imbalanced and Weakly Labelled Data in Machine Learning using Fuzzy and Rough Set Methods," Springer, 2018, pp. 81, 84.

[67] A. Fernández, S. García, M. Galar, R. C. Prati, B. Krawczyk and F. Herrera, "Learning from Imbalanced Data Sets," Springer, 2018, p. 37.

[68] "sklearn.tree.DecisionTreeClassifier," scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html. [Accessed 9 May 2024].

[69] "3.1. Cross-validation: evaluating estimator performance," scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html. [Accessed 14 May 2024].

[70] "Speeding up the training," CatBoost, [Online]. Available: https://catboost.ai/en/docs/concepts/speed-up-training. [Accessed 2024 May 14].

[71] "Parameter tuning," CatBoost, [Online]. Available: https://catboost.ai/en/docs/concepts/parameter-tuning. [Accessed 14 May 2024].

[72] "CatBoostClassifier," CatBoost, [Online]. Available: https://catboost.ai/en/docs/concepts/python-reference_catboostclassifier. [Accessed 14 May 2024].

[73] "Implemented metrics," CatBoost, [Online]. Available: https://catboost.ai/en/docs/features/loss-functions-desc. [Accessed 14 May 2024].

[74] "sklearn.svm.SVC," scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC. [Accessed 14 May 2014].

[75] "sklearn.neighbors.KNeighborsClassifier," scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html. [Accessed 14 May 2024].

[76] "sklearn.ensemble.RandomForestClassifier," scikit-learn, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html. [Accessed 15 May 2024].

[77] Y. R. Kumbam, "APA-DDoS Dataset," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/yashwanthkumbam/apaddos-dataset. [Accessed 14 May 2024].

[78] "Display Filter Reference: Internet Protocol Version 4," Wireshark, [Online]. Available: https://www.wireshark.org/docs/dfref/i/ip.html. [Accessed 14 May 2024].

[79] "Display Filter Reference: Transmission Control Protocol," Wireshark, [Online]. Available: https://www.wireshark.org/docs/dfref/t/tcp.html. [Accessed 14 May 2024].

[80] "Display Filter Reference: Frame," Wireshark, [Online]. Available: https://www.wireshark.org/docs/dfref/f/frame.html. [Accessed 14 May 2024].